



ใบอนุญาตโครงการปริญญาโท
สาขาวิชาเทคโนโลยีสารสนเทศ ภาควิชาคอมพิวเตอร์
คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

ชื่อปริญญาโท แอปพลิเคชันติดตาม deck ด้วยการสื่อสารระยะใกล้
Deck tracking application with near field communication

ผู้จัดทำ นาย วิจักขณ์มา ห่องทองแดง

ปีการศึกษา 2567

โครงการปริญญาโทนี้ได้รับการอนุมัติให้เป็นส่วนหนึ่งของการศึกษาตาม
หลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชาเทคโนโลยีสารสนเทศ
ภาควิชาคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

..... ประธานกรรมการสอบ
(ผู้ช่วยศาสตราจารย์ ดร.ณัฐโชติ พรหมฤทธิ์)

..... กรรมการสอบ
(อาจารย์ ดร.สัจจาภรณ์ ไวจรรยา)

..... อาจารย์ที่ปรึกษาและกรรมการสอบ
(อาจารย์ อภิเชก หงษ์วิทยากร)



แอปพลิเคชันติดตามdeckด้วยการสื่อสารระยะใกล้

Deck tracking application with near field communication

วิจักขณ์สมา ห่องทองแดง

**ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาในหลักสูตรวิทยาศาสตรบัณฑิต
สาขาวิชาเทคโนโลยีสารสนเทศ ภาควิชาคอมพิวเตอร์
คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร
ปีการศึกษา 2567**

ชื่อปริญญาโท
ชื่อปริญญาตรี
ชื่อปริญญาโท
ชื่อปริญญาตรี
ชื่อปริญญาโท
ชื่อปริญญาตรี
ชื่อปริญญาโท
ชื่อปริญญาตรี
ชื่อปริญญาโท
ชื่อปริญญาตรี

แอปพลิเคชันติดตามเด็กด้วยการสื่อสารระยะใกล้
Deck tracking application with near field communication
นาย วิจักขณ์มา ห่องทองแดง
อาจารย์ อภิเชก หงษ์วิทยากร
วิทยาศาสตร์บัณฑิต (เทคโนโลยีสารสนเทศ)
2567

บทคัดย่อ

ในโลกของเกมการ์ด กลยุทธ์และความแม่นยำคือหัวใจสำคัญของชัยชนะ ผู้เล่นจำเป็นต้องจัดการ "เด็ก" หรือชุดการ์ดอย่างรอบคอบ และติดตามสถานะการใช้งานระหว่างเกมอย่างใกล้ชิด อย่างไรก็ตาม การติดตามด้วยความจำเพียงอย่างเดียวมักก่อให้เกิดข้อผิดพลาดและส่งผลกระทบต่อประสิทธิภาพการเล่น

โครงการนี้จึงนำเสนอ NFC Deck Tracker แอปพลิเคชันที่โดดเด่นด้วยการนำเทคโนโลยี NFC (Near Field Communication) มาใช้ในการจัดการและติดตามเด็กการ์ดแบบเรียลไทม์ ซึ่งแตกต่างจากแอปพลิเคชันในตลาดที่ยังไม่มีการผสานเทคโนโลยี NFC เข้าไว้ด้วยกัน ผู้เล่นสามารถบันทึก ตรวจสอบ และวิเคราะห์ข้อมูลการ์ดได้อย่างแม่นยำและรวดเร็ว เพียงแค่แตะการ์ดกับอุปกรณ์สมาร์ทโฟน พร้อมทั้งมีอินเทอร์เฟซที่ใช้งานง่าย รองรับการใช้งานทั้งในโหมดออนไลน์และออฟไลน์

ไม่เพียงแต่เพิ่มความสะดวกในการจัดการเด็กและติดตามสถานะการเล่น แอปพลิเคชันนี้ยังช่วยให้ผู้เล่นสามารถวางแผนกลยุทธ์ได้ลึกซึ้งยิ่งขึ้น วิเคราะห์สถิติการใช้งานของการ์ดแต่ละใบ หรือแม้แต่แบ่งปันข้อมูลกับเพื่อนและชุมชนได้อย่างสะดวกและมีประสิทธิภาพมากยิ่งขึ้น ด้วยเทคโนโลยี NFC ที่เป็นเอกลักษณ์นี้ผู้พัฒนาหวังว่า NFC Deck Tracker เป็นจุดเริ่มต้นในการยกระดับประสบการณ์การเล่นเกมการ์ดในยุคดิจิทัลและเปิดทางสู่การประยุกต์ใช้งานที่กว้างขวางยิ่งขึ้นในอนาคต

คำสำคัญ : การซิงค์ข้อมูลแบบเรียลไทม์ การติดตามเด็ก การ์ดเกม สถาปัตยกรรม Clean, NFC, UX/UI

Keyword : Clean Architecture, Deck Tracking, NFC, Real-time Synced, Trading Card Game, UX/UI

กิตติกรรมประกาศ

ข้าพเจ้า นายวิจักขณ์สมา ห่องทองแดง ขอแสดงความขอบคุณ อาจารย์อภิเชก หงษ์วิทยากร อาจารย์ที่ปรึกษาสำหรับคำแนะนำ แนวทางการพัฒนา และการสนับสนุนอย่างต่อเนื่องตลอดระยะเวลาของการจัดทำปริญญานิพนธ์ฉบับนี้ ขอขอบคุณเพื่อน ๆ ทุกคนที่คอยให้กำลังใจช่วยเหลือและแลกเปลี่ยนความคิดเห็นตลอดกระบวนการทำงาน ขอขอบคุณครอบครัวที่คอยเป็นกำลังใจและสนับสนุนข้าพเจ้าเสมอมา ท้ายที่สุดนี้ข้าพเจ้าขอขอบคุณตัวเองสำหรับความพยายาม ความอดทน และความมุ่งมั่นที่ทำให้โครงการนี้สำเร็จลุล่วงลงได้ด้วยดี

วิจักขณ์สมา ห่องทองแดง

สารบัญ

	หน้า
บทคัดย่อ	ก
กิตติกรรมประกาศ	ข
สารบัญ	ค
สารบัญตาราง	ฅ
สารบัญรูป	ช
บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญของปัญหา	1
1.2 วัตถุประสงค์	2
1.3 ลักษณะและขอบเขต	2
1.3.1 คุณสมบัติทางเทคนิค	2
1.3.2 แอปพลิเคชันพีเจอาร์	2
1.4 อุปกรณ์และเครื่องมือที่ใช้	3
1.4.1 ฮาร์ดแวร์	3
1.4.2 ซอฟต์แวร์	3
1.5 ผลที่คาดว่าจะได้รับ	3
บทที่ 2 ผลงานที่เกี่ยวข้อง	4
2.1 Big AR Vanguard	4
2.2 Hearthstone Deck Tracker	5
2.3 NFC Tools	6
2.4 สรุปผลงานที่เกี่ยวข้อง	7
บทที่ 3 ทฤษฎีและความรู้ที่เกี่ยวข้อง	8
3.1 TCG (Trading Card Game)	8
3.2 NFC (Near Field Communication)	9
3.3 Flutter	10
3.4 Clean Architecture	11
3.5 Firebase	12
3.6 SQLite	13
บทที่ 4 ขั้นตอนและแผนการดำเนินงาน	14
4.1 ขั้นตอนการดำเนินงาน	14
4.1.1 ศึกษาข้อมูลและวิเคราะห์ความต้องการ	14
4.1.2 กำหนดขอบเขตของโครงการ	14
4.1.3 ศึกษาผลงานและทฤษฎีที่เกี่ยวข้อง	14
4.1.4 วิเคราะห์และออกแบบระบบ	14

4.1.5 พัฒนาระบบ	14
4.1.6 ทดสอบและปรับปรุงการทำงานของระบบ	14
4.1.7 จัดทำเอกสารโครงการ	14
4.2 แผนการดำเนินงาน	15
บทที่ 5 วิธีการดำเนินงาน	16
5.1 การออกแบบสถาปัตยกรรมระบบ	16
5.2 การออกแบบสถาปัตยกรรมซอฟต์แวร์	17
5.3 ตัวอย่างการทำงานของเลเยอร์ผ่านพีเจอาร์การตั้งค่า	19
5.3.1 การจัดการ State ด้วย Bloc	19
5.3.2 การใช้งาน Usecase	20
5.3.3 การเข้าถึงข้อมูลผ่าน Repository	20
5.3.4 การจัดการข้อมูลใน Datasource	21
5.4 การจัดการ Dependency Injection ด้วย Service Locator	22
5.5 การกำหนดขอบเขตระบบด้วย Use Case Diagram	23
5.6 การประยุกต์ใช้งานเทคโนโลยี NFC	31
5.7 กระบวนการทำงานของ NFC ภายในแอปพลิเคชัน	32
5.7.1 การควบคุม Session NFC	32
5.7.2 โหมดการอ่านแท็ก NFC	33
5.7.3 โหมดการเขียนแท็ก NFC	34
5.7.4 โครงสร้างข้อมูลภายในแท็ก NFC (Ntag213)	35
5.7.5 การติดตามการ์ดในเตี๊ยมด้วยแท็ก NFC	35
5.8 การเชื่อมต่อข้อมูลการ์ดจาก API ภายนอก	37
5.9 การจัดการข้อมูลและการซิงค์แบบสองทาง (Two-Way Sync)	39
5.9.1 ขั้นตอนการซิงค์ข้อมูลแบบสองทาง	39
5.9.2 การจัดการข้อมูลเมื่อเกิด Event ต่าง ๆ	39
5.10 การออกแบบโครงสร้างฐานข้อมูล	40
5.10.1 รายละเอียดของแต่ละตารางในโครงสร้างฐานข้อมูล	41
5.10.2 เหตุผลและแนวคิดในการออกแบบ	41
5.10.3 ความเชื่อมโยงระหว่างสอง schema	42
5.11 การออกแบบส่วนติดต่อผู้ใช้ (UI)	43
5.11.1 หน้าต้อนรับและเข้าใช้งาน	44
5.11.2 หน้าจัดการเตี๊ยม	45
5.11.3 หน้าสร้างและแก้ไขเตี๊ยม	46
5.11.4 หน้าติดตามและวิเคราะห์เตี๊ยม	47
5.11.5 หน้าอ่านแท็ก	48
5.11.6 หน้าเลือกคอลเล็กชันและค้นหาการ์ด	49
5.11.7 หน้ารายละเอียดการ์ด และสร้างการ์ดแบบกำหนดเอง	50

5.11.8 หน้าตั้งค่าและเลือกภาษา	51
5.11.9 หน้า About / Privacy Policy / Terms of Use	52
5.11.10 หน้าข้อผิดพลาด (404 – Page Not Found)	53
บทที่ 6 ผลการดำเนินงาน ข้อจำกัด และข้อเสนอแนะ	54
6.1 ผลการดำเนินงาน	54
6.2 ข้อจำกัด	60
6.3 ข้อเสนอแนะ	60
บรรณานุกรม	62
ภาคผนวก ก ขั้นตอนการนำโปรเจกต์จาก GitHub มาใช้งานเพื่อพัฒนา	63

สารบัญตาราง

	หน้า
ตารางที่ 2.1 เปรียบเทียบคุณสมบัติของแอปพลิเคชันที่เกี่ยวข้อง	7
ตารางที่ 4.1 แผนการดำเนินงาน	15
ตารางที่ 5.1 รายละเอียด Use Case #01 เข้าสู่ระบบ	24
ตารางที่ 5.2 รายละเอียด Use Case #02 สร้างเด็ก	25
ตารางที่ 5.3 รายละเอียด Use Case #03 แกะไขเด็ก	26
ตารางที่ 5.4 รายละเอียด Use Case #04 แชรส์เด็ก	26
ตารางที่ 5.5 รายละเอียด Use Case #05 ลบเด็ก	27
ตารางที่ 5.6 รายละเอียด Use Case #06 ตรวจสอบแท็ก	27
ตารางที่ 5.7 รายละเอียด Use Case #07 ติดตามการรด์	28
ตารางที่ 5.8 รายละเอียด Use Case #08 บันทึกประวัติการเล่น	28
ตารางที่ 5.9 รายละเอียด Use Case #09 แสดงสถิติการเล่น	29
ตารางที่ 5.10 รายละเอียด Use Case #10 แชรประวัติการเล่นด้วย QR Code	29
ตารางที่ 5.11 รายละเอียด Use Case #11 สร้างการรด์	30
ตารางที่ 5.12 รายละเอียด Use Case #12 ตั้งค่าแอป	30
ตารางที่ 5.13 ตัวอย่างโครงสร้างข้อมูลของ Record Payload ภายในแท็ก NFC	35

สารบัญรูป

	หน้า
รูปที่ 2.1 ส่วนติดต่อผู้ใช้ของแอปพลิเคชัน Big AR Vanguard.....	4
รูปที่ 2.2 ส่วนแสดงผลซ้อนทับของ Deck Tracker ระหว่างการเล่นเกมน	5
รูปที่ 2.3 หน้าต่างหลักของโปรแกรมสำหรับดูประวัติและสถิติการเล่น	5
รูปที่ 2.4 ส่วนติดต่อผู้ใช้ของแอปพลิเคชัน NFC Tools.....	6
รูปที่ 3.1 ตัวอย่างการ์ด TCG (Trading Card Game)	8
รูปที่ 3.2 หลักการทำงานของเทคโนโลยี NFC.....	9
รูปที่ 3.3 เฟรมเวิร์ค Flutter	10
รูปที่ 3.4 โครงสร้างสถาปัตยกรรม Clean Architecture	11
รูปที่ 3.5 แพลตฟอร์ม Firebase.....	12
รูปที่ 3.6 ฐานข้อมูล SQLite	13
รูปที่ 5.1 แผนภาพสถาปัตยกรรมของระบบที่ออกแบบตามหลักการ Clean Architecture	16
รูปที่ 5.2 แผนภาพแสดงกระบวนการทำงานระหว่างเลเยอร์ตามสถาปัตยกรรมซอฟต์แวร์	17
รูปที่ 5.3 โครงสร้างไดเรกทอรีของ Presentation Layer	18
รูปที่ 5.4 โครงสร้างไดเรกทอรีของ Domain Layer	18
รูปที่ 5.5 โครงสร้างไดเรกทอรีของ Data Layer	18
รูปที่ 5.6 โค้ดตัวอย่างของ ApplicationBloc สำหรับจัดการสถานะของการตั้งค่า	19
รูปที่ 5.7 โค้ดตัวอย่างของ UpdateSettingUseCase สำหรับจัดการตรรกะการอัปเดตค่าตั้งค่า.....	20
รูปที่ 5.8 โค้ดตัวอย่างของ UpdateSettingRepository สำหรับเป็นตัวกลางในการส่งต่อข้อมูล	20
รูปที่ 5.9 โค้ดตัวอย่างของ UpdateSettingLocalDatasource สำหรับการเชื่อมต่อกับแหล่งข้อมูล.....	21
รูปที่ 5.10 โค้ดตัวอย่างของ SharedPreferencesService สำหรับการบันทึกข้อมูลลงอุปกรณ์.....	21
รูปที่ 5.11 โค้ดตัวอย่างของ setupLocator สำหรับเป็นศูนย์กลางในการลงทะเบียนบริการทั้งหมด.....	22
รูปที่ 5.12 โค้ดตัวอย่างของ setupSharedPreferences สำหรับลงทะเบียน SharedPreferencesService	22
รูปที่ 5.13 Use Case Diagram แสดงขอบเขตการทำงานของระบบ.....	23
รูปที่ 5.14 ลักษณะโครงสร้างแท็ก NFC รุ่น Ntag213	31
รูปที่ 5.15 ตัวอย่างการติดแท็ก NFC บนซองใส่การ์ด (Sleeve)	31
รูปที่ 5.16 แผนภาพลำดับงานของกระบวนการควบคุม NFC Session.....	32
รูปที่ 5.17 แผนภาพลำดับงานสำหรับโหมดการอ่านแท็ก NFC	33
รูปที่ 5.18 แผนภาพลำดับงานสำหรับโหมดการเขียนแท็ก NFC	34
รูปที่ 5.19 แผนภาพลำดับงานของระบบติดตามการ์ดด้วยแท็ก NFC.....	36
รูปที่ 5.20 แผนภาพคลาสแสดงโครงสร้างการเชื่อมต่อ API ด้วย Factory Pattern	37
รูปที่ 5.21 แผนภาพลำดับเหตุการณ์แสดงการทำงานของ Game Factory.....	38
รูปที่ 5.22 แผนภาพแสดงความสัมพันธ์ข้อมูลของระบบ.....	40
รูปที่ 5.23 โครงสร้างข้อมูลแบบ Document-Oriented ใน Firestore.....	42

รูปที่ 5.24	แผนผังลำดับหน้าจอที่แสดงโครงสร้าง 4 ระดับของแอปพลิเคชัน.....	43
รูปที่ 5.25	การเริ่มต้นใช้งาน: หน้าต้อนรับ การเข้าสู่ระบบแบบออฟไลน์ และการเข้าสู่ระบบแบบออนไลน์.....	44
รูปที่ 5.26	การจัดการเด็ก: หน้าเมื่อไม่มีเด็ก การเลือกสร้างเด็กใหม่ และการลบเด็ก.....	45
รูปที่ 5.27	การสร้างเด็ก: หน้าเด็กเปล่า การแสดงผลการ์ดในเด็ก และโหมดแก้ไข.....	46
รูปที่ 5.28	การติดตามและวิเคราะห์เด็ก: หน้าแสดงรายการการ์ด โหมดติดตามการเล่น และโหมดวิเคราะห์สถิติ.....	47
รูปที่ 5.29	การอ่านแท็ก NFC: หน้าสแกนแท็ก การแสดงประวัติ และเมนูเลือกเกม.....	48
รูปที่ 5.30	การเลือกและค้นหาคาร์ด: หน้าเลือกคอลเลกชัน และหน้าค้นหารายการการ์ด.....	49
รูปที่ 5.31	การจัดการข้อมูลการ์ด: หน้าแสดงข้อมูล, การเพิ่มการ์ดลงเด็ก และการสร้างการ์ดเอง.....	50
รูปที่ 5.32	การตั้งค่าแอปพลิเคชัน: หน้าตั้งค่าหลัก และหน้าเลือกภาษา.....	51
รูปที่ 5.33	ข้อมูลแอปพลิเคชัน: เกี่ยวกับแอป นโยบายความเป็นส่วนตัว และข้อกำหนดการใช้งาน.....	52
รูปที่ 5.34	หน้าจอแสดงข้อผิดพลาด 404 (Page Not Found).....	53
รูปที่ 6.1	ขั้นตอนการลงชื่อเข้าใช้งาน: หน้าต้อนรับ การเลือกวิธีการเข้าสู่ระบบ และการเลือกบัญชีผู้ใช้.....	54
รูปที่ 6.2	ขั้นตอนการสร้างเด็กใหม่: การเลือกเมนูสร้างเด็ก การเตรียมเพิ่มการ์ด และการเลือกคอลเลกชัน.....	55
รูปที่ 6.3	ขั้นตอนการสร้างการ์ด: การสร้างการ์ดใหม่ การกรอกรายละเอียด และการยืนยันเพื่อสร้างการ์ด.....	55
รูปที่ 6.4	ขั้นตอนการเพิ่มการ์ดเข้าเด็ก: การค้นหาคาร์ด การเลือกจำนวนและยืนยัน และการแสดงผล.....	56
รูปที่ 6.5	ขั้นตอนการสอนใช้ (Tutorial) ฟังก์ชันเขียน NFC: การบันทึกเด็ก คำแนะนำการใช้งานครั้งแรก.....	56
รูปที่ 6.6	ขั้นตอนการแชร์ข้อมูลเด็ก: หน้าเด็กที่สร้างเสร็จสิ้น การคัดลอกข้อมูล และตัวอย่างข้อมูลเมื่อนำไปวาง.....	57
รูปที่ 6.7	ขั้นตอนการทดสอบการเขียน NFC: สถานะก่อนการเขียน การเขียนข้อมูลสำเร็จ และการเขียนข้อมูลซ้ำ.....	57
รูปที่ 6.8	ขั้นตอนการติดตามเด็ก: หน้าจอเริ่มต้นพร้อมคำแนะนำ การอัปเดตสถานะการ์ด และการแสดงผลสถิติ.....	58
รูปที่ 6.9	ฟังก์ชันรีเซตและดูประวัติ: ตัวเลือกการรีเซตเด็ก ประวัติการเล่น และสถิติการเล่น.....	58
รูปที่ 6.10	การทดสอบฟังก์ชันอ่าน NFC: กรณีอ่านข้อมูลสำเร็จ และกรณีข้อมูลไม่รองรับ.....	59
รูปที่ 6.11	ขั้นตอนการเปลี่ยนภาษาและธีม: การเลือกเมนูภาษา การเปลี่ยนเป็นภาษาญี่ปุ่น การเปลี่ยนธีมสว่าง.....	59
รูปที่ 6.12	แผนภาพแสดงสถาปัตยกรรมของระบบที่สามารถต่อยอดได้ในอนาคต.....	61
ภาคผนวก ก. 1	หน้าจอ Git Repository ของโปรเจกต์.....	63
ภาคผนวก ก. 2	การใช้คำสั่ง git clone เพื่อคัดลอกโปรเจกต์.....	63
ภาคผนวก ก. 3	โครงสร้างไดเรกทอรีของโปรเจกต์ใน Visual Studio Code.....	64
ภาคผนวก ก. 4	การใช้คำสั่ง flutter pub get เพื่อติดตั้ง Dependencies.....	64
ภาคผนวก ก. 5	หน้าจอโปรเจกต์ใน Supabase Dashboard.....	65
ภาคผนวก ก. 6	การเข้าสู่เมนู Storage ใน Supabase.....	65
ภาคผนวก ก. 7	หน้าต่างสำหรับสร้าง Storage Bucket ใหม่.....	66
ภาคผนวก ก. 8	การกำหนด Policy สำหรับการเข้าถึงข้อมูลใน Storage.....	66
ภาคผนวก ก. 9	หน้าจอการตั้งค่าโปรเจกต์ (Project Settings) ใน Supabase.....	67
ภาคผนวก ก. 10	การตั้งค่า Key และ URL ในไฟล์ .env.....	67
ภาคผนวก ก. 11	หน้าจอโปรเจกต์ใน Firebase Console.....	68
ภาคผนวก ก. 12	ขั้นตอนการสร้างโปรเจกต์ใหม่ใน Firebase.....	68
ภาคผนวก ก. 13	หน้าจอสำหรับดาวน์โหลดไฟล์ google-services.json.....	69

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญของปัญหา

เกมการ์ด TCG (Trading Card Game) เป็นเกมที่ได้รับความนิยมอย่างแพร่หลายทั่วโลกเนื่องจากผู้เล่นสามารถใช้ความคิดสร้างสรรค์และวางแผนกลยุทธ์เพื่อเอาชนะคู่ต่อสู้ โดยจุดเด่นของเกมการ์ดประเภทนี้ คือความสามารถในการสร้างและปรับแต่ง "เด็ค" หรือ "สำรับการ์ด" ได้อย่างอิสระ โดยเด็คเปรียบเสมือนอาวุธลับที่สะท้อนสไตล์การเล่นและความชอบของผู้เล่นแต่ละคน นอกจากนี้เกมการ์ดยังมีการ์ดใหม่และกติกาที่เปลี่ยนแปลงอยู่เสมอทำให้ผู้เล่นจำเป็นต้องอัปเดตเด็คของตนให้พร้อมสำหรับการแข่งขันอยู่ตลอดเวลา

หนึ่งในปัญหาหลักของผู้เล่นเกมการ์ด คือ การจัดการเด็คและการติดตามการ์ดที่เหลืออยู่ระหว่างเกมซึ่งข้อมูลเหล่านี้มีผลโดยตรงต่อการตัดสินใจของผู้เล่น เพราะผู้เล่นจำเป็นต้องคาดเดาว่าการ์ดที่เหลือในเด็คมีอะไรบ้างเพื่อวางแผนและเพิ่มโอกาสในการเอาชนะคู่ต่อสู้ แม้ว่าในปัจจุบันจะมีเครื่องมือช่วยจัดการเด็คที่ติดอยู่แล้วแต่ยังขาดเครื่องมือที่สามารถตรวจสอบการ์ดที่เหลืออยู่ในเด็คและแสดงการ์ดที่ถูกเล่นไปแล้ว ทำให้ผู้เล่นจะต้องจำตำแหน่งการ์ดด้วยตนเองซึ่งอาจส่งผลให้การติดตามการ์ดยังขาดความแม่นยำและมีโอกาสเกิดข้อผิดพลาดขึ้นได้

เพื่อยืนยันถึงปัญหาดังกล่าวและศึกษาความต้องการของผู้ใช้งานจริงในกลุ่มเฉพาะ ผู้จัดทำจึงได้ออกแบบสอบถามเกี่ยวกับความต้องการและพฤติกรรมของผู้เล่นโดยใช้เครื่องมือ Google Forms และเผยแพร่ผ่านชุมชนออนไลน์บนเว็บไซต์ Reddit [1] ในกลุ่มที่เกี่ยวข้องกับเกมการ์ด เช่น r/TCG, r/cardgames, r/SampledSize และ r/freemagic โดยเปิดรับคำตอบระหว่างวันที่ 1 มีนาคม – 30 เมษายน พ.ศ. 2568 และมีผู้สนใจเข้าร่วมตอบแบบสอบถามทั้งสิ้น 16 คน ผลการสำรวจสะท้อนให้เห็นถึงปัญหาที่ผู้เล่นประสบอย่างชัดเจน โดยสรุปประเด็นสำคัญได้ดังนี้ ปัญหาด้านการจัดการและการจดจำการ์ด ผู้ตอบแบบสอบถามส่วนใหญ่ประสบปัญหาเกี่ยวกับการจัดการเด็ค โดย 60.0% ระบุว่าเคย "ลืมว่ามีการ์ดอะไรอยู่ในเด็ค" และ 53.3% รู้สึกว่า "มีหลายเด็คแล้วจัดการไม่ทัน" นอกจากนี้ ในระหว่างการเล่น ปัญหาที่พบบ่อยที่สุดคือ "เสียเวลาในการนับการ์ดที่เหลือ" 46.7% และ "ลืมว่าการ์ดไปไหนเล่นไปแล้ว" 40.0% ซึ่งข้อมูลเหล่านี้ชี้ให้เห็นถึงความต้องการเครื่องมือที่จะมาช่วยลดภาระในการจดจำและเพิ่มความสะดวกสบาย ความสนใจในแอปพลิเคชันเพื่อแก้ปัญหา ผู้ตอบแบบสอบถามแสดงความสนใจในการใช้งานแอปพลิเคชัน NFC Deck Tracker ในระดับสูงมากถึง 87.5% (แบ่งเป็น "สนใจมากอยากลองใช้" 25.0% และ "สนใจคิดว่าจะมีประโยชน์" 62.5%) ซึ่งสะท้อนให้เห็นถึงศักยภาพของแอปพลิเคชันในการตอบสนองความต้องการของกลุ่มเป้าหมายได้เป็นอย่างดี

เทคโนโลยี NFC (Near Field Communication) เป็นระบบที่ช่วยให้การแลกเปลี่ยนข้อมูลระหว่างวัตถุและอุปกรณ์ดิจิทัลสามารถดำเนินการได้อย่างรวดเร็วและแม่นยำ เพียงแค่นำอุปกรณ์มาแตะหรือวางใกล้กัน นอกจากนี้ NFC ยังเป็นเทคโนโลยีที่เข้าถึงได้ง่ายและมีต้นทุนไม่สูงนักจึงถูกนำมาใช้ในหลากหลายอุตสาหกรรม เช่น การชำระเงินแบบไร้สัมผัสและการบริหารจัดการสินค้าในคลัง อย่างไรก็ตามในวงการเกมการ์ดยังมีการใช้งานเทคโนโลยีนี้น้อยมาก ซึ่งสะท้อนถึงโอกาสในการนำ NFC มาพัฒนาต่อยอดและสร้างประสบการณ์ใหม่ๆ ให้กับผู้เล่น

เพื่อแก้ปัญหาดังกล่าวแอปพลิเคชัน NFC Deck Tracker จึงนำเทคโนโลยี NFC มาใช้ในการจัดการเด็คและติดตามการ์ดที่เหลืออยู่ระหว่างเกมได้อย่างรวดเร็วและแม่นยำ เพียงแค่แตะอุปกรณ์เพียงครั้งเดียวผู้เล่นสามารถบันทึกและตรวจสอบข้อมูลเด็คของตนเองได้แบบเรียลไทม์ ลดความยุ่งยากจากการจดจำตำแหน่งการ์ดและลดข้อผิดพลาดในการติดตามการ์ด แอปนี้ไม่เพียงแต่ช่วยเพิ่มความสะดวกสบายให้กับผู้เล่นแต่ยังช่วยยกระดับประสบการณ์การเล่นเกมให้ทันสมัยและตอบโจทย์ผู้เล่นทุกระดับตั้งแต่ผู้เล่นทั่วไปไปจนถึงนักแข่งขันระดับมืออาชีพ

1.2 วัตถุประสงค์

- 1) เพื่อพัฒนาระบบติดตามการ์ดที่เหลือนอยู่ในตู้ระหว่างเกมให้มีความแม่นยำและรวดเร็วยิ่งขึ้น

1.3 ลักษณะและขอบเขต

แอปพลิเคชัน NFC Deck Tracker เป็นเครื่องมือสำหรับผู้เล่นเกมการ์ดที่พัฒนาขึ้นบนพื้นฐานทางเทคนิคและมีขอบเขตการทำงานดังนี้

1.3.1 คุณสมบัติทางเทคนิค

- พัฒนาด้วยเฟรมเวิร์ค Flutter รองรับการทำงานแบบ Cross-platform บนระบบปฏิบัติการ iOS และ Android
- ออกแบบด้วยสถาปัตยกรรมซอฟต์แวร์ Clean Architecture เพื่อความยืดหยุ่นและง่ายต่อการบำรุงรักษาและพัฒนาต่อยอดในอนาคต

1.3.2 แอปพลิเคชันฟีเจอร์

- ผู้ใช้สามารถสร้างและปรับแต่งเด็คการ์ดของตนเองได้อย่างอิสระ
- ผู้ใช้สามารถบันทึก แก้ไข ลบ และแบ่งปันข้อมูลเด็คผ่านคลิปปอร์ดได้
- ข้อมูลเด็คถูกจัดเก็บในฐานข้อมูล SQLite เพื่อรองรับการใช้งานแบบออฟไลน์
- รองรับการเพิ่มการ์ดเข้าเด็คจากฐานข้อมูลของเกมต่าง ๆ ผ่าน API ภายนอก และจากการ์ดที่ผู้ใช้สร้างขึ้น
- ระบบจัดเก็บข้อมูลการ์ดจาก API ลงในฐานข้อมูล SQLite ภายในเครื่อง เพื่อการเรียกใช้งานครั้งต่อไปที่รวดเร็วและมีการอัปเดตข้อมูลเฉพาะการ์ดใหม่เมื่อ API มีการเปลี่ยนแปลง
- รองรับการบันทึกข้อมูลรหัสเกมและรหัสการ์ดลงบนแท็ก NFC
- สามารถใช้สแกนโทรศัพท์สแกนแท็ก NFC เพื่อตรวจสอบข้อมูลและติดตามสถานะการ์ดในตู้ระหว่างการเล่นเกมแบบเรียลไทม์
- แสดงผลข้อมูลการเล่นในรูปแบบแผนภูมิแท่ง สรุปจำนวนการ์ดที่ จั่ว/คืน เปอร์เซนต์การใช้การ์ด และจำนวนการ์ดที่ไม่ได้ใช้งาน
- บันทึกประวัติการเล่นของแต่ละแมตช์ตามลำดับเวลา
- สามารถแบ่งปันประวัติการเล่นกับผู้เล่นอื่นผ่าน QR Code
- รองรับการเข้าสู่ระบบผ่านบัญชี Google และการใช้งานในโหมด Guest
- เมื่อผู้ใช้เข้าสู่ระบบด้วยบัญชี Google ข้อมูลจะถูกซิงโครไนซ์กับ Firebase เพื่อการใช้งานข้ามอุปกรณ์
- ผู้ใช้สามารถลงชื่อ เข้า-ออก จากระบบได้ภายหลัง
- รองรับการเปลี่ยนภาษาภายในแอปพลิเคชัน

1.4 อุปกรณ์และเครื่องมือที่ใช้

1.4.1 ฮาร์ดแวร์

- Galaxy A51 (ระบบปฏิบัติการ Android)
- Ntag213 13.56Mhz 180bytes size 21*1 1mm

1.4.2 ซอฟต์แวร์

- Draw.io
- Figma
- Firebase
- Flutter
- GitHub
- Postman
- Supabase
- Visual Studio Code

1.5 ผลที่คาดว่าจะได้รับ

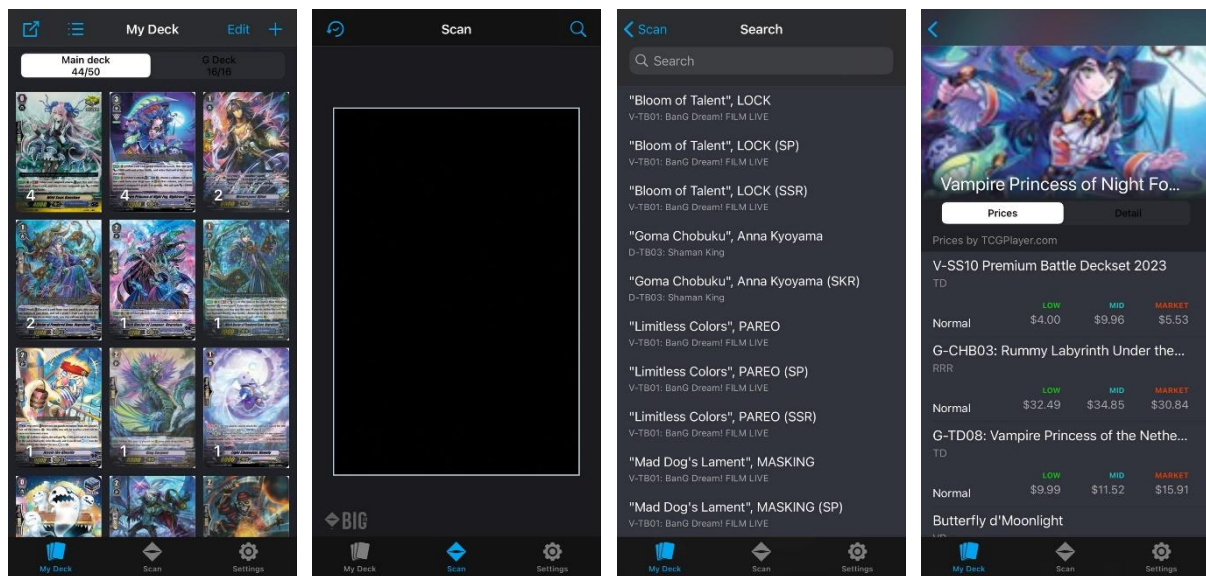
- 1) ผู้ใช้สามารถ สร้าง ปรับแต่ง และจัดการเด็กของตนเอง ได้อย่างสะดวกและมีประสิทธิภาพ โดยรองรับการบันทึกและซิงค์ข้อมูลเด็กผ่าน Cloud (Firebase) และ Local (SQLite) ทำให้การจัดการเด็กเป็นระบบมากขึ้น และสามารถเข้าถึงข้อมูลเด็กของตนเองจากอุปกรณ์ต่างๆ
- 2) ระบบติดตามการ์ดช่วยให้การเล่นมีความแม่นยำและรวดเร็วยิ่งขึ้น ผู้ใช้สามารถตรวจสอบจำนวนการ์ดที่เหลืออยู่ในเด็กแบบเรียลไทม์ ลดข้อผิดพลาดจากการจดจำด้วยตนเองและช่วยให้สามารถวางแผนกลยุทธ์ระหว่างเกมได้อย่างมีประสิทธิภาพ
- 3) ผู้ใช้ได้รับประสบการณ์การใช้งานที่ทันสมัยและเหมาะกับผู้เล่นทุกระดับ แอปพลิเคชันใช้งานง่าย รองรับการทำงานโหมดออนไลน์และออฟไลน์เพื่อให้ตอบโจทย์ทั้งผู้เล่นทั่วไปและนักแข่งขันมืออาชีพ

บทที่ 2 ผลงานที่เกี่ยวข้อง

บทนี้กล่าวถึงผลงานที่เกี่ยวข้องซึ่งนำมาใช้เป็นแนวทางในการพัฒนาโครงการ NFC Deck Tracker โดยมุ่งเน้นการวิเคราะห์ ลักษณะการใช้งานข้อเด่น และข้อด้อย ของแต่ละผลงานที่เกี่ยวข้อง

2.1 Big AR Vanguard

Big AR Vanguard [2] เป็นแอปพลิเคชันที่พัฒนาขึ้นเพื่อช่วยผู้เล่นเกมการ์ด Cardfight!! Vanguard ในการสร้างเต็และค้นหาการ์ดได้อย่างสะดวก โดยมีจุดเด่นคือการใช้เทคโนโลยี AR (Augmented Reality) สแกนการ์ดจริงผ่านกล้องมือถือเพื่อดึงข้อมูลขึ้นมาได้ทันที และยังรองรับการตรวจสอบราคาคาร์ดโดยอ้างอิงข้อมูลจาก TCGplayer ซึ่งเป็นแหล่งข้อมูลที่เชื่อถือได้ ดังรูปที่ 2.1 แสดงตัวอย่างหน้าจอการทำงานหลักได้แก่ หน้าจัดการเต็ หน้าสแกนการ์ด หน้าผลการค้นหา และหน้าแสดงราคา อย่างไรก็ตามแอปพลิเคชันนี้ยังคงมีข้อจำกัดคือรองรับเฉพาะเกม Cardfight!! Vanguard และไม่ได้เชื่อมต่อกับผู้พัฒนาเกมโดยตรง



รูปที่ 2.1 ส่วนติดต่อผู้ใช้ของแอปพลิเคชัน Big AR Vanguard

ข้อเด่น

1. สะดวกในการจัดการเต็สำหรับผู้เล่น Cardfight Vanguard
2. รองรับการค้นหาการ์ดได้ทั้งการพิมพ์ชื่อการ์ดและการใช้ AR สแกนการ์ดจริง
3. รองรับการตรวจสอบราคาคาร์ด ช่วยให้ผู้เล่นสามารถติดตามมูลค่าของการ์ดในตลาดได้อย่างแม่นยำ
4. มีฐานข้อมูลที่อัปเดตอยู่เสมอทำให้ผู้ใช้สามารถเข้าถึงข้อมูลการ์ดและราคาจากแหล่งที่น่าเชื่อถือ

ข้อด้อย

1. รองรับเฉพาะเกม Cardfight Vanguard ไม่สามารถใช้งานร่วมกับเกมการ์ดประเภทอื่นได้
2. ไม่มีการเชื่อมต่อโดยตรงกับผู้พัฒนาเกมทำให้ข้อมูลบางส่วนขึ้นอยู่กับแหล่งข้อมูลภายนอก

2.2 Hearthstone Deck Tracker

Hearthstone Deck Tracker [3] เป็นซอฟต์แวร์ที่ช่วยให้ผู้เล่นเกม Hearthstone สามารถติดตามการ์ดที่เล่นไปแล้วและวิเคราะห์เทคของตนเองได้อย่างแม่นยำ โดยระบบจะอัปเดตข้อมูลเทคของผู้เล่นแบบเรียลไทม์ แสดงจำนวนการ์ดที่เหลืออยู่และการ์ดที่ถูกเล่นไปแล้ว นอกจากนี้ยังสามารถติดตามการ์ดที่คู่ต่อสู้ใช้เพื่อช่วยในการคาดการณ์กลยุทธ์ของอีกฝ่ายได้ ผู้ใช้สามารถบันทึกผลการเล่นของแต่ละแมตช์พร้อมทั้งดูสถิติการชนะ-แพ้ และวิเคราะห์ประสิทธิภาพของเทคเทียบกับเมตาเกมในปัจจุบัน ดังรูปที่ 2.2 ส่วนแสดงผลระหว่างเล่นเกม และรูปที่ 2.3 หน้าต่างแสดงประวัติและสถิติการเล่น แม้ว่าซอฟต์แวร์นี้จะไม่ใช้ส่วนหนึ่งของเกมอย่างเป็นทางการแต่ก็ช่วยให้ผู้เล่นตัดสินใจระหว่างการแข่งขันได้อย่างมีประสิทธิภาพ



รูปที่ 2.2 ส่วนแสดงผลซ้อนทับของ Deck Tracker ระหว่างการเล่นเกม



รูปที่ 2.3 หน้าต่างหลักของโปรแกรมสำหรับดูประวัติและสถิติการเล่น

ข้อเด่น

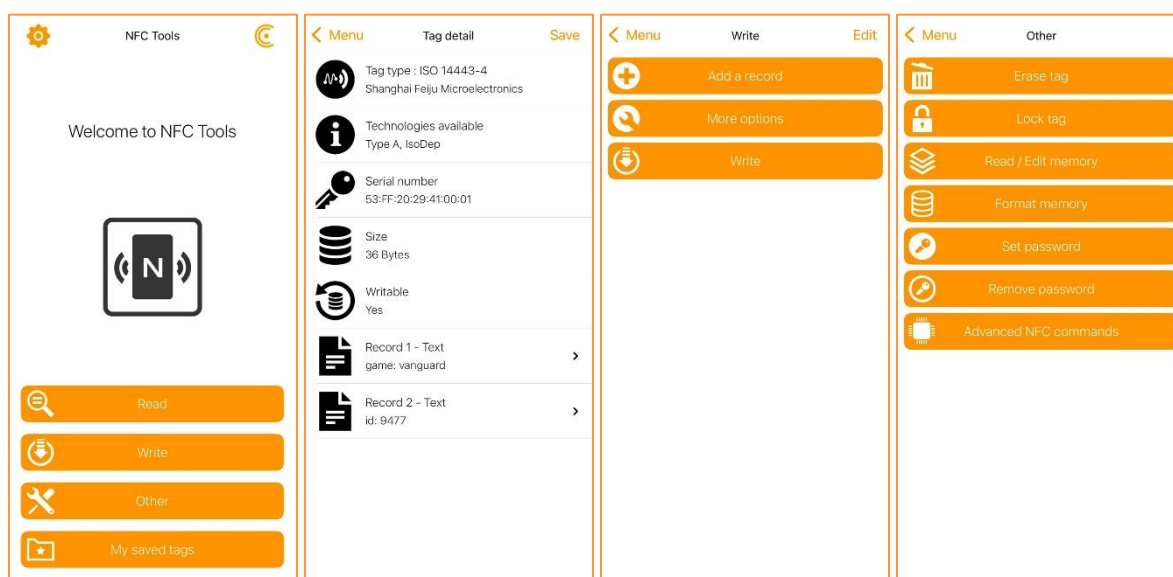
1. แสดงจำนวนการ์ดที่เหลืออยู่ในเทคและการ์ดที่ถูกเล่นไปแล้ว ช่วยให้ผู้เล่นสามารถติดตามสถานะของเทคได้
2. ติดตามการ์ดที่คู่แข่งใช้เพื่อช่วยวิเคราะห์แนวทางการเล่นและคาดการณ์กลยุทธ์ของอีกฝ่าย
3. คำนวณเปอร์เซ็นต์อัตราชนะตามสถิติการเล่นกับคู่แข่งประเภทต่างๆ

ข้อด้อย

1. สามารถบันทึกข้อมูลการ์ดของคู่แข่งได้เฉพาะการ์ดที่ถูกเปิดเผยแล้วเท่านั้น

2.3 NFC Tools

NFC Tools [4] เป็นแอปพลิเคชันที่ช่วยให้ผู้ใช้สามารถ อ่าน เขียน ลบ และจัดการข้อมูลบนแท็ก NFC ได้อย่างสะดวก รองรับการร่วมงานร่วมกับแท็ก NFC มาตรฐาน เช่น ISO 14443 และ NDEF ซึ่งสามารถใช้งานได้กับอุปกรณ์ที่รองรับ NFC เช่น สมาร์ทโฟนและแท็บเล็ต ผู้ใช้สามารถ อ่านข้อมูลแท็ก NFC เพียงแตะสมาร์ทโฟนกับแท็ก ระบบจะแสดงรายละเอียดต่าง ๆ เช่น ประเภทแท็ก หมายเลขซีเรียล ขนาดหน่วยความจำ และข้อมูลที่บันทึกอยู่ในแท็ก นอกจากนี้ NFC Tools ยังรองรับ การเขียนข้อมูลลงแท็ก NFC ไม่ว่าจะเป็น ข้อความ URL หมายเลขโทรศัพท์ หรือข้อมูลแบบกำหนดเอง แอปยังมีฟีเจอร์ที่ช่วยให้ผู้ใช้สามารถ เพิ่ม ลบ หรือแก้ไขข้อมูลที่บันทึกอยู่ในแท็ก NFC ได้อย่างอิสระ รวมถึงฟีเจอร์ล็อกแท็กเพื่อป้องกันการเปลี่ยนแปลงข้อมูลภายหลัง ดังรูปที่ 2.4 แสดงตัวอย่างหน้าจอการทำงานหลักได้แก่ หน้าหลักของแอปพลิเคชัน หน้ารายละเอียดแท็กที่อ่าน หน้าเมนูสำหรับเขียนแท็ก และหน้าเครื่องมืออื่น ๆ



รูปที่ 2.4 ส่วนติดต่อผู้ใช้ของแอปพลิเคชัน NFC Tools

ข้อเด่น

1. รองรับแท็ก NFC มาตรฐานหลายประเภทเช่น ISO 14443, NDEF ทำให้สามารถใช้งานร่วมกับอุปกรณ์ที่รองรับ NFC ได้หลากหลาย
2. สามารถบันทึกข้อมูลประเภทต่าง ๆ ลงแท็ก NFC เช่น ข้อความ URL หมายเลข และข้อมูลไบนารี เพื่อรองรับการใช้งานที่หลากหลาย
3. รองรับฟีเจอร์การจัดการแท็กเพิ่มเติม เช่น ลบข้อมูล ฟลैชแท็ก ล็อกแท็ก และตั้งรหัสผ่านเพื่อเพิ่มความปลอดภัยในการใช้งาน
4. อินเทอร์เฟซใช้งานง่าย มีการออกแบบที่ เรียบง่ายและเป็นมิตรกับผู้ใช้ ทำให้สะดวกต่อการจัดการแท็ก NFC

ข้อด้อย

1. ไม่รองรับการอ่าน/เขียนแท็กที่มีการเข้ารหัสพิเศษ เช่น บัตรเครดิตหรือบัตรโดยสารที่มีระบบเข้ารหัสข้อมูล
2. ฟีเจอร์บางอย่าง เช่น การตั้งรหัสผ่าน อาจรองรับเฉพาะแท็กบางรุ่น ทำให้การใช้งานมีข้อจำกัดในบางกรณี

2.4 สรุปผลงานที่เกี่ยวข้อง

จากการศึกษาผลงานที่เกี่ยวข้องสามารถสรุปและเปรียบเทียบคุณสมบัติหลักของแต่ละแอปพลิเคชันเพื่อแสดงให้เห็นถึงความแตกต่างและจุดเด่นได้ ดังแสดงในตารางที่ 2.1 (✓ หมายถึงสามารถทำคุณสมบัตินั้นได้)

ตารางที่ 2.1 เปรียบเทียบคุณสมบัติของแอปพลิเคชันที่เกี่ยวข้อง

คุณสมบัติ	Big AR Vanguard	Hearthstone DTK	NFC Tools	NFC Deck Tracker
ค้นหาการ์ดด้วย AR	✓			
ตรวจสอบราคาในตลาด	✓			
แสดงข้อมูลการ์ด	✓	✓		✓
สร้างเด็ค	✓	✓		✓
ติดตามสถานะการ์ด		✓		✓
บันทึกประวัติการเล่น		✓		✓
วิเคราะห์สถิติการเล่น		✓		✓
วิเคราะห์อัตราชนะ		✓		
อ่านและเขียนแท็ก NFC			✓	✓

Big AR Vanguard มีจุดเด่นในด้านการค้นหาการ์ดด้วยเทคโนโลยี AR รวมถึงสามารถตรวจสอบราคการ์ดในตลาดได้ อีกทั้งยังรองรับการสร้างเด็คอย่างครบถ้วน ซึ่งเหมาะกับผู้เล่นที่ต้องการจัดการการ์ดในเชิงภาพและราคา Hearthstone Deck Tracker มุ่งเน้นที่การติดตามสถานะของการ์ดที่ถูกเล่นไปแล้วในเกม พร้อมทั้งมีความสามารถในการวิเคราะห์สถิติการเล่น และคำนวณอัตราชนะของเด็คเพื่อช่วยให้ผู้เล่นสามารถปรับกลยุทธ์และวางแผนได้อย่างมีประสิทธิภาพยิ่งขึ้น NFC Tools โดดเด่นในด้านการ อ่าน-เขียน และจัดการข้อมูลบนแท็ก NFC ซึ่งช่วยให้สามารถใช้งาน NFC ได้อย่างยืดหยุ่นและหลากหลาย เหมาะกับการใช้งานทั่วไปในหลายบริบท NFC Deck Tracker ซึ่งเป็นแอปที่พัฒนาในโครงการนี้ ความโดดเด่นจากการผสมรวมคุณสมบัติจากทั้งสามแอปพลิเคชันที่กล่าวมาโดยรองรับการสร้างเด็คจากเกมต่าง ๆ เช่นเดียวกับ Big AR Vanguard ติดตามสถานะการ์ดและวิเคราะห์สถิติการเล่นได้เช่นเดียวกับ Hearthstone Deck Tracker และยัง อ่าน-เขียน ข้อมูลบนแท็ก NFC ได้เช่นเดียวกับ NFC Tools

บทที่ 3 ทฤษฎีและความรู้ที่เกี่ยวข้อง

บทนี้กล่าวถึงทฤษฎีและองค์ความรู้ที่เกี่ยวข้องกับโครงการ NFC Deck Tracker ซึ่งมีความสำคัญต่อการพัฒนาและออกแบบระบบโดยมีความหมายและองค์ประกอบของเกมการ์ด หลักการทำงานของเทคโนโลยี NFC, Flutter, สถาปัตยกรรมซอฟต์แวร์และฐานข้อมูล เพื่อให้ระบบสามารถทำงานได้อย่างราบรื่นและตอบโจทย์การใช้งานของผู้เล่นจริง

3.1 TCG (Trading Card Game)

TCG (Trading Card Game) [5] หรือเกมการ์ดสะสม เป็นประเภทเกมที่ผู้เล่นสะสมการ์ดหลากหลายคุณสมบัติและความสามารถ เพื่อนำมาสร้าง "เด็ค" หรือ "สำรับการ์ด" สำหรับใช้ในการแข่งขันกับผู้เล่นคนอื่น ดังรูปที่ 3.1 จุดเด่นของ TCG คือความอิสระในการสร้างและปรับแต่งเด็ค ซึ่งเปรียบเสมือนอาวุธลับที่สะท้อนสไตล์การเล่นและความชอบของผู้เล่นแต่ละคน นอกจากนี้ เกมการ์ดยังมีการ์ดใหม่ ๆ การปรับสมดุลข้อจำกัดในการใช้ และกติกาที่เปลี่ยนแปลงอยู่เสมอ ทำให้ผู้เล่นจำเป็นต้องอัปเดตเด็คของตนให้พร้อมสำหรับการแข่งขันอยู่ตลอดเวลา



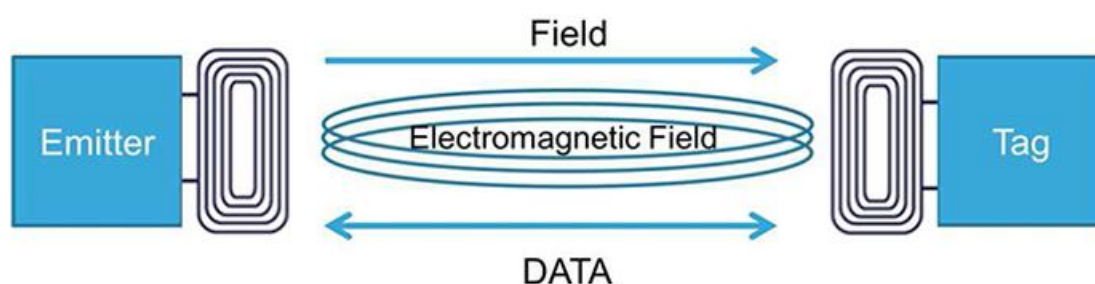
รูปที่ 3.1 ตัวอย่างการ์ด TCG (Trading Card Game)

องค์ประกอบหลักของ TCG ได้แก่ การ์ด เด็ค กติกา ระบบการแลกเปลี่ยน และการจัดทัวร์นาเมนต์ การ์ดแต่ละใบมีคุณสมบัติเฉพาะตัว เช่น การโจมตี การป้องกัน หรือเอฟเฟกต์พิเศษ ผู้เล่นต้องจัดเด็คให้สอดคล้องกับกลยุทธ์ของตนเอง โดยมีจำนวนการ์ดที่ใช้ในเด็คขึ้นอยู่กับกติกาของเกม แต่ละเกมมีกฎเฉพาะที่กำหนดวิธีการเล่นและกลยุทธ์ที่สามารถใช้ได้ ผู้เล่นยังสามารถแลกเปลี่ยนการ์ดกับผู้อื่นเพื่อปรับแต่งเด็คให้แข็งแกร่งขึ้นได้

จากการศึกษาโครงสร้างและวิธีการเล่นของ TCG ทำให้ทราบถึงความซับซ้อนที่ผู้เล่นต้องเผชิญ ซึ่งเป็นปัญหาหลักที่โครงการนี้ต้องการแก้ไขได้แก่ การจัดการเด็คและการติดตามการ์ดระหว่างการเล่น ข้อมูลเหล่านี้มีความสำคัญต่อการตัดสินใจเชิงกลยุทธ์ แต่การจดจำข้อมูลทั้งหมดด้วยตนเองนั้นเป็นภาระและเสี่ยงต่อการเกิดข้อผิดพลาด ความเข้าใจในปัญหานี้จึงเป็นรากฐานสำคัญในการออกแบบฟีเจอร์ของแอปพลิเคชันให้สามารถตอบสนองความต้องการของผู้เล่นได้ตรงจุด โดยได้พัฒนาฟีเจอร์ การสร้างและจัดการเด็ค การติดตามสถานะการ์ดแบบเรียลไทม์ การแสดงผลสถิติ ความรู้ความเข้าใจในบริบทของเกม TCG จึงเป็นสิ่งจำเป็นที่ทำให้สามารถพัฒนาเครื่องมือที่ช่วยแก้ปัญหาให้แก่ผู้เล่น

3.2 NFC (Near Field Communication)

NFC (Near Field Communication) [6] เป็นเทคโนโลยีสื่อสารไร้สายระยะใกล้ที่ใช้ในการแลกเปลี่ยนข้อมูลระหว่างอุปกรณ์โดยไม่ต้องเชื่อมต่ออินเทอร์เน็ต ทำงานที่ความถี่ 13.56 MHz และรองรับการสื่อสารในระยะไม่เกิน 10 เซนติเมตร ดังรูปที่ 3.2 เทคโนโลยี NFC ถูกนำมาใช้อย่างแพร่หลายในชีวิตประจำวัน เช่น การชำระเงินผ่านมือถือผ่านระบบ Google Pay และ Apple Pay การใช้แท่นบัตรโดยสารในระบบขนส่งสาธารณะ รวมถึงการควบคุมอุปกรณ์ IoT เช่น การปลดล็อกประตูหรือการเชื่อมต่อกับอุปกรณ์อัจฉริยะ NFC ใช้มาตรฐานที่กำหนดโดย ISO 14443 [7], NDEF [8] และสามารถใช้งานร่วมกับเทคโนโลยี MIFARE [9] เพื่อให้มั่นใจว่าการสื่อสารระหว่างอุปกรณ์มีความปลอดภัยและเชื่อถือได้



รูปที่ 3.2 หลักการทำงานของเทคโนโลยี NFC

ประเภทของ NFC

1. Active NFC อุปกรณ์ที่สามารถส่งและรับข้อมูลได้ เช่น สมาร์ทโฟนที่รองรับ NFC
2. Passive NFC อุปกรณ์ที่ไม่มีแหล่งพลังงานในตัวเอง เช่น แท็ก NFC ซึ่งต้องอาศัยพลังงานจากอุปกรณ์อ่าน

โหมดการทำงานของ NFC

1. Reader/Writer Mode สมาร์ทโฟนสามารถ อ่าน-เขียน ข้อมูลลงแท็ก NFC
2. Peer-to-Peer Mode อุปกรณ์สองเครื่องสามารถส่งข้อมูลหากันได้โดยตรง เช่น การแชร์ไฟล์
3. Card Emulation Mode อุปกรณ์ NFC สามารถจำลองเป็นบัตรอิเล็กทรอนิกส์ เช่น บัตรเครดิต

จากการศึกษาเทคโนโลยี NFC ทำให้ทราบถึงคุณลักษณะและโหมดการทำงานต่าง ๆ ซึ่งเป็นความรู้สำคัญที่ถูกนำมาใช้เป็นแกนหลักในการพัฒนาโครงงานเพื่อแก้ปัญหาการติดตามการ์ดที่ขาดความแม่นยำ โครงงานนี้ได้นำความรู้เรื่องโหมดการทำงาน Reader/Writer Mode มาประยุกต์ใช้โดยตรง โดยมีหลักการคือสมาร์ตโฟนของผู้ใช้จะทำหน้าที่เป็นอุปกรณ์ Active ในฐานะเครื่องอ่าน (Reader) การ์ดเกม (TCG Card) แต่ละใบจะถูกติดด้วย Passive NFC Tag ซึ่งภายในจะบันทึกข้อมูลเฉพาะที่ระบุว่าการ์ดใบนั้นคืออะไร เมื่อผู้เล่นนำการ์ดมาแตะที่สมาร์ตโฟนแอปพลิเคชันจะอ่านข้อมูลจากแท็กบนการ์ดทันทีทำให้ระบบสามารถระบุและบันทึกสถานะของการ์ดแต่ละใบแบบเรียลไทม์ได้อย่างรวดเร็วและแม่นยำ การประยุกต์ใช้ความรู้นี้จึงเป็นหัวใจสำคัญที่ทำให้สามารถเชื่อมโยงการเล่นการ์ดเกมในโลกจริงเข้ากับการประมวลผลข้อมูลในรูปแบบดิจิทัลได้ซึ่งแตกต่างจากแอปพลิเคชัน Deck Tracker ทั่วไปที่ไม่สามารถระบุสถานะของการ์ดทางกายภาพได้จริง

3.3 Flutter

Flutter [10] รูปที่ 3.3 คือชุดเครื่องมือสำหรับพัฒนาส่วนติดต่อผู้ใช้ที่สร้างโดย Google มีลักษณะเป็นโอเพนซอร์สจุดประสงค์หลักคือการพัฒนาแอปพลิเคชันที่สามารถทำงานข้ามแพลตฟอร์มได้จากโค้ดเบสเพียงชุดเดียว รองรับทั้งระบบปฏิบัติการ iOS, Android, Web และ Desktop เทคโนโลยีหลักที่ Flutter ใช้คือภาษาโปรแกรม Dart ซึ่งเป็นภาษาที่ถูกปรับให้เหมาะสมกับการสร้าง UI หลักการทำงานที่สำคัญของ Flutter คือการใช้เอนจินกราฟิกของตัวเองในการวาดส่วนประกอบ UI หรือที่เรียกว่า วิดเจ็ต (Widget) ลงบนหน้าจอโดยตรง แทนการเรียกใช้วิดเจ็ตดั้งเดิมของแต่ละระบบปฏิบัติการ วิธีการนี้ทำให้ UI ที่สร้างขึ้นมีหน้าตาและการทำงานที่สอดคล้องกันในทุกแพลตฟอร์ม ความยืดหยุ่นในการออกแบบได้อย่างเต็มที่ นอกจากนี้ โค้ดภาษา Dart ยังสามารถคอมไพล์เป็นโค้ดเนทีฟ (Native Code) ของแต่ละแพลตฟอร์มได้โดยตรง ทำให้แอปพลิเคชันที่พัฒนาด้วย Flutter มีประสิทธิภาพการทำงานที่รวดเร็ว

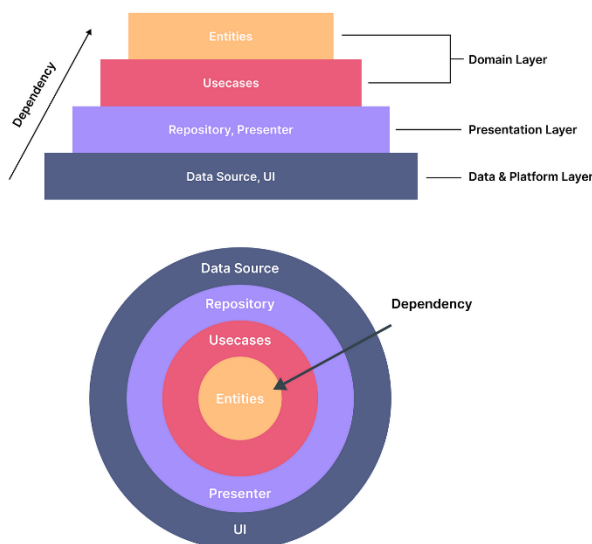


รูปที่ 3.3 เฟรมเวิร์ค Flutter

จากการศึกษาคุณลักษณะของ Flutter ทำให้เลือกใช้เป็นเครื่องมือหลักสำหรับพัฒนาโครงการโดยได้นำองค์ความรู้ในสองส่วนสำคัญมาประยุกต์ใช้คือ ความสามารถในการพัฒนาแบบ Cross-platform ความรู้ที่ว่า Flutter สามารถสร้างแอปพลิเคชันสำหรับ iOS และ Android ได้พร้อมกันจากการเขียนโค้ดเพียงครั้งเดียว เป็นปัจจัยสำคัญที่สอดคล้องกับเป้าหมายของโครงการที่ต้องการเข้าถึงกลุ่มผู้เล่นเกมการ์ดให้ได้กว้างขวางที่สุด โดยไม่จำกัดอยู่แค่ระบบปฏิบัติการใดระบบปฏิบัติการหนึ่ง การเลือกใช้ Flutter จึงช่วยประหยัดเวลาและทรัพยากรในการพัฒนาได้อย่างมีนัยสำคัญและความยืดหยุ่นของระบบ UI ความเข้าใจในสถาปัตยกรรมที่ใช้วิดเจ็ตเป็นศูนย์กลางของ Flutter สามารถออกแบบและสร้างส่วนติดต่อผู้ใช้ที่ ทันสมัย มีเอกลักษณ์ และใช้งานง่าย ซึ่งเป็นปัจจัยสำคัญในการสร้างความพึงพอใจและประสบการณ์ที่ดีให้แก่ผู้ใช้ โครงการจึงนำความสามารถด้านนี้มาใช้เพื่อสร้างสรรค์หน้าจอแอปพลิเคชันที่ดึงดูดสายตาและตอบโจทย์การใช้งานของผู้เล่นเกมการ์ดโดยเฉพาะ การตัดสินใจเลือกใช้ Flutter จึงเกิดจากการทำความเข้าใจในศักยภาพของเทคโนโลยี และเล็งเห็นว่าคุณสมบัติหลักทั้งสองด้านสามารถตอบสนองต่อเป้าหมายของโครงการได้อย่างลงตัว ทั้งในด้านการเข้าถึงผู้ใช้และคุณภาพของประสบการณ์การใช้งาน

3.4 Clean Architecture

Clean Architecture [11] รูปที่ 3.4 เป็นแนวคิดในการออกแบบซอฟต์แวร์ที่ช่วยให้โค้ดมีโครงสร้างที่เป็นระเบียบ ลดความซับซ้อน และง่ายต่อการดูแลรักษา โดยแนวคิดนี้ถูกนำมาใช้กับ Flutter เพื่อช่วยให้แอปพลิเคชันมีการจัดการโค้ดที่ดีและรองรับการขยายตัวในอนาคต โครงสร้างของ Clean Architecture แบ่งโค้ดออกเป็นเลเยอร์ตามหน้าที่ของแต่ละส่วนทำให้สามารถ ทดสอบ ปรับเปลี่ยน และพัฒนาเพิ่มเติมได้ง่าย



รูปที่ 3.4 โครงสร้างสถาปัตยกรรม Clean Architecture

โครงสร้างของ Clean Architecture

1. Presentation Layer เป็นส่วนที่ทำหน้าที่เป็นส่วนติดต่อผู้ใช้ (UI) ทั้งการแสดงผลและรับข้อมูลจากผู้ใช้งาน โดยใช้ Flutter Widgets และ State Management เช่น Bloc หรือ Cubit เพื่อจัดการสถานะของ UI และส่งคำสั่งไปยัง Business Logic
2. Domain Layer เป็นแกนกลางของแอปพลิเคชันที่ประกอบด้วย Business Logic และกฎเกณฑ์หลักโดยไม่ขึ้นกับส่วนอื่น ๆ เลเยอร์นี้จะรวมถึง Entities (โมเดลข้อมูลหลัก) Use Cases (Logic การทำงาน) และ Repositories (Interface สำหรับการเข้าถึงข้อมูล)
3. Data Layer ทำหน้าที่จัดการข้อมูลทั้งหมด รวมถึงการเชื่อมต่อกับแหล่งข้อมูลจริง เช่น API หรือฐานข้อมูล เลเยอร์นี้ implements Repository Interface ที่กำหนดใน Domain Layer โดยมีการใช้งาน Data Sources เพื่อเข้าถึงข้อมูลจาก Firebase SQLite หรือ NFC API และจัดการการแปลงข้อมูลระหว่าง Models และ Entities

จากการศึกษาหลักการของ Clean Architecture ทำให้ทราบถึงแนวทางการจัดการความซับซ้อนของโครงการ ซึ่งมีการบูรณาการเทคโนโลยีที่หลากหลายเข้าด้วยกัน ทั้งการเชื่อมต่อกับ เทคโนโลยี NFC, การติดต่อกับ API ภายนอก และการจัดการข้อมูลทั้งแบบออนไลน์และออฟไลน์ผ่าน Firebase และ SQLite ความรู้เรื่องการแบ่งแยกส่วนการทำงาน ได้ถูกนำมาประยุกต์ใช้โดยตรงเพื่อจัดระเบียบโครงสร้างของแอปพลิเคชันอย่างชัดเจน ทำให้แต่ละส่วนของโปรเจกต์มีความเป็นอิสระสูงสามารถ พัฒนา ทดสอบ และแก้ไขได้ง่ายโดยไม่กระทบกระทั่งกันเป็นรากฐานของโปรเจกต์ที่ยืดหยุ่น และพร้อมสำหรับการต่อยอดในอนาคตได้

3.5 Firebase

Firebase [12] รูปที่ 3.5 คือแพลตฟอร์มสำหรับพัฒนาแอปพลิเคชันจาก Google ที่จัดอยู่ในกลุ่ม Backend-as-a-Service (BaaS) ซึ่งให้บริการโครงสร้างพื้นฐานฝั่งเซิร์ฟเวอร์ที่พร้อมใช้งาน ช่วยให้นักพัฒนาสามารถมุ่งเน้นที่การสร้างฟีเจอร์ของแอปพลิเคชันเป็นหลัก



รูปที่ 3.5 แพลตฟอร์ม Firebase

บริการหลักของ Firebase

1. ฐานข้อมูลแบบเรียลไทม์ (Real-time Database) Firebase ให้บริการฐานข้อมูลแบบ NoSQL ที่โฮสต์อยู่บนคลาวด์ คุณสมบัติเด่นคือการซิงโครไนซ์ข้อมูลแบบเรียลไทม์ (Real-time Sync) เมื่อข้อมูลในฐานข้อมูลมีการเปลี่ยนแปลง ข้อมูลบนอุปกรณ์ของผู้ใช้ทุกคนที่เชื่อมต่ออยู่จะถูกอัปเดตตามไปด้วยทันที
2. ระบบยืนยันตัวตน (Firebase Authentication) เป็นบริการที่ช่วยจัดการระบบผู้ใช้ได้อย่างปลอดภัยและสะดวก รองรับการยืนยันตัวตนผ่านหลายช่องทาง เช่น อีเมลและรหัสผ่าน หรือบัญชีโซเชียลมีเดียต่างๆ โดยมีชุดคำสั่ง (SDK) และ UI สำเร็จรูปที่ช่วยลดความซับซ้อนในการพัฒนา
3. สถาปัตยกรรมที่ขยายขนาดได้ (Scalable Architecture) โครงสร้างพื้นฐานของ Firebase ถูกออกแบบมาให้สามารถรองรับการขยายตัวของแอปพลิเคชันได้โดยอัตโนมัติ ตั้งแต่การใช้งานในกลุ่มเล็กๆ ไปจนถึงการรองรับผู้ใช้งานจำนวนมาก

จากการศึกษาคุณสมบัติและบริการต่างๆ ของ Firebase จึงได้นำมาประยุกต์ใช้เพื่อแก้ปัญหาและตอบสนองความต้องการนี้ ประยุกต์ใช้ในฐานะ Backend-as-a-Service (BaaS) ความรู้ที่ว่า Firebase เป็นแพลตฟอร์ม BaaS ถูกนำมาใช้เพื่อลดความซับซ้อนและระยะเวลาในการพัฒนา โดยโครงการไม่จำเป็นต้องสร้างและดูแลเซิร์ฟเวอร์หลังบ้านเอง ทำให้สามารถทุ่มเททรัพยากรไปที่การพัฒนาฟังก์ชันหลักของแอปพลิเคชัน ประยุกต์ใช้ Firebase Authentication มาใช้ในการสร้างระบบสมาชิกเพื่อให้ผู้ใช้สามารถลงทะเบียนและเข้าสู่ระบบได้อย่างปลอดภัย ประยุกต์ใช้ Real-time Database ความสามารถในการซิงค์ข้อมูลแบบเรียลไทม์ถูกนำมาประยุกต์ใช้เพื่อทำ Two-Way Sync ซึ่งจำเป็นอย่างยิ่งสำหรับการบันทึกและปรับปรุงข้อมูลของผู้ใช้บนคลาวด์ทำให้ข้อมูลของผู้ใช้มีความถูกต้องและเป็นปัจจุบันตรงกันในทุกอุปกรณ์

3.6 SQLite

SQLite [13] รูปที่ 3.6 เป็นฐานข้อมูลแบบฝังตัว (Embedded Database) ที่สามารถทำงานภายในอุปกรณ์โดยตรงโดยไม่ต้องอาศัยเซิร์ฟเวอร์ ทำให้เหมาะสำหรับการจัดเก็บข้อมูลภายในแอปพลิเคชันโดยเฉพาะในกรณีที่ต้องการ การเข้าถึงข้อมูลแบบออฟไลน์ SQLite มีโครงสร้างเป็น Relational Database Management System (RDBMS) ที่รองรับการจัดเก็บและเรียกใช้ข้อมูลผ่าน SQL (Structured Query Language) ซึ่งช่วยให้สามารถ เพิ่ม ลบ แก้ไข และค้นหาข้อมูลได้อย่างมีประสิทธิภาพ



รูปที่ 3.6 ฐานข้อมูล SQLite

จากการศึกษาคุณสมบัติของ SQLite การทำงานแบบออฟไลน์ฐานข้อมูลแบบฝังตัวได้ถูกนำมาประยุกต์ใช้เพื่อสร้างพื้นที่จัดเก็บข้อมูลของผู้ใช้ไว้บนอุปกรณ์โดยตรงทำให้ผู้ใช้สามารถ เรียกดู แก้ไข หรือจัดการได้ของตนเองได้ตลอดเวลาแม้จะอยู่ในสถานที่ที่ไม่มีสัญญาณอินเทอร์เน็ตก็ตาม การนำความรู้เรื่อง SQLite มาประยุกต์ใช้จึงเป็นกลยุทธ์สำคัญที่ทำให้แอปพลิเคชันมีความเสถียรและตอบสนองได้รวดเร็วโดยยึดหลัก Offline-First เพื่อสร้างประสบการณ์การใช้งานที่ดีที่สุดแก่ผู้ใช้ โดยไม่ถูกจำกัดด้วยการเชื่อมต่ออินเทอร์เน็ต

บทที่ 4 ขั้นตอนและแผนการดำเนินงาน

บทนี้กล่าวถึงลำดับขั้นตอนในการพัฒนาโครงการ NFC Deck Tracker ตั้งแต่การวิเคราะห์ความต้องการไปจนถึงการส่งมอบโครงการ โดยสามารถแบ่งออกเป็นขั้นตอนหลัก ดังนี้

4.1 ขั้นตอนการดำเนินงาน

4.1.1 ศึกษาข้อมูลและวิเคราะห์ความต้องการ

- ศึกษาความต้องการของกลุ่มเป้าหมาย (ผู้เล่นเกมการ์ด TCG) เพื่อนำมาประยุกต์ใช้ในการพัฒนา
- ศึกษาเทคโนโลยี NFC และวิธีการใช้ NFC ในแอปพลิเคชันจัดการการ์ดของเกมการ์ด

4.1.2 กำหนดขอบเขตของโครงการ

- กำหนดฟีเจอร์หลักที่แอปพลิเคชันต้องพัฒนา
- ระบุเทคโนโลยีที่ใช้ (NFC, Flutter, SQLite, Firebase, API)

4.1.3 ศึกษาผลงานและทฤษฎีที่เกี่ยวข้อง

- ศึกษาระบบการทำงานของแอปพลิเคชันที่มีอยู่แล้วและวิเคราะห์ข้อดี-ข้อเสีย
- ศึกษาการจัดการฐานข้อมูลด้วย SQLite และ Firebase
- ศึกษา Flutter Clean Architecture เพื่อใช้ในการออกแบบระบบ

4.1.4 วิเคราะห์และออกแบบระบบ

- ออกแบบ UI ของแอปพลิเคชันโดยคำนึงถึงความสะดวกในการใช้งานของผู้เล่น
- ออกแบบโครงสร้างฐานข้อมูลให้รองรับการทำงานทั้งแบบออฟไลน์ (SQLite) และออนไลน์ (Firebase)
- ออกแบบสถาปัตยกรรมซอฟต์แวร์ตามแนวทาง Clean Architecture

4.1.5 พัฒนาระบบ

- พัฒนาฟีเจอร์หลักได้แก่ ระบบสร้างและจัดการการ์ด ระบบ อ่าน-เขียน ข้อมูลบน NFC ระบบติดตามการ์ด
- เชื่อมต่อกับฐานข้อมูล SQLite และ Firebase

4.1.6 ทดสอบและปรับปรุงการทำงานของระบบ

- Module Testing ทดสอบการทำงานแต่ละโมดูล
- Integration Testing ทดสอบการทำงานร่วมกันระหว่าง NFC Database UI

4.1.7 จัดทำเอกสารโครงการ

- เตรียมเอกสารสำหรับการนำเสนอ
- สรุปผลการดำเนินโครงการ ข้อเสนอแนะสำหรับการพัฒนาเพิ่มเติม และส่งมอบโครงการ

4.2 แผนการดำเนินงาน

ตารางที่ 4.1 แผนการดำเนินงาน

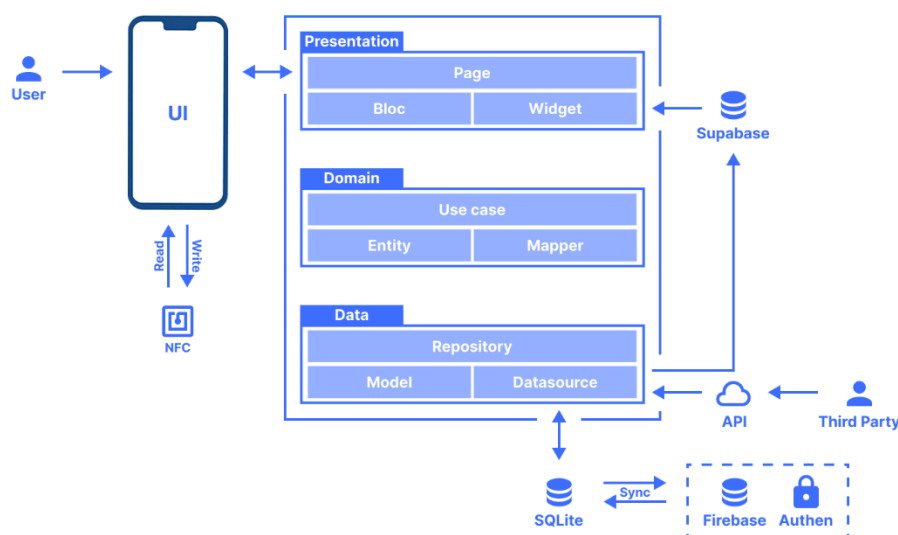
ขั้นตอนการดำเนินงาน	ต.ค. 67	พ.ย. 67	ธ.ค. 67	ม.ค. 68	ก.พ. 68	มี.ค. 68	ม.ย. 68	พ.ค. 68	มิ.ย. 68	ก.ค. 68
ศึกษาข้อมูลและ วิเคราะห์ความต้องการ	↔									
กำหนดขอบเขตของโครงการ	↔									
ศึกษาผลงานและ ทฤษฎีที่เกี่ยวข้อง	↔	↔								
วิเคราะห์และออกแบบระบบ			↔	↔						
พัฒนาระบบ				↔	↔	↔	↔	↔		
ทดสอบและปรับปรุงการ ทำงานของระบบ					↔	↔	↔	↔	↔	
จัดทำเอกสารโครงการ	↔	↔	↔	↔	↔	↔	↔	↔	↔	↔

บทที่ 5 วิธีการดำเนินงาน

บทนี้อธิบายถึงขั้นตอนการออกแบบและพัฒนาระบบแอปพลิเคชัน NFC Deck Tracker โดยจะเริ่มต้นด้วยการนำเสนอภาพรวมสถาปัตยกรรมของระบบเพื่อให้เห็นโครงสร้างหลักทั้งหมด จากนั้นจึงวิเคราะห์ความต้องการของระบบผ่านแผนภาพยูสเคสและคำอธิบายรายละเอียดเพื่อกำหนดขอบเขตและฟังก์ชันการทำงานของผู้ใช้ ต่อด้วยรายละเอียดการทำงานของส่วนประกอบย่อยต่างๆ ภายในสถาปัตยกรรมซึ่งครอบคลุมถึงการออกแบบฐานข้อมูล และปิดท้ายด้วยการออกแบบส่วนติดต่อผู้ใช้

5.1 การออกแบบสถาปัตยกรรมระบบ

สถาปัตยกรรมของระบบถูกออกแบบมาเพื่อรองรับการเชื่อมต่อและทำงานร่วมกับบริการภายนอกหลากหลาย โดยใช้สถาปัตยกรรมซอฟต์แวร์ตามหลักการ Clean Architecture ซึ่งแบ่งการทำงานออกเป็น 3 ชั้นหลักเพื่อแยกส่วนการทำงานอย่างชัดเจน ได้แก่ ชั้นของการนำเสนอ (Presentation Layer) ที่รับผิดชอบส่วนติดต่อผู้ใช้และการโต้ตอบ ชั้นของตรรกะทางธุรกิจ (Domain Layer) ซึ่งเป็นแกนกลางที่รวบรวมกฎและตรรกะของแอปพลิเคชันและ ชั้นข้อมูล (Data Layer) ที่ทำหน้าที่จัดการการเข้าถึงแหล่งข้อมูลทั้งหมด ดังรูปที่ 5.1 แผนภาพสถาปัตยกรรมโดยรวมของระบบ

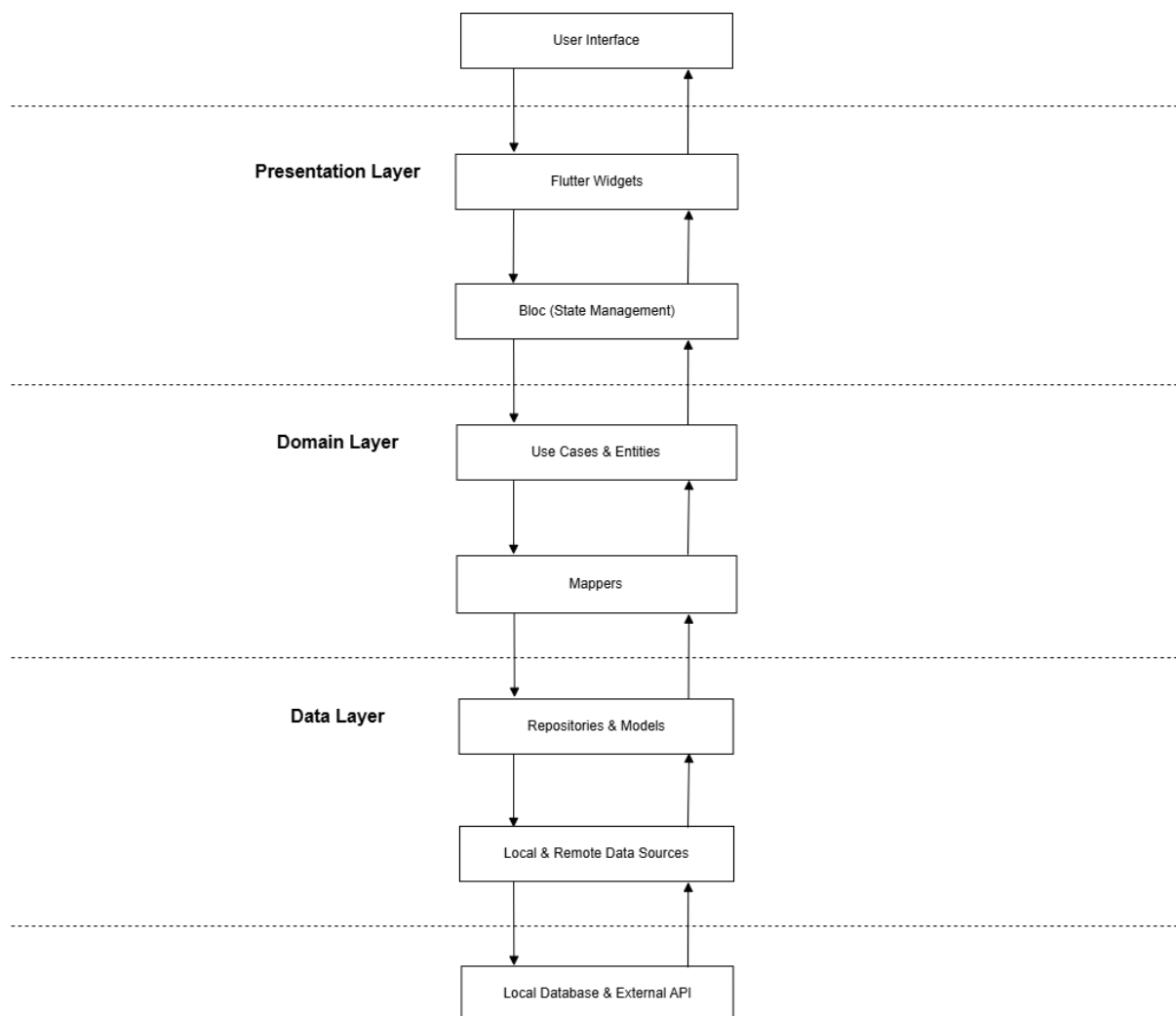


รูปที่ 5.1 แผนภาพสถาปัตยกรรมของระบบที่ออกแบบตามหลักการ Clean Architecture

การทำงานของระบบจะเชื่อมต่อกับบริการและส่วนประกอบภายนอกเริ่มจากผู้ใช้สามารถโต้ตอบกับแอปพลิเคชันและใช้งานแท็ก NFC เพื่อบันทึกและอ่านข้อมูลการ์ดได้โดยตรง นอกจากนี้ระบบยังแสดงข้อมูลของการ์ดจากเกมต่าง ๆ โดยดึงข้อมูลมาจาก API ภายนอกที่ให้บริการเพื่อนำมาใช้งานในแอปพลิเคชัน โดยข้อมูลหลักที่ใช้งานในแอปพลิเคชันจะถูกจัดเก็บลงใน SQLite ซึ่งเป็นฐานข้อมูลภายในเครื่องเพื่อรองรับการใช้งานแบบออฟไลน์ และเมื่อผู้ใช้เข้าสู่ระบบด้วยบัญชี Google ระบบจะใช้ Firebase เป็นพื้นที่จัดเก็บข้อมูลบนคลาวด์ พร้อมทั้งมีการทำงานแบบ Two-Way Sync เพื่อให้ข้อมูลระหว่างคลาวด์และในเครื่องตรงกันเสมอ ในส่วนของ Supabase ทำหน้าที่เป็น ฐานข้อมูลในการจัดการกับรูปภาพเท่านั้นโดยเฉพาะการ์ดที่ผู้ใช้สร้างขึ้นเองซึ่งแอปพลิเคชันจะใช้งานโดยการดึง Image URL มาแสดงผล

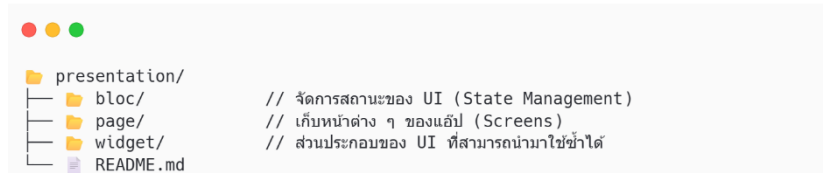
5.2 การออกแบบสถาปัตยกรรมซอฟต์แวร์

การออกแบบสถาปัตยกรรมซอฟต์แวร์สำหรับแอปพลิเคชันนี้ได้นำหลักการ Clean Architecture มาประยุกต์ใช้โดยมีหัวใจสำคัญคือการแบ่งโครงสร้างของโค้ดออกเป็นชั้น (Layers) ที่มีความรับผิดชอบชัดเจนและไม่ขึ้นต่อกัน เพื่อให้ง่ายต่อการพัฒนา ทดสอบ และบำรุงรักษาในระยะยาว กระบวนการทำงานโดยรวมจะเริ่มต้นจาก User Interface ซึ่งเป็นส่วนที่รับอินพุตจากผู้ใช้และส่งคำขอไปยัง Presentation Layer จากนั้นคำขอจะถูกประมวลผลผ่าน Domain Layer ซึ่งเป็นแกนหลักของตรรกะทางธุรกิจ ก่อนจะสื่อสารกับ Data Layer เพื่อดึงหรือบันทึกข้อมูลจากแหล่งข้อมูลทั้งภายในและภายนอก (Local & Remote Data Sources) สุดท้ายข้อมูลจะถูกส่งกลับมายัง User Interface เพื่อแสดงผลให้ผู้ใช้ในลักษณะที่เข้าใจและเป็นมิตรต่อการใช้งาน ดังรูปที่ 5.2 กระบวนการทำงานระหว่างเลเยอร์แสดง



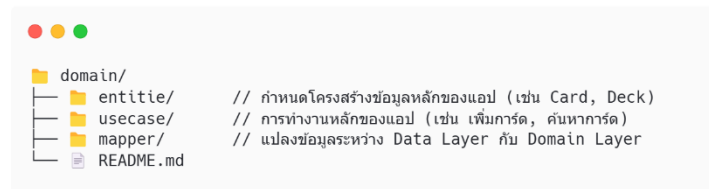
รูปที่ 5.2 แผนภาพแสดงกระบวนการทำงานระหว่างเลเยอร์ตามสถาปัตยกรรมซอฟต์แวร์

Presentation Layer รูปที่ 5.3 ทำหน้าที่จัดการส่วนติดต่อผู้ใช้ (UI) และการโต้ตอบทั้งหมดของแอปพลิเคชัน โดยมีส่วนประกอบหลักคือ Pages ซึ่งเปรียบเสมือนหน้าจอต่าง ๆ ที่ผู้ใช้มองเห็น ในแต่ละ Page เป็นการนำ Flutter Widgets มาประกอบกันเพื่อสร้างหน้าตาและการแสดงผลข้อมูล และใช้ Flutter Bloc ในการควบคุมสถานะและจัดการกับตรรกะของ UI หน้านั้น ๆ การแยกส่วนประกอบแบบนี้ช่วยให้สามารถแสดงข้อมูลได้แบบเรียลไทม์ ลดความซับซ้อนของโค้ด และทำให้การจัดการสถานะเป็นระบบที่ควบคุมได้ง่ายยิ่งขึ้น



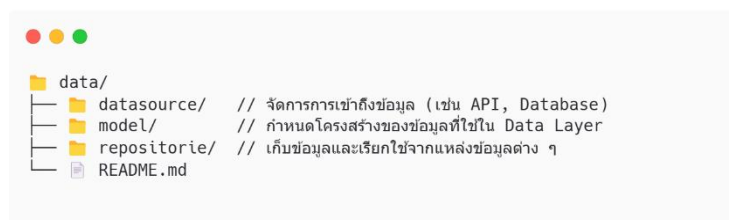
รูปที่ 5.3 โครงสร้างไดเรกทอรีของ Presentation Layer

Domain Layer รูปที่ 5.4 ทำหน้าที่เป็นศูนย์กลางของตรรกะทางธุรกิจของแอปพลิเคชัน โดยมี Entities ซึ่งเป็นโมเดลข้อมูลหลักที่ใช้ในระบบ และ Use Cases ที่ควบคุมการทำงานของแอป เช่น การเพิ่มการ์ดลงเด็ค การสแกนแท็ก NFC และการซิงค์ข้อมูลกับ Firebase นอกจากนี้ยังมี Mappers ซึ่งใช้ในการแปลงข้อมูลจาก Data Layer ให้อยู่ในรูปแบบที่สอดคล้องกับ Entities ใน Domain Layer ช่วยให้ข้อมูลที่ได้รับมีถูกต้องและอยู่ในรูปแบบที่เหมาะสมต่อการใช้งานภายในแอป



รูปที่ 5.4 โครงสร้างไดเรกทอรีของ Domain Layer

Data Layer รูปที่ 5.5 ทำหน้าที่จัดการข้อมูลทั้งหมดของแอปพลิเคชัน โดยมี Repositories เป็นตัวกลางระหว่าง Domain Layer และ Data Sources ทำให้การเรียกใช้งานข้อมูลจากแหล่งต่าง ๆ เป็นไปอย่างเป็นระบบ Data Sources อาจเป็น Firebase สำหรับการซิงค์ข้อมูลแบบเรียลไทม์ SQLite สำหรับการจัดเก็บข้อมูลแบบออฟไลน์หรือ API สำหรับการดึงข้อมูลจากภายนอก ข้อมูลที่ได้จากแหล่งข้อมูลเหล่านี้จะถูกเก็บใน Models ซึ่งเป็นโครงสร้างข้อมูลที่ได้รับมาก่อนที่จะถูก Mappers นำไปแปลงให้อยู่ในรูปแบบของ Domain Layer



รูปที่ 5.5 โครงสร้างไดเรกทอรีของ Data Layer

5.3 ตัวอย่างการทำงานของเลเยอร์ผ่านพีเจอร์ทดการตั้งค่า

เพื่อให้เห็นภาพรวมของการแบ่งเลเยอร์ตามแนวคิด Clean Architecture ที่ใช้ในแอปพลิเคชัน NFC Deck Tracker บทนี้จึงยกตัวอย่างจากหนึ่งในพีเจอร์ทดการตั้งค่าที่สำคัญของแอป คือ ระบบการตั้งค่าทั่วไปของผู้ใช้งาน (App Settings) ซึ่งประกอบด้วยการจัดการค่าต่าง ๆ เช่น ภาษา ธีม และสถานะการเข้าสู่ระบบ โดยตัวอย่างนี้จะแสดงให้เห็นการประสานงานกันของแต่ละเลเยอร์ ได้แก่ Presentation Layer, Domain Layer และ Data Layer อย่างชัดเจน ผ่านกระบวนการไหลต บันทึกลง และอัปเดตค่าการตั้งค่าโดยมีรายละเอียดในหัวข้อย่อยต่อไปนี้

5.3.1 การจัดการ State ด้วย Bloc

ApplicationBloc ถูกใช้ในชั้น Presentation Layer สำหรับจัดการค่าการตั้งค่าของผู้ใช้ เช่น ภาษา (locale) โดยจะควบคุมและเก็บสถานะทั้งหมดไว้ใน ApplicationState และสามารถอัปเดตค่าผ่านเมธอด updateSetting() ซึ่งประสานงานกับ Usecase ในชั้น Domain Layer เพื่อโหลดหรือบันทึกข้อมูล แล้วนำมาส่งกลับมายัง UI ด้วยการ emit state ใหม่ทันที ดังรูปที่ 5.6 โค้ดตัวอย่างของ ApplicationBloc สำหรับจัดการสถานะของการตั้งค่า

```
class ApplicationBloc extends Bloc<ApplicationEvent, ApplicationState> {
  final ClearUserDataUsecase clearUserDataUsecase;
  final InitSettingUsecase initSettingUsecase;
  final UpdateSettingUsecase updateSettingUsecase;

  ApplicationBloc({
    required this.clearUserDataUsecase,
    required this.initSettingUsecase,
    required this.updateSettingUsecase,
  }) : super(ApplicationState.initial()) {
    on<InitApplicationEvent>(_onInitApplication);
    on<UpdateSettingEvent>(_onUpdateSetting);
    on<SetPageIndexEvent>(_onSetPageIndex);
    on<ClearUserDataEvent>(_onClearUserData);
  }

  Future<void> _onUpdateSetting(
    UpdateSettingEvent event,
    Emitter<ApplicationState> emit,
  ) async {
    await updateSettingUsecase.call(key: event.key, value: event.value);
    emit(_mapUpdatedState(state, event.key, event.value));
  }

  ApplicationState _mapUpdatedState(ApplicationState state, String key, dynamic value) {
    switch (key) {
      case AppConfig.keyLocale:
        return state.copyWith(locale: Locale(value));
      case AppConfig.keyIsDark:
        return state.copyWith(isDark: value);
      case AppConfig.keyGuestId:
        return state.copyWith(guestId: value);
      case AppConfig.keyRecentId:
        return state.copyWith(recentId: value);
      case AppConfig.keyRecentGame:
        return state.copyWith(recentGame: value);
      case AppConfig.keyTutorial:
        return state.copyWith(tutorialNfcIcon: value);
      default:
        return state;
    }
  }
}
```

รูปที่ 5.6 โค้ดตัวอย่างของ ApplicationBloc สำหรับจัดการสถานะของการตั้งค่า

5.3.2 การใช้งาน Usecase

Usecase ทำหน้าที่เป็นตัวกลางระหว่าง Presentation Layer กับ Domain Layer โดยรับคำสั่งจาก Bloc แล้วส่งต่อไปยัง Repository เพื่อประมวลผลข้อมูลหรือดำเนินการใด ๆ ที่เกี่ยวข้องกับตรรกะทางธุรกิจ (Business Logic) ในกรณีของแอปพลิเคชันนี้ UpdateSettingUsecase ดังรูปที่ 5.7 ทำหน้าที่รับค่าการตั้งค่าใหม่จากผู้ใช้แล้วส่งคำสั่งให้ Repository บันทึกข้อมูลนั้นต่อไป

```
import 'package:nfc_deck_tracker/data/repository/update_setting.dart';

class UpdateSettingUsecase {
  final UpdateSettingRepository updateSettingRepository;

  UpdateSettingUsecase({
    required this.updateSettingRepository,
  });

  Future<void> call({
    required String key,
    required dynamic value,
  }) async {
    await updateSettingRepository.update(key: key, value: value);
  }
}
```

รูปที่ 5.7 โค้ดตัวอย่างของ UpdateSettingUsecase สำหรับจัดการตรรกะการอัปเดตค่าตั้งค่า

5.3.3 การเข้าถึงข้อมูลผ่าน Repository

Repository คือชั้นเชื่อมต่อระหว่าง Domain Layer กับ Data Layer ซึ่งทำหน้าที่รับคำสั่งจาก Usecase และจัดการกับแหล่งข้อมูล เช่น SQLite หรือ Firebase โดยตรง ในตัวอย่างนี้ UpdateSettingRepository ดังรูปที่ 5.8 รับคำสั่งจาก Usecase แล้วส่งต่อไปยัง Data Source ที่ทำหน้าที่จัดเก็บค่าการตั้งค่าของผู้ใช้

```
import '../datasource/local/update_setting.dart';

class UpdateSettingRepository {
  final UpdateSettingLocalDatasource updateSettingLocalDatasource;

  UpdateSettingRepository({
    required this.updateSettingLocalDatasource,
  });

  Future<void> update({
    required String key,
    required dynamic value,
  }) async {
    await updateSettingLocalDatasource.update(key: key, value: value);
  }
}
```

รูปที่ 5.8 โค้ดตัวอย่างของ UpdateSettingRepository สำหรับเป็นตัวกลางในการส่งต่อข้อมูล

5.3.4 การจัดการข้อมูลใน Datasource

Datasource เป็นชั้นที่อยู่ล่างสุดในโครงสร้าง Clean Architecture โดยมีหน้าที่หลักในการติดต่อและเชื่อมต่อกับแหล่งข้อมูลจริง ไม่ว่าจะเป็นแหล่งข้อมูลภายในเครื่อง (Local) เช่น ฐานข้อมูลภายในหรือไฟล์ที่จัดเก็บในอุปกรณ์ผู้ใช้หรือแหล่งข้อมูลจากภายนอก (Remote) อย่างเช่น API หรือฐานข้อมูลบนเซิร์ฟเวอร์ ในกรณีนี้ UpdateSettingLocalDatasource ดังรูปที่ 5.9 ทำหน้าที่รับคำสั่งและส่งต่อไปยังส่วนที่จัดการข้อมูลจริง สำหรับการจัดเก็บข้อมูลการตั้งค่าซึ่งเป็นข้อมูลแบบ key-value ภายในเครื่องใช้ SharedPreferencesService ดังรูปที่ 5.10 เป็นตัวกลางในการบันทึกข้อมูลลงในหน่วยความจำของอุปกรณ์

```
import '@shared_preferences_service.dart';

class UpdateSettingLocalDatasource {
  final SharedPreferencesService _sharedPreferencesService;

  UpdateSettingLocalDatasource(this._sharedPreferencesService);

  Future<void> update({
    required String key,
    required dynamic value,
  }) async {
    await _sharedPreferencesService.save(key: key, value: value);
  }
}
```

รูปที่ 5.9 โค้ดตัวอย่างของ UpdateSettingLocalDatasource สำหรับการเชื่อมต่อกับแหล่งข้อมูล

```
class SharedPreferencesService {
  final SharedPreferences _sharedPreferences;

  SharedPreferencesService(this._sharedPreferences);

  Future<void> save({
    required String key,
    required dynamic value,
  }) async {
    try {
      bool success = false;

      switch (value.runtimeType) {
        case String:
          success = await _sharedPreferences.setString(key, value);
          break;
        case int:
          success = await _sharedPreferences.setInt(key, value);
          break;
        case bool:
          success = await _sharedPreferences.setBool(key, value);
          break;
        default:
      }

      if (success) {
        debugPrint('🟢 Saved value for key "$key" value "$value" successfully');
      } else {
        debugPrint('❌ Failed to save value for key "$key"');
      }
    } catch (e) {
      debugPrint('❌ Failed to save string for key "$key": $e');
    }
  }
}
```

รูปที่ 5.10 โค้ดตัวอย่างของ SharedPreferencesService สำหรับการบันทึกข้อมูลลงอุปกรณ์

5.4 การจัดการ Dependency Injection ด้วย Service Locator

เพื่อให้การจัดการการเรียกใช้งานบริการ (Service) ต่าง ๆ ภายในแอปพลิเคชันเป็นไปอย่างมีประสิทธิภาพ และสามารถแยกการทำงานของแต่ละเลเยอร์ออกจากกันได้อย่างชัดเจน ระบบจึงได้นำแนวคิด Dependency Injection (DI) มาประยุกต์ใช้ร่วมกับแพ็คเกจ GetIt ซึ่งเป็น Service Locator ที่นิยมใช้ในแอป Flutter

โครงสร้างของระบบได้แยกส่วนที่เกี่ยวข้องกับ DI ไว้ในเลเยอร์ /core/service โดยมีไฟล์หลัก locator.dart สำหรับกำหนดและลงทะเบียนบริการทั้งหมดที่แอปพลิเคชันต้องใช้งาน เช่น ฐานข้อมูล SQLite, SharedPreferences, Repository, Usecase, BloC และอื่น ๆ เพื่อให้สามารถเรียกใช้งานได้จากส่วนอื่นของระบบอย่างสะดวกและเป็นระบบเดียวกัน ทั้งยังช่วยให้สามารถจัดการและตรวจสอบความผิดพลาดได้ง่ายในทีเดียว โดยมีไฟล์ service_locator.dart ดังรูปที่ 5.11 เป็นศูนย์กลางในการตั้งค่าบริการทั้งหมด

```
final GetIt locator = GetIt.instance;

Future<void> setupLocator() async {
  try {
    debugPrint('⚙️ Setting up Service Locator...');

    await setupDatabase();
    await setupSqlite();
    await setupSharedPreferences();
    await setupLocalDataSource();
    await setupRemoteDataSource();
    await setupRepository();
    await setupUsecase();
    await setupCubit();
    await setupMisc();
    await locator.allReady();

    debugPrint('🟢 Service Locator setup completed successfully.');
```

รูปที่ 5.11 โค้ดตัวอย่างของ setupLocator สำหรับเป็นศูนย์กลางในการลงทะเบียนบริการทั้งหมด

ฟังก์ชัน setupLocator() ดังรูปที่ 5.12 จะมีการเรียกใช้ฟังก์ชันย่อยเพื่อลงทะเบียนแต่ละบริการ (Service) ให้เสร็จสิ้นในลำดับที่ถูกต้อง ตัวอย่างเช่น การลงทะเบียน SharedPreferencesService จะถูกแยกไปอยู่ในไฟล์ของตนเองเพื่อความเป็นระเบียบ ในแต่ละฟังก์ชันย่อยมีการใช้ locator.registerSingletonAsync() เพื่อลงทะเบียน Service และใช้ try-catch เพื่อป้องกันข้อผิดพลาด พร้อมทั้งแสดงข้อความสถานะผ่าน debugPrint เพื่อให้ง่ายต่อการตรวจสอบ

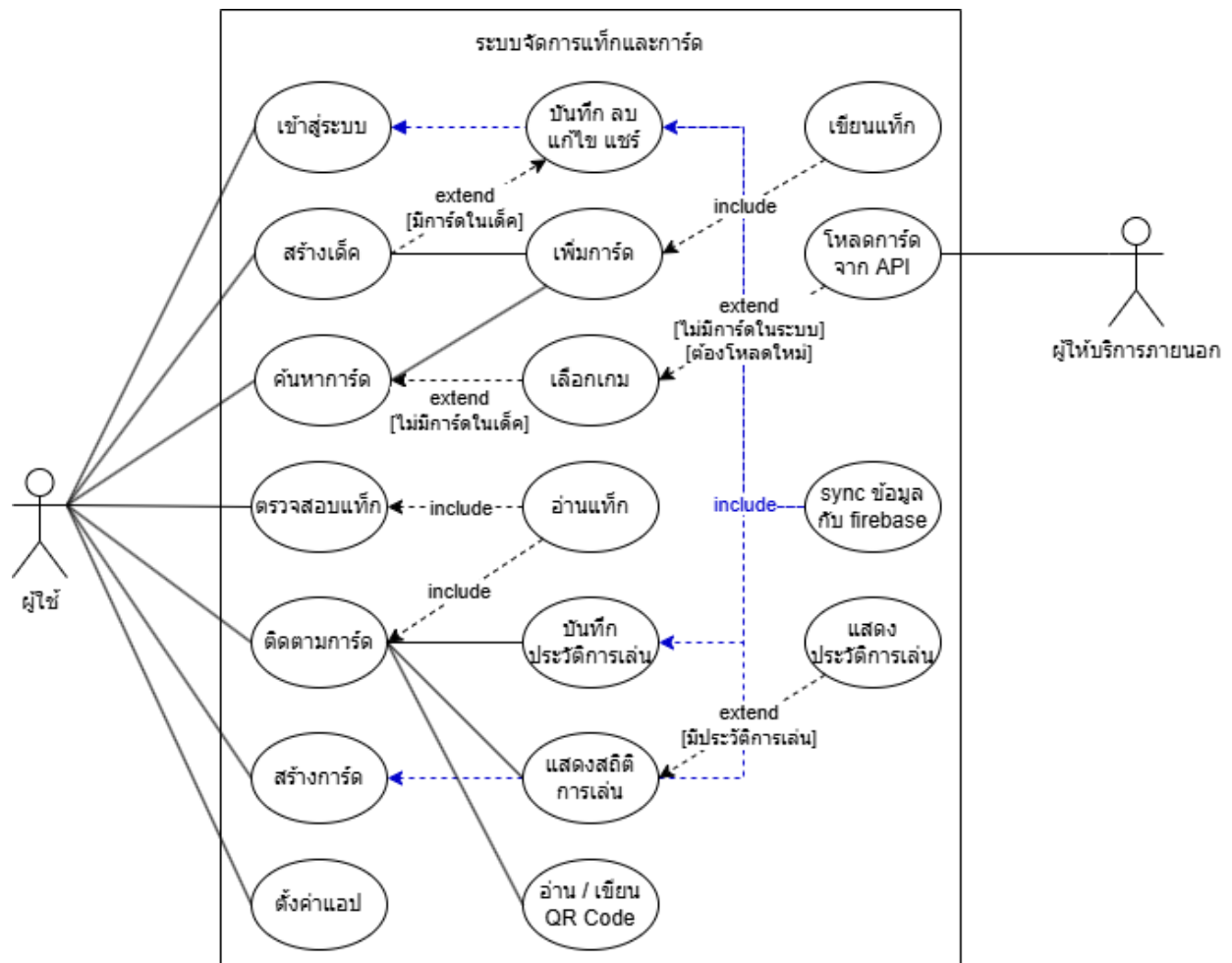
```
Future<void> setupSharedPreferences() async {
  try {
    locator.registerSingletonAsync<SharedPreferencesService>(() async {
      final sharedPreferences = await SharedPreferences.getInstance();
      return SharedPreferencesService(sharedPreferences);
    });

    debugPrint('✔️ SharedPreferences registered successfully.');
```

รูปที่ 5.12 โค้ดตัวอย่างของ setupSharedPreferences สำหรับลงทะเบียน SharedPreferencesService

5.5 การกำหนดขอบเขตระบบด้วย Use Case Diagram

ในการออกแบบระบบ NFC Deck Tracker ได้มีการวิเคราะห์ความต้องการเพื่อกำหนดขอบเขตและฟังก์ชันหลักของบริการโดยใช้ Use Case Diagram ดังรูปที่ 5.13 เป็นเครื่องมือแสดงปฏิสัมพันธ์ระหว่างผู้ใช้งาน (Actor) และระบบ จากแผนภาพจะเห็นว่าระบบมีผู้ใช้งานหลักคือ “ผู้ใช้” ซึ่งสามารถดำเนินกิจกรรมหลักต่าง ๆ เช่น การจัดการเด็กและการ์ด (สร้าง ค้นหา ติดตาม) การบันทึกและวิเคราะห์สถิติการเล่น และการอ่าน/เขียนข้อมูลผ่าน NFC นอกจากนี้ระบบยังมีการเชื่อมต่อกับ “ผู้ให้บริการภายนอก” และมีกระบวนการเบื้องหลัง เช่น การซิงค์ข้อมูลกับ Firebase



รูปที่ 5.13 Use Case Diagram แสดงขอบเขตการทำงานของระบบ

ตารางที่ 5.1 รายละเอียด Use Case #01 เข้าสู่ระบบ

Use Case #01	เข้าสู่ระบบ
Actor	ผู้ใช้
Description	ผู้ใช้สามารถเข้าสู่ระบบด้วยบัญชี Google หรือใช้งานในโหมด Guest
Pre-conditions	ผู้ใช้เปิดแอปพลิเคชันขึ้นมา และยังไม่ได้อยู่ในสถานะเข้าสู่ระบบ
Post-conditions	ผู้ใช้เข้าสู่ระบบสำเร็จ และระบบนำทางผู้ใช้ไปยังหน้า "My Decks"
Main Flow	1. ผู้ใช้เปิดแอปพลิเคชัน 2. ระบบตรวจสอบสถานะและพบว่าผู้ใช้อย่างไรยังไม่ได้เข้าสู่ระบบ 3. ระบบแสดงหน้าจอให้เลือกวิธีการเข้าสู่ระบบ ระหว่าง "Sign in with Google" และ "Guest" 4. ผู้ใช้เลือก "Sign in with Google" 5. ระบบแสดงหน้าต่างสำหรับยืนยันตัวตนผ่านบัญชี Google 6. ผู้ใช้ทำการยืนยันตัวตนสำเร็จ 7. ระบบบันทึกสถานะการเข้าสู่ระบบของผู้ใช้ 8. ระบบนำทางผู้ใช้ไปยังหน้า "My Decks"
Alternative Flow	กรณีผู้ใช้เลือกเข้าสู่ระบบแบบ Guest 4. ผู้ใช้เลือก "Guest" 5. ระบบสร้างสถานะผู้ใช้ชั่วคราว Guest 6. ระบบนำทางผู้ใช้ไปยังหน้า "My Decks" กรณีการยืนยันตัวตนล้มเหลว 6. การยืนยันตัวตนผ่าน Google ล้มเหลว 7. ระบบแสดงข้อความแจ้งเตือนข้อผิดพลาด 8. ผู้ใช้ยังคงอยู่ที่หน้าจอเข้าสู่ระบบ กรณีไม่มีอินเทอร์เน็ต 3. ระบบแสดงหน้าจอให้เลือกวิธีการเข้าสู่ระบบ "Guest" เท่านั้น 4. ระบบสร้างสถานะผู้ใช้ชั่วคราว Guest 5. ระบบนำทางผู้ใช้ไปยังหน้า "My Decks"

ตารางที่ 5.2 รายละเอียด Use Case #02 สร้างเต็ค

Use Case #02	สร้างเต็ค
Actor	ผู้ใช้
Description	ผู้ใช้สามารถสร้างเต็คใหม่
Pre-conditions	ผู้ใช้เข้าสู่ระบบเรียบร้อยแล้ว และอยู่ในหน้า "My Decks"
Post-conditions	เต็คใหม่พร้อมการ์ดที่เลือกถูกบันทึกสำเร็จ และแสดงอยู่ในรายการของหน้า "My Decks"
Main Flow	1. ผู้ใช้อยู่ที่หน้า "My Decks" 2. ผู้ใช้กดไอคอน "สร้างเต็ค" 3. ระบบแสดงหน้า "Deck Builder" ที่เป็นเต็คว่างเปล่า และให้ผู้ใช้ตั้งชื่อเต็ค 4. ผู้ใช้กดปุ่ม "+" เพื่อเพิ่มการ์ด 5. ระบบนำทางไปยังหน้ารายการเกม/คอลเลคชัน 6. ผู้ใช้เลือกเกมที่สนใจ 7. ระบบแสดงหน้ารายการการ์ดของเกมนั้น 8. ผู้ใช้ค้นหาการ์ดที่ต้องการผ่าน Search Bar 9. ผู้ใช้เลือกการ์ดและกำหนดจำนวนที่ต้องการเพิ่มเข้าเต็ค 10. ผู้ใช้ทำการค้นหาและเพิ่มการ์ด (ทำซ้ำขั้นตอนที่ 8-9) จนพอใจ 11. ผู้ใช้กลับมาที่หน้า "Deck Builder" ซึ่งตอนนี้มีรายการการ์ดที่เลือกไว้แสดงอยู่ 12. ผู้ใช้กดปุ่ม "บันทึก" 13. ระบบทำการบันทึกเต็คใหม่ลงในฐานข้อมูล 14. ระบบนำผู้ใช้กลับไปยังหน้า "My Decks"
Alternative Flow	กรณีที่ยังไม่มีข้อมูลเกมในเครื่อง 6. ณ ขั้นตอนที่ 6 หากผู้ใช้เลือกเกมที่ยังไม่มีข้อมูลในระบบ 7. ระบบจะทำการดึงข้อมูลการ์ดจาก API ภายนอก และบันทึกลง SQLite ในเครื่องโดยอัตโนมัติ กรณีผู้ใช้ยกเลิกการสร้างเต็ค 12. ณ ขั้นตอนใด ๆ ก่อนการกดบันทึก ผู้ใช้กดย้อนกลับไปยังหน้า "My Decks" ระบบยกเลิกการสร้างเต็ค และข้อมูลที่เลือกไว้จะไม่ถูกบันทึก

ตารางที่ 5.3 รายละเอียด Use Case #03 แก้ไขเด็ก

Use Case #03	แก้ไขเด็ก
Actor	ผู้ใช้
Description	ผู้ใช้สามารถเปลี่ยนชื่อเด็กหรือการ์ดที่อยู่ในเด็ก
Pre-conditions	1. ผู้ใช้เข้าสู่ระบบเรียบร้อยแล้ว และอยู่ในหน้า "My Decks" 2. มีเด็กที่ต้องการแก้ไขอยู่ในรายการ
Post-conditions	ข้อมูลของเด็ก (ชื่อ หรือรายการการ์ด) ได้รับการอัปเดตและบันทึกเรียบร้อยแล้ว
Main Flow	1. ผู้ใช้อยู่ที่หน้า "My Decks" 2. ผู้ใช้แตะที่เด็กที่ต้องการแก้ไข 3. ระบบนำทางไปยังหน้า "Deck Builder" และแสดงข้อมูลปัจจุบันของเด็กที่เลือก 4. ผู้ใช้ทำการแก้ไขตามต้องการ เช่น: - แตะที่ชื่อเด็กเพื่อเปลี่ยนชื่อใหม่ - กดปุ่ม "+" เพื่อเพิ่มการ์ดใหม่ (ตามขั้นตอนของ Use Case "สร้างเด็ก") - กดที่การ์ดในเด็กเพื่อลดจำนวนหรือลบออก 5. เมื่อแก้ไขจนพอใจ ผู้ใช้กดปุ่ม "บันทึก" 6. ระบบทำการบันทึกข้อมูลเด็กที่อัปเดตแล้วลงฐานข้อมูล 7. ระบบนำผู้ใช้กลับไปยังหน้า "My Decks"
Alternative Flow	

ตารางที่ 5.4 รายละเอียด Use Case #04 แשר์เด็ก

Use Case #04	แשר์เด็ก
Actor	ผู้ใช้
Description	ผู้ใช้สามารถแשר์เด็กโดยมีรายชื่อการ์ดในเด็กและจำนวนการ์ดที่ใช้ผ่านคลิปปอร์ต
Pre-conditions	1. ผู้ใช้เข้าสู่ระบบเรียบร้อยแล้ว และอยู่ในหน้า "My Decks" 2. มีเด็กที่ต้องการแก้ไขอยู่ในรายการ 3. มีการ์ดในเด็ก
Post-conditions	รายชื่อการ์ดและจำนวนในเด็กถูกคัดลอกไปยังคลิปปอร์ตของอุปกรณ์เรียบร้อยแล้ว
Main Flow	1. ผู้ใช้ไปที่หน้า "My Decks" และเลือกเด็กที่ต้องการแשר์ 2. ระบบแสดงรายละเอียดของเด็กในหน้า "Deck Builder" 3. ผู้ใช้กดไอคอน "แשר์" 4. ระบบทำการสร้างข้อความที่เป็นรายชื่อการ์ดทั้งหมดพร้อมจำนวนในเด็กนั้น 5. ระบบคัดลอกข้อความดังกล่าวไปยังคลิปปอร์ตของอุปกรณ์ 6. ระบบแสดงข้อความยืนยัน
Alternative Flow	

ตารางที่ 5.5 รายละเอียด Use Case #05 ลบเด็ก

Use Case #05	ลบเด็ก
Actor	ผู้ใช้
Description	ผู้ใช้สามารถลบเด็กที่สร้างไว้หากไม่ต้องการใช้งานแล้ว
Pre-conditions	1. ผู้ใช้เข้าสู่ระบบเรียบร้อยแล้ว และอยู่ในหน้า "My Decks" 2. มีเด็กที่ต้องการแก้ไขอยู่ในรายการ
Post-conditions	เด็กที่ผู้ใช้เลือกถูกลบออกจากระบบ และไม่แสดงในหน้า "My Decks" อีกต่อไป
Main Flow	1. ผู้ใช้อยู่ที่หน้า "My Decks" 2. ผู้ใช้กดไอคอน "ปากกา" เพื่อเข้าสู่โหมดแก้ไข 3. ระบบแสดงไอคอน "กากบาท" บนเด็กแต่ละรายการ 4. ผู้ใช้กดไอคอน "กากบาท" บนเด็กที่ต้องการจะลบ 5. ระบบทำการลบข้อมูลเด็กที่เลือกออกจากฐานข้อมูล 6. ระบบรีเฟรชหน้า "My Decks" และเด็กที่ถูกลบได้หายไปจากรายการ
Alternative Flow	กรณียกเลิกการลบ 2. ณ ขั้นตอนที่ 2 ผู้ใช้กดไอคอน "ปากกา" เพื่อออกจากโหมดแก้ไข

ตารางที่ 5.6 รายละเอียด Use Case #06 ตรวจสอบแท็ก

Use Case #06	ตรวจสอบแท็ก
Actor	ผู้ใช้
Description	ผู้ใช้สามารถตรวจสอบแท็กที่เคยเขียนข้อมูลการ์ดว่าเป็นการ์ดอะไร
Pre-conditions	1. ผู้ใช้ได้เปิดแอปพลิเคชัน และอยู่ในหน้าอ่านแท็ก 2. ผู้ใช้มีแท็ก NFC ที่ได้บันทึกข้อมูลการ์ดไว้แล้ว
Post-conditions	ระบบแสดงข้อมูลของการ์ดที่ถูกบันทึกไว้บนแท็กให้ผู้ใช้เห็น
Main Flow	1. ผู้ใช้อยู่ในหน้าที่พร้อมอ่านแท็ก 2. ผู้ใช้นำแท็ก NFC มาแตะที่บริเวณ NFC ของสมาร์ทโฟน 3. ระบบทำการอ่านข้อมูล (รหัสเกมและรหัสการ์ด) จากแท็ก 4. ระบบนำรหัสการ์ดที่อ่านได้ไปค้นหาข้อมูลการ์ดฉบับเต็ม จากฐานข้อมูลในเครื่อง 5. ระบบแสดงผลข้อมูลการ์ดฉบับเต็มบนหน้าจอให้ผู้ใช้ทราบ
Alternative Flow	กรณีแท็กไม่มีข้อมูล 3. ณ ขั้นตอนที่ 3 ระบบอ่านแท็กแล้วพบว่าไม่มีข้อมูลการ์ดบันทึกอยู่ 4. ระบบแสดงข้อความแจ้งเตือน "ไม่พบข้อมูลบนแท็ก" กรณีไม่สามารถอ่านแท็กได้ 3. ณ ขั้นตอนที่ 3 เกิดข้อผิดพลาดในการอ่าน 4. ระบบแสดงข้อความแจ้งเตือน "ไม่สามารถอ่านแท็กได้ กรุณาลองอีกครั้ง" 5. ระบบกลับสู่สถานะพร้อมสแกน (ขั้นตอนที่ 1)

ตารางที่ 5.7 รายละเอียด Use Case #07 ติดตามการ์ด

Use Case #07	ติดตามการ์ด
Actor	ผู้ใช้
Description	เมื่อผู้ใช้เล่นเกมสามารถติดตามสถานะการ์ดที่เหลืออยู่ในเต็คแบบเรียลไทม์
Pre-conditions	1. ผู้ใช้เข้าสู่หน้าจอ "Deck builder" ของเต็คนั้นแล้ว 2. ผู้ใช้กดไอคอน "เล่น" เพื่อไปยังหน้า "Deck Tracker" 3. การ์ดในเต็คจริงของผู้ใช้ได้ถูกติดตั้งและบันทึกข้อมูลลงบน NFC Tag แล้ว
Post-conditions	1. สถานะของเต็คในแอปพลิเคชันตรงกับสถานะของเต็คในเกมจริง 2. ประวัติการเล่นของแมตช์นั้นถูกบันทึกไว้ในระบบ
Main Flow	1. ผู้ใช้เริ่มเล่นเกม และมีหน้าจอ "Deck Tracker" แสดงสถานะเริ่มต้นของเต็ค 2. เมื่อมีการเล่นการ์ด ผู้ใช้นำการ์ดใบนั้นมาแตะที่สมาร์ตโฟน 3. ระบบทำการอ่านข้อมูลการ์ดจาก NFC Tag 4. ระบบนำข้อมูลที่ได้อ่านไปเปรียบเทียบกับสถานะเต็คปัจจุบัน 5. ระบบอัปเดตสถานะของการ์ดในเต็ค และแสดงผลบนหน้าจอแบบเรียลไทม์ 6. ระบบบันทึกเหตุการณ์ที่เกิดขึ้น 7. ผู้ใช้เล่นเกมต่อ และทำซ้ำขั้นตอนที่ 2-6 ตลอดทั้งเกม
Alternative Flow	กรณีสแกนการ์ดซ้ำ 4. ณ ขั้นตอนที่ 4 หากระบบพบว่าการ์ดที่สแกนเคยอ่านแล้วให้สลับการกระทำ กรณีที่ไม่มีการ์ดในเต็ค 4. ณ ขั้นตอนที่ 4 หากระบบพบว่าการ์ดที่สแกนไม่มีในเต็คระบบจะแสดงข้อความแจ้งเตือนว่า "การ์ดใบนี้ไม่มีในเต็ค"

ตารางที่ 5.8 รายละเอียด Use Case #08 บันทึกประวัติการเล่น

Use Case #08	บันทึกประวัติการเล่น
Actor	ผู้ใช้
Description	ผู้ใช้สามารถบันทึกประวัติการเล่นเพื่อเรียกดู
Pre-conditions	Use Case "ติดตามการ์ด (Track Card)" กำลังทำงานอยู่
Post-conditions	ข้อมูลเหตุการณ์การเล่นถูกบันทึกลงในฐานข้อมูล และสามารถเรียกดูย้อนหลังหรือนำไปวิเคราะห์สถิติได้
Main Flow	1. Use Case นี้จะเริ่มทำงานโดยอัตโนมัติทุกครั้งที่เกิดเหตุการณ์ใน Use Case "ติดตามการ์ด" 2. ระบบสร้างบันทึกเหตุการณ์ ซึ่งประกอบด้วยข้อมูลการ์ดและเวลาที่เกิดเหตุการณ์ 3. ระบบนำบันทึกเหตุการณ์ไปต่อท้ายประวัติของเกมที่กำลังเล่นอยู่ 4. เมื่อผู้ใช้จบเกมใน Use Case "ติดตามการ์ด" ระบบจะทำการบันทึกประวัติการเล่นทั้งหมดของแมตช์นั้นลงฐานข้อมูลอย่างถาวร 5. หลังจากบันทึกสำเร็จ ระบบอาจเปิดโอกาสให้ผู้ใช้สามารถดูสถิติของเกมที่ผ่านมาได้
Alternative Flow	

ตารางที่ 5.9 รายละเอียด Use Case #09 แสดงสถิติการเล่น

Use Case #09	แสดงสถิติการเล่น
Actor	ผู้ใช้
Description	ผู้ใช้สามารถดูสถิติการเล่น เช่น อัตราการใช้การ์ดแต่ละใบ รายชื่อการ์ดที่ไม่ได้ใช้ จำนวนการ์ดที่จั่วและคืนกลับเข้าเต็ค ฯลฯ
Pre-conditions	มีข้อมูลประวัติการเล่นของอย่างน้อยหนึ่งเกมบันทึกอยู่ในระบบ
Post-conditions	ผู้ใช้ได้เห็นข้อมูลเชิงสถิติของเกมที่ถูกเลือก
Main Flow	1. Use Case นี้จะเริ่มทำงานเมื่อผู้ใช้เลือกที่จะดูสถิติจากเกมที่มีการบันทึกประวัติไว้แล้ว 2. ระบบทำการดึงข้อมูลประวัติการเล่นของเกมที่ถูกเลือก 3. ระบบประมวลผลข้อมูลใน Log เพื่อคำนวณค่าสถิติต่าง ๆ ตามที่ระบุในคำอธิบาย 4. ระบบแสดงผลสถิติในรูปแบบที่เข้าใจง่าย เช่น กราฟ หรือตาราง 5. ผู้ใช้ดูข้อมูลสถิติ
Alternative Flow	กรณีไม่มีประวัติการเล่น 1. ณ ขั้นตอนที่ 1 หากผู้ใช้พยายามจะดูสถิติ แต่ยังไม่มีการบันทึกประวัติการเล่นใด ๆ ระบบจะ ปิดการใช้งานเมนูสำหรับดูสถิติ

ตารางที่ 5.10 รายละเอียด Use Case #10 แชรประวัติการเล่นด้วย QR Code

Use Case #10	แชร์ประวัติการเล่นด้วย QR Code
Actor	ผู้ใช้
Description	ผู้ใช้สามารถแชร์ประวัติการเล่นด้วย QR Code โดยข้อมูลที่ดึงมาจะรวมเข้ากับประวัติการเล่นของคู่แข่งและเรียงตามเวลา
Pre-conditions	1. ผู้เล่นทั้งสองคนประวัติการเล่นของแมตช์นั้น ๆ 2. ผู้เล่นคนใดคนหนึ่งสร้างประวัติด้วย QR Code
Post-conditions	ประวัติการเล่นของทั้งสองฝั่งถูกรวมและจัดเรียงตามลำดับเวลา ทำให้ทั้งสองฝ่ายสามารถดูเหตุการณ์ทั้งหมดของเกมได้
Main Flow	1. ผู้เล่นคนที่ 1 เข้าไปที่ประวัติการเล่นของตนเอง และเลือก "สร้าง QR Code" 2. ระบบสร้าง Session ID ใน QR Code แล้วแสดงบนหน้าจอของผู้เล่นคนที่ 1 3. ผู้เล่นคนที่ 2 เลือกเมนู "สแกน QR Code" 4. ระบบ เปิดกล้องบนอุปกรณ์ของผู้เล่นคนที่ 2 5. ผู้เล่นคนที่ 2 นำกล้องไปสแกน QR Code บนหน้าจอของผู้เล่นคนที่ 1 6. ระบบของผู้เล่นคนที่ 2 อ่านค่า Session ID จาก QR Code และดึงข้อมูลประวัติการเล่นของตนอีกฝ่ายบน Server 7. ระบบจะทำการรวมประวัติการเล่นและจัดเรียงเหตุการณ์ทั้งหมดตามเวลา 8. หน้าจอของผู้เล่นที่สแกน QR Code แสดงผลประวัติการเล่นของทั้งเกมที่เหมาะสม
Alternative Flow	กรณีไม่มีประวัติการเล่น 1. ณ ขั้นตอนที่ 1 หากผู้ใช้พยายามจะดูสถิติ แต่ยังไม่มีการบันทึกประวัติการเล่นใด ๆ ระบบจะ ปิดการใช้งานเมนูสำหรับดูสถิติ

ตารางที่ 5.11 รายละเอียด Use Case #11 สร้างการ์ด

Use Case #11	สร้างการ์ด
Actor	ผู้ใช้
Description	ผู้ใช้สามารถสร้างการ์ดของตนเองโดยกำหนดชื่อ รูปภาพ และคุณสมบัติต่าง ๆ แล้วบันทึก
Pre-conditions	ผู้ใช้อยู่ในหน้าค้นหาการ์ดจากคอลเลกชันส่วนตัว
Post-conditions	การ์ดที่ผู้ใช้สร้างขึ้นใหม่ถูกบันทึกลงใน "My Collections" และพร้อมสำหรับนำไปใช้ในเด็ค
Main Flow	1. ผู้ใช้กดปุ่ม "สร้างการ์ด" หรือ "+" ในหน้าคอลเลกชันส่วนตัว 2. ระบบแสดงฟอร์มสำหรับกรอกรายละเอียดของการ์ดใหม่ 3. ผู้ใช้ทำการกรอกข้อมูลต่าง ๆ 4. เมื่อกรอกข้อมูลครบถ้วน ผู้ใช้กดปุ่ม "บันทึก" 5. ระบบตรวจสอบความสมบูรณ์ของข้อมูล 6. ระบบบันทึกข้อมูลการ์ดใหม่ลงในฐานข้อมูลภายใต้ "My Collections" ของผู้ใช้
Alternative Flow	กรณีข้อมูลไม่สมบูรณ์ 5. ณ ขั้นตอนที่ 5 หากระบบพบว่าผู้ใช้ยังไม่ได้กรอกข้อมูลที่จำเป็น 6. ระบบจะไม่แสดงปุ่ม "บันทึก" จนกว่าผู้ใช้กรอกข้อมูลให้ครบถ้วน 7. ผู้ใช้ยังคงอยู่ที่ฟอร์มสร้างการ์ด กรณีผู้ใช้ยกเลิกการสร้าง 4. ณ ขั้นตอนใด ๆ ก่อนการกดบันทึก ผู้ใช้กดย้อนกลับ หรือปุ่มยกเลิก 5. ระบบยกเลิกการสร้างการ์ด และข้อมูลที่กรอกไว้จะไม่ถูกบันทึก

ตารางที่ 5.12 รายละเอียด Use Case #12 ตั้งค่าแอป

Use Case #12	ตั้งค่าแอป
Actor	ผู้ใช้
Description	ผู้ใช้สามารถจัดการบัญชีของตน Sign In / Sign Out และเปลี่ยนภาษาของแอป
Pre-conditions	ผู้ใช้เข้าสู่ระบบเรียบร้อยแล้ว และอยู่ในหน้า "Settings"
Post-conditions	การตั้งค่าของแอปพลิเคชันถูกเปลี่ยนแปลงและบันทึกตามที่ผู้ใช้เลือก
Main Flow	1. ระบบแสดงเมนูการตั้งค่าต่าง ๆ 2. ผู้ใช้เลือกเมนู "ภาษา" 3. ระบบแสดงรายการภาษาที่รองรับ พร้อมทั้งระบุภาษาที่ใช้งานในปัจจุบัน 4. ผู้ใช้เลือกภาษาใหม่ที่ต้องการ 5. ระบบทำการบันทึกการตั้งค่าภาษาใหม่ และปรับเปลี่ยนการแสดงผลของแอปเป็นภาษาที่เลือกทันที
Alternative Flow	กรณีผู้ใช้ต้องการออกจากระบบ 2. ณ ขั้นตอนที่ 2 ผู้ใช้เลือกเมนู "ออกจากระบบ" 3. ระบบทำการล้างข้อมูลสถานะการเข้าสู่ระบบ และนำผู้ใช้กลับไปยังหน้า "เข้าสู่ระบบ"

5.6 การประยุกต์ใช้งานเทคโนโลยี NFC

แท็ก NFC ที่เลือกใช้ในระบบคือรุ่น Ntag213 [14] ดังรูปที่ 5.14 ลักษณะโครงสร้างแท็ก NFC รุ่น Ntag213 ซึ่งเป็นแท็กที่รองรับมาตรฐาน ISO 14443 และ NDEF ทำงานที่คลื่นความถี่ 13.56 MHz มีขนาดหน่วยความจำ 180 bytes และมีตัวแท็กขนาด 21×11 mm จุดเด่นของแท็กรุ่นนี้คือสามารถเขียนและอ่านข้อมูลซ้ำได้ และสามารถติดกับวัตถุอื่นเพื่อใช้งานได้สะดวก



รูปที่ 5.14 ลักษณะโครงสร้างแท็ก NFC รุ่น Ntag213

แท็ก NFC ถูกนำไปติดที่ Sleeve การ์ด ดังรูปที่ 5.15 เพื่อให้สามารถใช้ในการบันทึกข้อมูลของการ์ดและสแกนแท็กระหว่างการเล่นผ่านแอปพลิเคชัน ระบบยังสามารถรองรับแท็กอื่นที่มีความสามารถในการ เขียน-อ่าน ข้อมูลได้ แต่ต้องเป็นแท็กที่สามารถใช้งานร่วมกับมาตรฐานที่แอปรองรับเท่านั้น โดยนำแท็กไปติดบนช่องใส่การ์ด

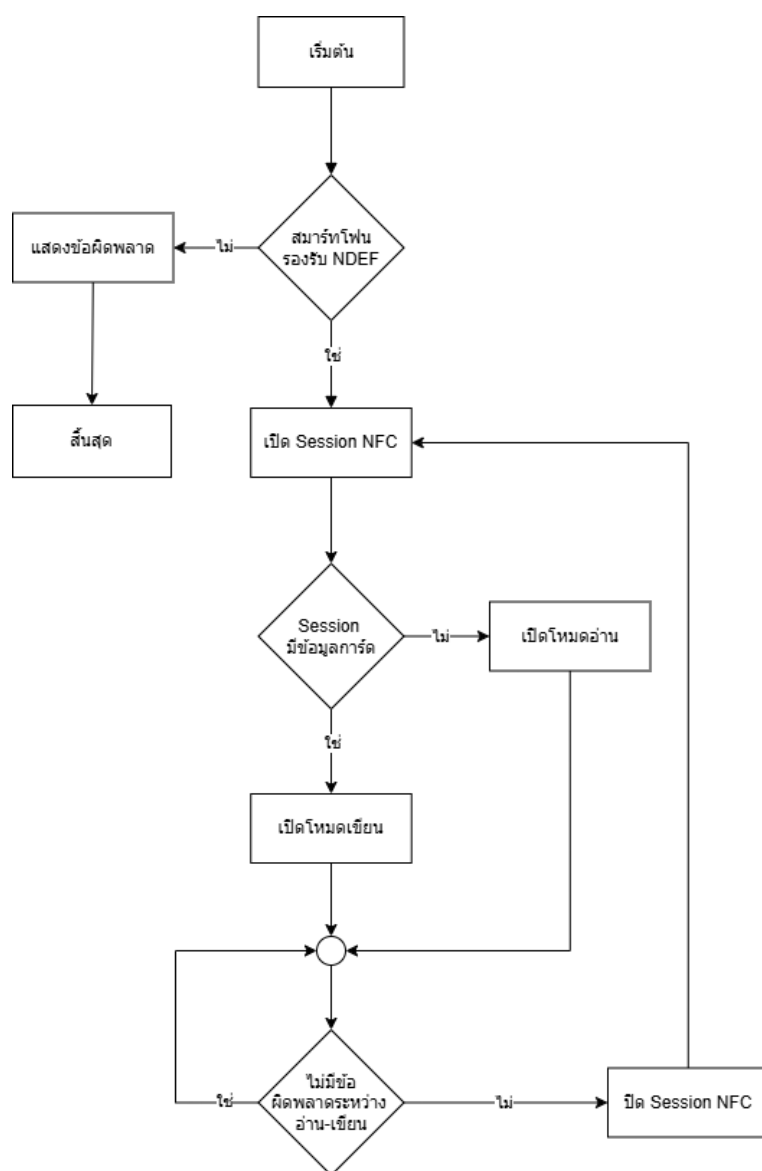


รูปที่ 5.15 ตัวอย่างการติดแท็ก NFC บนช่องใส่การ์ด (Sleeve)

5.7 กระบวนการทำงานของ NFC ภายในแอปพลิเคชัน

5.7.1 การควบคุม Session NFC

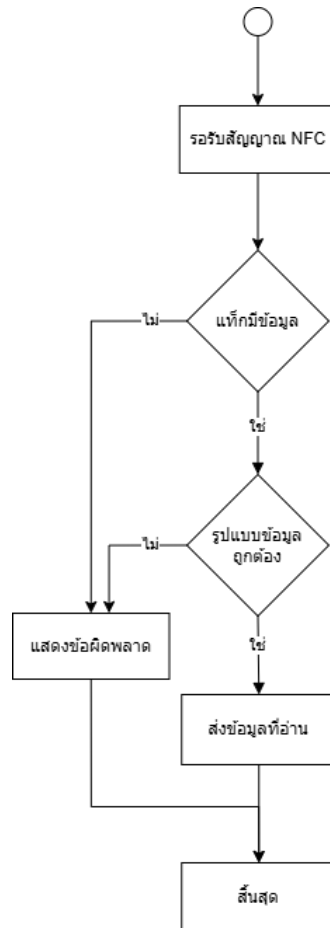
ในกระบวนการทำงานของ NFC ระบบมีการควบคุมด้วย Session เพื่อจัดการการ อ่าน-เขียน ข้อมูลบนแท็ก NFC โดยเริ่มต้นระบบจะตรวจสอบว่าอุปกรณ์รองรับการใช้งาน NFC หรือไม่ หากไม่สามารถใช้งานได้ระบบจะแสดงข้อความแจ้งข้อผิดพลาดให้กับผู้ใช้ หากสามารถใช้งานได้ระบบจะเปิด Session NFC และเมื่อผู้ใช้นำแท็กมาแตะ ระบบจะตรวจสอบก่อนว่าแท็กรองรับมาตรฐาน NDEF หรือไม่ หากไม่รองรับจะมีการแจ้งเตือนข้อผิดพลาด แต่ถ้าหากรองรับ ระบบจะตรวจสอบต่อไปว่าใน Session มีข้อมูลการ์ดหรือไม่ ถ้าพบข้อมูลระบบจะเปิดโหมดเขียนเพื่อเขียนข้อมูลการ์ดลงในแท็ก NFC แต่ถ้าหากไม่มีข้อมูลระบบจะเปิดโหมดอ่านเพื่ออ่านข้อมูลการ์ดในแท็ก จากนั้นระบบจะดำเนินการตามเงื่อนไขที่กำหนดของแต่ละโหมดซึ่งหากเกิดข้อผิดพลาดระหว่าง อ่าน-เขียน ระบบจะเปิด Session NFC ใหม่อีกครั้ง โดยมีลำดับการทำงานดังรูปที่ 5.16



รูปที่ 5.16 แผนภาพลำดับงานของกระบวนการควบคุม NFC Session

5.7.2 โหมดการอ่านแท็ก NFC

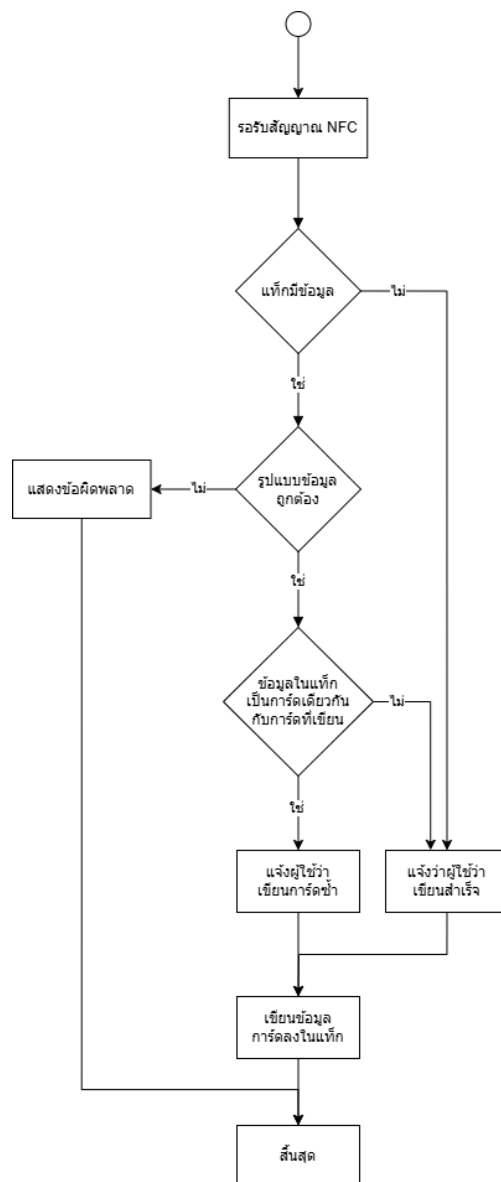
ทำหน้าที่ตรวจสอบและดึงข้อมูลจากแท็ก โดยระบบเริ่มต้นจากการอ่านข้อมูลบนแท็กเพื่อตรวจสอบว่ามีข้อมูลอยู่หรือไม่ หากไม่มีข้อมูลระบบจะสิ้นสุดการทำงาน แต่หากมีข้อมูลอยู่ระบบจะตรวจสอบความถูกต้องของข้อมูลว่าสามารถนำไปใช้งานได้หรือไม่ หากข้อมูลสามารถใช้งานได้ระบบจะส่งข้อมูลไปยังระบบแสดงผลให้ผู้ใช้งาน แต่หากข้อมูลไม่สามารถใช้งานได้ระบบจะสิ้นสุดการทำงานและแจ้งข้อผิดพลาด โดยมีลำดับการทำงานดังรูปที่ 5.17



รูปที่ 5.17 แผนภาพลำดับงานสำหรับโหมดการอ่านแท็ก NFC

5.7.3 โหมดการเขียนแท็ก NFC

ทำหน้าที่บันทึกข้อมูลการ์ดลงบนแท็ก NFC โดยระบบจะเริ่มต้นด้วยการตรวจสอบข้อมูลเดิมบนแท็ก หากพบข้อมูล ระบบจะดึงข้อมูลนั้นมาสร้างเป็น TagEntity จากนั้นจะเปรียบเทียบ cold และ cald ของ TagEntity กับข้อมูล CardEntity ที่เตรียมไว้สำหรับการเขียน กรณีที่ข้อมูลตรงกัน: ระบบจะเขียนข้อมูล CardEntity ลงแท็กตามปกติ พร้อมแสดงข้อความเตือนว่าเป็นการเขียนทับข้อมูลเดิม กรณีที่ข้อมูลไม่ตรงกัน: ระบบจะเขียนข้อมูล CardEntity ลงแท็ก และแจ้งผู้ใช้งานว่าการเขียนข้อมูลสำเร็จโดยไม่มีการเตือนเรื่องการเขียนทับ แต่หากแท็กไม่มีข้อมูลเดิมอยู่ ระบบจะข้ามขั้นตอนการเปรียบเทียบข้อมูล และเขียนข้อมูลใหม่จาก CardEntity ลงบนแท็กโดยตรง เมื่อเสร็จสิ้น ระบบจะแจ้งข้อความยืนยันการเขียนข้อมูลสำเร็จ ไม่ว่าจะเป็นการเขียนข้อมูลใหม่ การเขียนทับข้อมูลเดิม หรือเกิดข้อผิดพลาดใด ๆ (เช่น ข้อมูลการ์ดไม่สมบูรณ์หรือข้อมูลเกินความจุ) ระบบจะแสดงผลลัพธ์ที่ชัดเจนแก่ผู้ใช้เพื่อให้ผู้ใช้รับทราบสถานะการทำงานทั้งหมด โดยมีลำดับการทำงานดังรูปที่ 5.18



รูปที่ 5.18 แผนภาพลำดับงานสำหรับโหมดการเขียนแท็ก NFC

5.7.4 โครงสร้างข้อมูลภายในแท็ก NFC (Ntag213)

เมื่อข้อมูลถูกเขียนลงบนแท็ก NFC ชนิด Ntag213 ข้อมูลเหล่านั้นจะถูกจัดเก็บในรูปแบบ NDEF Message ซึ่งประกอบด้วย NDEF Records แต่ละ Record จะมีส่วนของ Payload ที่บรรจุข้อมูลจริงที่ต้องการบันทึก ในกรณีของการประยุกต์ใช้งานนี้ Payload จะประกอบด้วยรหัส Collection ID (cold) และ Card ID (cald) ระบบจะแปลงชุดตัวเลขไบต์ที่เป็น Payload ให้กลับมาเป็นข้อความที่มนุษย์สามารถอ่านได้พร้อมทั้งจัดรูปแบบให้เป็นข้อมูลที่ใช้งานได้ง่ายจริงในแอปพลิเคชัน ยกตัวอย่างเช่น

Original Payload 1: [2, 101, 110, 99, 111, 73, 100, 58, 32, 52, 98, 100, 98, 55, 50, 100, 51, 45, 99, 57, 53, 50, 45, 52, 49, 57, 49, 45, 57, 99, 101, 56, 45, 53, 52, 51, 51, 100, 55, 98, 48, 101, 54, 100, 52] ชุดไบต์นี้เมื่อแปลงเป็นข้อความ ASCII และตัดอักขระ 3 ตัวแรก ซึ่งจะได้ข้อมูลที่ต้องการคือ cold: 4bdb72d3-c952-4191-9ce8-5433d7b0e6d4

Original Payload 2: [2, 101, 110, 99, 97, 73, 100, 58, 32, 56, 48, 97, 48, 54, 101, 56, 57, 45, 57, 55, 97, 51, 45, 52, 56, 52, 56, 45, 56, 52, 49, 45, 99, 98, 100, 53, 98, 49, 52, 98, 50, 56, 48, 101] ชุดไบต์นี้เมื่อแปลงเป็นข้อความ ASCII และตัดอักขระ 3 ตัวแรก ซึ่งจะได้ข้อมูลที่ต้องการคือ cald: 80a06e89-97a3-4848-8641-cbd5b14b280e

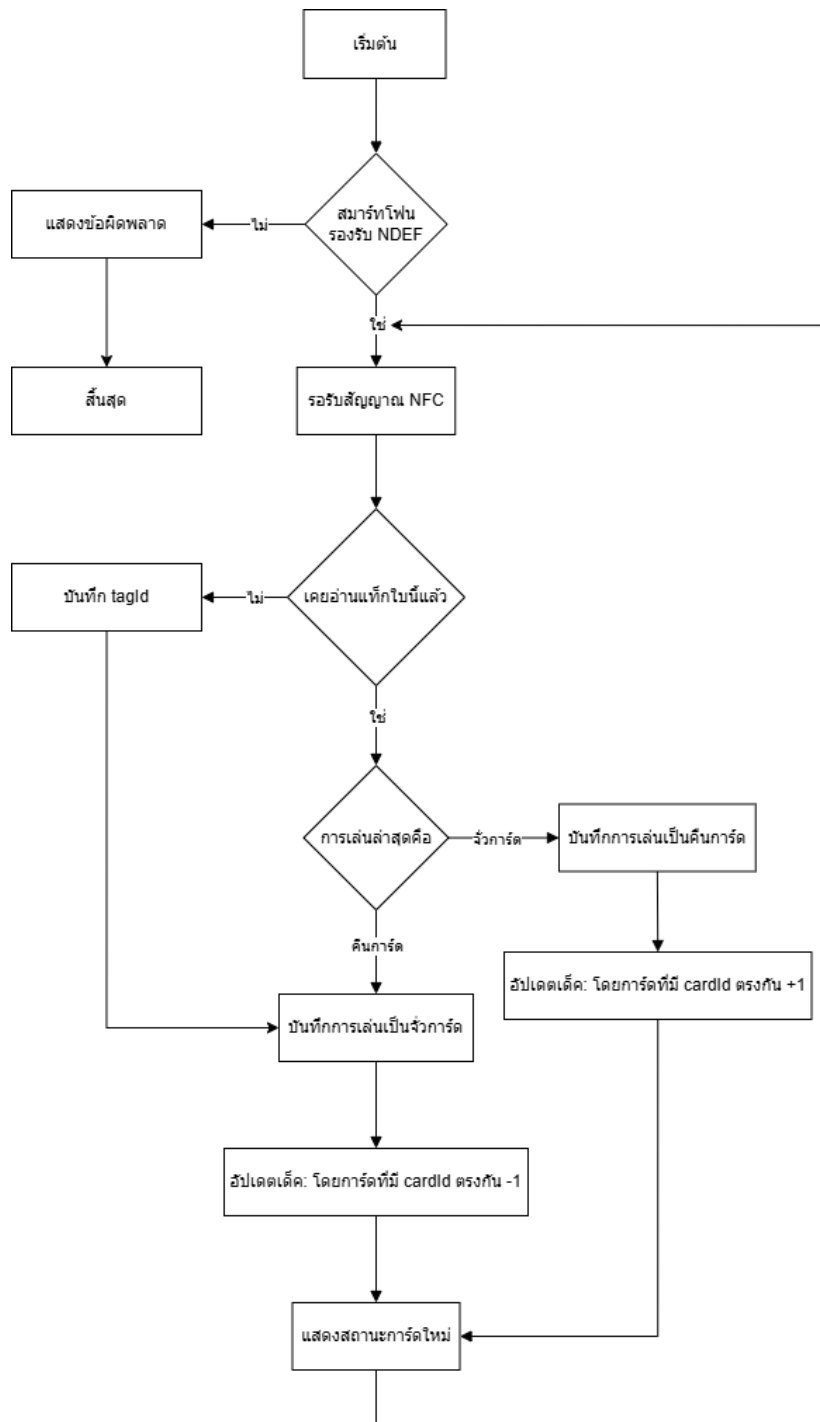
ดังนั้น ภายในแท็ก NFC จะมี NDEF Records ที่แสดงข้อมูลรหัสคอลเลกชันและรหัสการ์ดในรูปแบบที่พร้อมใช้งานได้ทันที สรุปในตารางที่ 5.13

ตารางที่ 5.13 ตัวอย่างโครงสร้างข้อมูลของ Record Payload ภายในแท็ก NFC

Record Payload (หลัง substring(3))	คำอธิบาย
cold: 4bdb72d3-c952-4191-9ce8-5433d7b0e6d4	Collection ID (รหัสคอลเลกชัน)
cald: 80a06e89-97a3-4848-8641-cbd5b14b280e	Card ID (รหัสการ์ด)

5.7.5 การติดตามการ์ดในเด็กด้วยแท็ก NFC

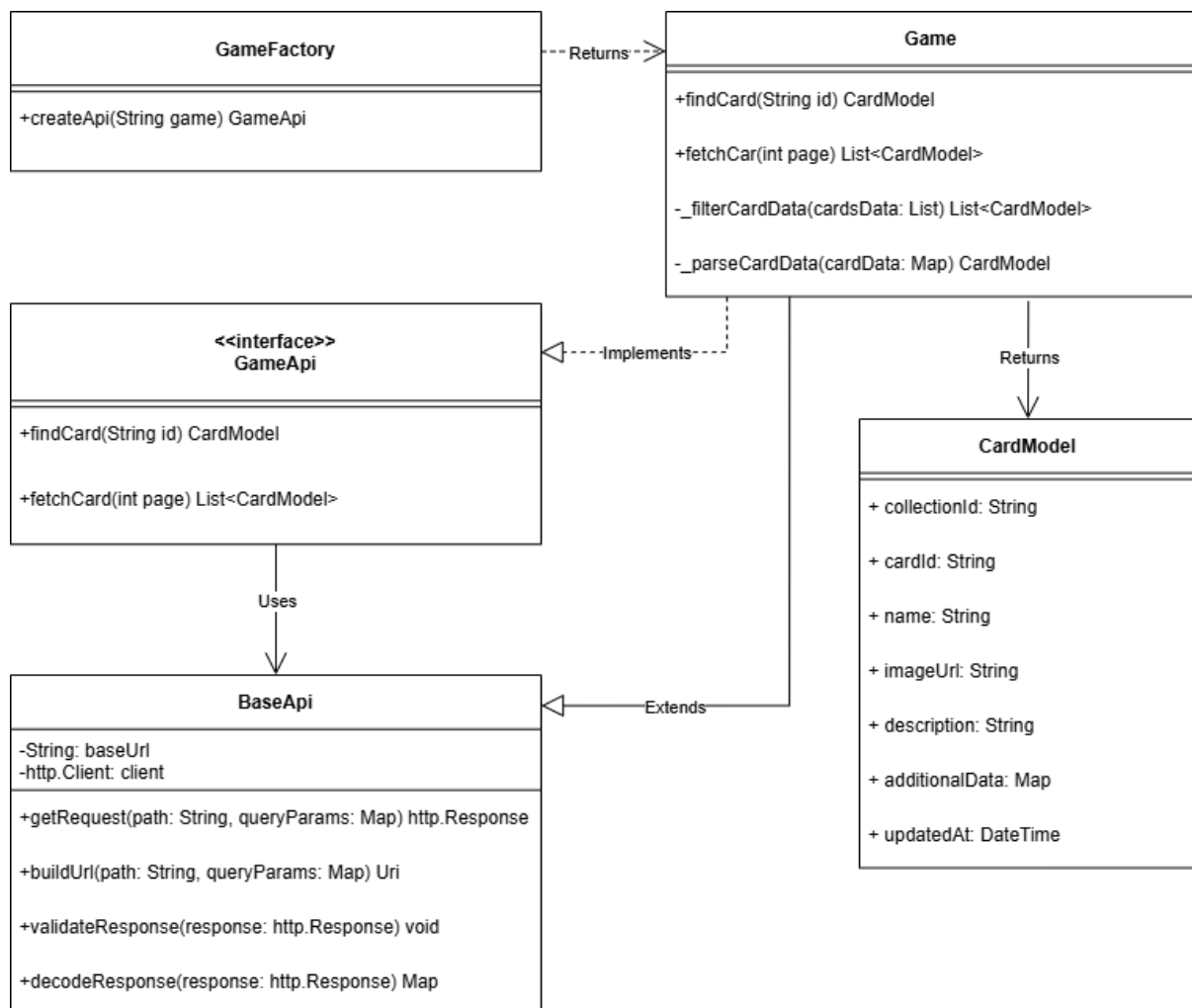
ระบบติดตามการ์ดในเด็กผู้ใช้สามารถสแกนแท็ก NFC ที่มีข้อมูลการ์ดเพื่ออัปเดตสถานะการเล่นของการ์ด โดยเริ่มต้นระบบจะเปิด Session NFC หากการเปิดไม่สำเร็จระบบจะแจ้งข้อผิดพลาดให้ผู้ใช้ทราบ แต่หากสามารถเปิดได้ ระบบจะรอให้ผู้ใช้สแกนแท็ก NFC เมื่อผู้ใช้สแกนแท็ก ระบบจะตรวจสอบก่อนว่าแท็กรองรับมาตรฐาน NDEF หรือไม่ หากไม่รองรับจะมีการแจ้งเตือนข้อผิดพลาด แต่หากรองรับ ระบบจะดำเนินการอ่านข้อมูลและตรวจสอบว่าการ์ดที่อ่านได้นั้นเคยถูกใช้งานมาก่อนหรือไม่ หากเป็นแท็กใหม่ระบบจะบันทึกข้อมูลแท็กและอัปเดตสถานะการ์ดให้เป็นการเล่นครั้งแรก แต่หากแท็กนั้นถูกใช้งานมาก่อน ระบบจะตรวจสอบข้อมูลการเล่นล่าสุดเพื่อกำหนดการอัปเดตสถานะการ์ด ในกรณีที่การเล่นล่าสุดของการ์ดเป็นการจั่วออกจากเด็ก ระบบจะเพิ่มจำนวนการ์ดที่มีอยู่ในเด็กโดยอัตโนมัติ แต่หากการเล่นล่าสุดเป็นการคืนการ์ดกลับเข้าเด็ก ระบบจะลดจำนวนการ์ดกลับเข้าไปในเด็กโดยอ้างอิงจาก Card ID ที่ตรงกัน หลังจากอัปเดตสถานะการ์ดแล้ว ระบบจะแสดงข้อมูลเด็กที่ได้รับการปรับปรุงให้ผู้ใช้เห็นเพื่อให้สามารถติดตามจำนวนการ์ดในเด็กได้แบบเรียลไทม์ เมื่อการอัปเดตสถานะเสร็จสิ้นระบบจะรอให้ผู้ใช้สแกนแท็ก NFC ครั้งถัดไป เพื่อดำเนินการตรวจสอบและอัปเดตการ์ดใบอื่น ๆ ในระหว่างเกม หากผู้ใช้เลือกที่จะสิ้นสุดการใช้งานระบบจะปิด Session NFC และจบกระบวนการติดตามการ์ดในเด็ก โดยมีลำดับการทำงานดังรูปที่ 5.19



รูปที่ 5.19 แผนภาพลำดับงานของระบบติดตามการ์ดด้วยแท็ก NFC

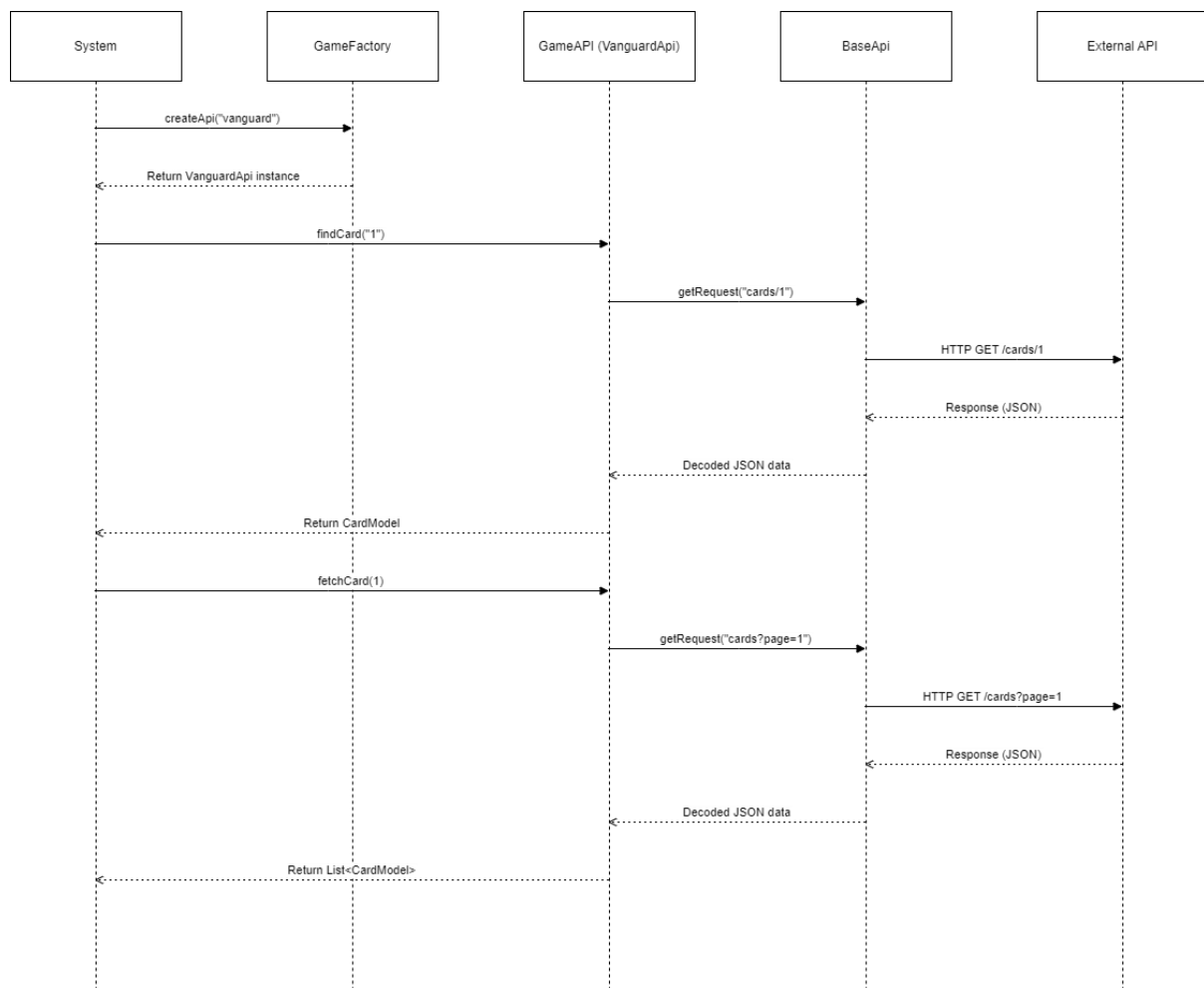
5.8 การเชื่อมต่อข้อมูลการ์ดจาก API ภายนอก

ระบบใช้ Factory Pattern ในการดึงข้อมูลการ์ดจาก API เพื่อรองรับเกมที่หลากหลาย GameFactory ทำหน้าที่ตรวจสอบว่าเกมที่ร้องขอได้รับการสนับสนุนหรือไม่ หากรองรับ ระบบจะสร้างและคืนค่า API ที่เหมาะสมสำหรับเกมนั้น API ที่ถูกสร้างขึ้นสามารถดึงข้อมูลการ์ดได้สองรูปแบบ คือ การค้นหาตามรหัส findCard และการดึงข้อมูลแบบเป็นชุด fetchCard โดยใช้ BaseApi เป็นตัวกลางในการเชื่อมต่อกับแหล่งข้อมูลภายนอก เมื่อได้รับข้อมูล ระบบจะประมวลผลและจัดเก็บใน CardModel ซึ่งประกอบด้วย ชื่อ รูปภาพ คำอธิบาย และข้อมูลที่เกี่ยวข้อง หากเป็นการดึงข้อมูลแบบหลายหน้า ระบบจะกรองเฉพาะการ์ดที่เกี่ยวข้องก่อนส่งให้ผู้ใช้ การออกแบบด้วย Factory Pattern ทำให้ระบบสามารถเพิ่ม API สำหรับเกมใหม่ได้โดยไม่ต้องแก้ไขโครงสร้างหลักส่งผลให้รองรับการขยายตัวได้อย่างยืดหยุ่น โดยมีโครงสร้างดังรูปที่ 5.20



รูปที่ 5.20 แผนภาพคลาสแสดงโครงสร้างการเชื่อมต่อ API ด้วย Factory Pattern

แผนผังรูปที่ 5.21 แสดงกระบวนการทำงานของระบบในการเชื่อมต่อและดึงข้อมูลการ์ดจาก API ภายนอก โดยใช้แนวทาง Factory Pattern เพื่อรองรับเกมการ์ดจากหลากหลายแหล่งข้อมูลผ่านโครงสร้างที่ยืดหยุ่นและสามารถขยายเพิ่มเติมได้ในอนาคต เริ่มต้นจากระบบหลัก (System) เรียกใช้คำสั่ง createApi จากคลาส GameFactory เพื่อสร้างอินสแตนซ์ของ API ที่สอดคล้องกับเกมที่ระบุ เช่น Vanguard ซึ่งอินสแตนซ์ดังกล่าวจะสืบทอดจาก BaseApi และมีการ implement อินเทอร์เฟซ GameApi เพื่อให้สามารถเรียกใช้งานเมธอดพื้นฐานได้เหมือนกันทุกเกม เมื่อระบบต้องการดึงข้อมูลการ์ด ระบบจะเรียกใช้เมธอด findCard หรือ fetchCard ซึ่งจะดำเนินการผ่าน GameAPI ไปยัง BaseApi โดย BaseApi จะรับหน้าที่ในการสร้างคำขอ (HTTP Request) ส่งไปยัง External API เพื่อดึงข้อมูลตาม URL ที่กำหนด และเมื่อได้รับข้อมูล JSON กลับมา จะแปลงข้อมูล (decode) ให้อยู่ในรูปแบบ CardModel ก่อนส่งกลับไปยังชั้นระบบหลัก ด้วยโครงสร้างนี้ทำให้สามารถเพิ่ม API สำหรับเกมใหม่ได้โดยไม่ต้องแก้ไขโค้ดหลักของระบบ ช่วยให้การขยายระบบในอนาคตเป็นไปได้อย่างสะดวกยิ่งขึ้น



รูปที่ 5.21 แผนภาพลำดับเหตุการณ์แสดงการทำงานของ Game Factory

5.9 การจัดการข้อมูลและการซิงค์แบบสองทาง (Two-Way Sync)

ระบบ NFC Deck Tracker มีการออกแบบกระบวนการจัดการข้อมูลโดยแยกการเก็บข้อมูลออกเป็นสองส่วน ได้แก่ Local Storage (SQLite): ใช้งานเมื่ออุปกรณ์ไม่มีอินเทอร์เน็ต และ Remote Storage (Firebase): ใช้งานเมื่อผู้ใช้เข้าสู่ระบบผ่าน Google Account โดยการทำงานของซิงค์ข้อมูลจะทำงานแบบ สองทาง (Two-Way Sync) ซึ่งจะถูกเรียกใช้เมื่อมีการโหลดข้อมูลเด็ด เช่น การเปิดหน้ารายการเด็ดทั้งหมดของผู้ใช้ โดยมีลำดับการทำงาน ดังนี้

5.9.1 ขั้นตอนการซิงค์ข้อมูลแบบสองทาง

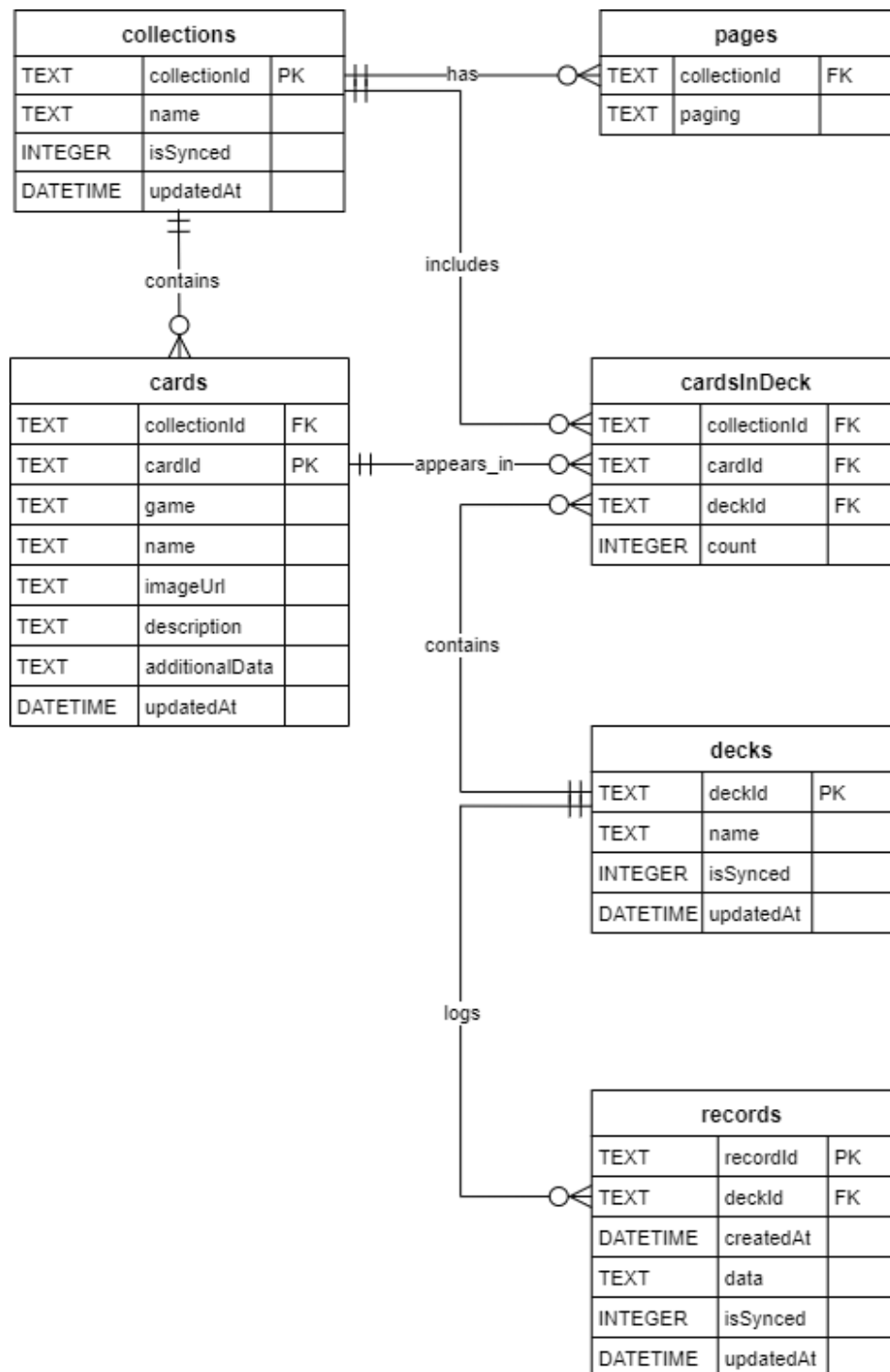
1. โหลดข้อมูลจาก Local Storage (ครั้งเดียว): แอปจะดึงข้อมูลจากฐานข้อมูลภายในเครื่อง SQLite มาแสดงทันที เพื่อให้ผู้ใช้เห็นข้อมูลได้รวดเร็ว แม้ในกรณีไม่มีอินเทอร์เน็ต
2. ตรวจสอบการเข้าสู่ระบบ: หากพบว่าอุปกรณ์มีการล็อกอินด้วยบัญชี Google ตรวจสอบจาก userID ระบบจะดำเนินการซิงค์ข้อมูลกับ Firebase
3. นำเข้าหรืออัปเดตข้อมูลจาก Remote → Local: ระบบจะเปรียบเทียบข้อมูลจาก Firebase และ SQLite: ถ้าพบข้อมูลใหม่ใน Firebase ที่ยังไม่มีใน Local → สร้างใหม่ใน SQLite ถ้าข้อมูลใน Firebase มีการอัปเดตใหม่กว่าที่อยู่ใน SQLite → อัปเดตข้อมูลใน SQLite
4. ส่งข้อมูลจาก Local → Remote: สำหรับข้อมูลที่สร้างหรือแก้ไขภายในเครื่อง Local แต่ยังไม่ถูกซิงค์ isSynced = false ระบบจะพยายามส่งข้อมูลไปยัง Firebase หากสำเร็จ → อัปเดตสถานะ isSynced = true และอัปเดตข้อมูลใน SQLite ให้ตรงกัน
5. ลบข้อมูลใน Local ที่ถูกลบไปจาก Remote แล้ว: ระบบจะตรวจสอบข้อมูลที่เคยซิงค์ isSynced = true แต่ไม่มีอยู่ใน Firebase อีกต่อไป รายการเหล่านี้จะถูกลบออกจาก SQLite เพื่อให้ข้อมูลตรงกัน
6. คำนวณรายการเด็ดทั้งหมดจาก Local: หลังจากขั้นตอนทั้งหมดเสร็จสิ้น รายการข้อมูลทั้งหมดที่อยู่ใน SQLite และผ่านการซิงค์แล้วจะถูกคืนค่าไปแสดงใน UI

5.9.2 การจัดการข้อมูลเมื่อเกิด Event ต่าง ๆ

ในการใช้งานแอปพลิเคชันเมื่อผู้ใช้กระทำ Event ต่าง ๆ เช่น การบันทึกเด็ดใหม่ การแก้ไข หรือการลบเด็ด ระบบจะดำเนินการจัดการข้อมูลทันทีในฐานข้อมูลภายในเครื่อง (SQLite) เพื่อให้สามารถใช้งานแบบออฟไลน์ได้ และหากผู้ใช้เข้าสู่ระบบด้วยบัญชี Google แล้ว ระบบจะดำเนินการจัดการข้อมูลเดียวกันใน Firebase โดยอัตโนมัติ เพื่อให้ข้อมูลมีความสอดคล้องกันทั้งในเครื่องและบน Cloud รวมถึงมีการอัปเดตสถานะการซิงค์อย่างเหมาะสมด้วย

5.10 การออกแบบโครงสร้างฐานข้อมูล

แผนผังรูปที่ 5.22 แสดงถึงโครงสร้างฐานข้อมูลที่ถูกออกแบบเพื่อรองรับการทำงานของระบบ NFC Deck Tracker โดยแบ่งข้อมูลออกเป็นกลุ่มต่าง ๆ ตามหน้าที่การใช้งานหลัก เช่น คอลเล็กชัน การ์ด เด็ค ประวัติการเล่น และข้อมูลภายในระบบ



รูปที่ 5.22 แผนภาพแสดงความสัมพันธ์ข้อมูลของระบบ

5.10.1 รายละเอียดของแต่ละตารางในโครงสร้างฐานข้อมูล

เพื่อให้สามารถจัดเก็บและจัดการข้อมูลได้อย่างยืดหยุ่น ระบบฐานข้อมูลของแอปพลิเคชันจึงถูกออกแบบให้มีทั้งหมด 6 ตาราง โดยแต่ละตารางมีบทบาทเฉพาะในการรองรับกระบวนการทำงานของระบบ ดังนี้

- Collections ใช้เก็บข้อมูลเกมหรือคอลเล็กชันของการ์ดในระบบ โดยรวมทั้งเกมที่มีอยู่จริง (ดึงมาจาก API ภายนอก) และคอลเล็กชันที่ผู้ใช้สร้างขึ้นเอง ให้อยู่ร่วมกันในตารางเดียว เพื่อลดความซับซ้อนในการแยกจัดการหลายตารางในระบบ
- Pages ใช้เก็บข้อมูลเกี่ยวกับ pagination ของ API แต่ละเกม ซึ่งมีโครงสร้างที่แตกต่างกัน ระบบจึงต้องบันทึกข้อมูลนี้ไว้เพื่อรองรับการดึงข้อมูลแบบต่อเนื่อง อย่างถูกต้องในแต่ละรอบการดึงข้อมูล
- Cards ใช้เก็บรายละเอียดของการ์ดแต่ละใบในแต่ละคอลเล็กชัน เช่น รหัส ชื่อ รูปภาพ คำอธิบาย และข้อมูลเพิ่มเติมที่อาจมาจาก API หรือผู้ใช้กรอกเอง โดยใช้ composite key คือ (collectionId, cardId) เพื่อรองรับการ์ดที่มีรหัสซ้ำกันแต่ไม่ใช่เกมเดียวกัน
- Decks ใช้เก็บข้อมูลเด็คของผู้ใช้ เช่น ชื่อเด็ค และสถานะการซิงก์ โดยมีความสัมพันธ์กับการ์ดผ่านตาราง cardsInDeck
- CardsInDeck ทำหน้าที่เป็นตารางกลางในการเชื่อมความสัมพันธ์แบบ many-to-many ระหว่าง cards และ decks พร้อมทั้งระบุจำนวนการ์ดแต่ละใบในเด็คผ่านฟิลด์ count
- Records ใช้เก็บบันทึกการเล่นในแต่ละครั้งของเด็ค เช่น เวลาที่เล่น (createdAt), ข้อมูลการเล่น (data) โดยข้อมูลเหล่านี้สามารถนำไปใช้แสดงเป็นประวัติการเล่นหรือใช้ในการวิเคราะห์เชิงสถิติภายในระบบ

นอกจากนี้ตารางที่เกี่ยวข้องกับการซิงก์ข้อมูล ได้แก่ collections, decks และ records ยังมีฟิลด์ isSynced เพื่อระบุว่าวัตถุนั้นได้ถูกซิงก์กับ Firebase แล้วหรือไม่ (0 = ยังไม่ซิงก์ 1 = ซิงก์แล้ว) และฟิลด์ updatedAt สำหรับบันทึกวันที่และเวลาที่มีการแก้ไขข้อมูลล่าสุด เพื่อให้ระบบสามารถตรวจได้ว่าการเปลี่ยนแปลงของข้อมูลและทำการซิงก์ข้อมูลแบบ incremental และเพื่อสนับสนุนการทำงานร่วมกันของ Relational (SQLite สำหรับจัดเก็บข้อมูลในเครื่อง) และ Non-Relational (Firestore สำหรับจัดเก็บข้อมูลบนคลาวด์) แม้ทั้งสองจะมีโครงสร้างต่างกันแต่ถูกออกแบบให้เหมาะสมกับบริบทการใช้งานเฉพาะด้านของแต่ละระบบทั้งในด้านความยืดหยุ่น ประสิทธิภาพ และต้นทุนการใช้งาน

5.10.2 เหตุผลและแนวคิดในการออกแบบ

Relational Database Schema (SQL) โครงสร้างนี้ถูกออกแบบมาในลักษณะเชิงสัมพันธ์ โดยแยกข้อมูลออกเป็น 6 ตาราง ได้แก่ collections, pages, cards, decks, cardsInDeck, และ records เพื่อความยืดหยุ่นในการใช้งาน CRUD (Create, Read, Update, Delete) รวมถึงช่วยให้ระบบสามารถบังคับโครงสร้างข้อมูลและความสัมพันธ์ระหว่างตารางได้อย่างเข้มงวด ซึ่งมีข้อดีคือ:

- ช่วยลดการทำงานซ้ำซ้อน (data redundancy)
- เพิ่มความปลอดภัยและความแม่นยำของข้อมูล
- รองรับการใช้ข้อมูลเฉพาะทางและการ join ที่ซับซ้อนได้อย่างมีประสิทธิภาพ

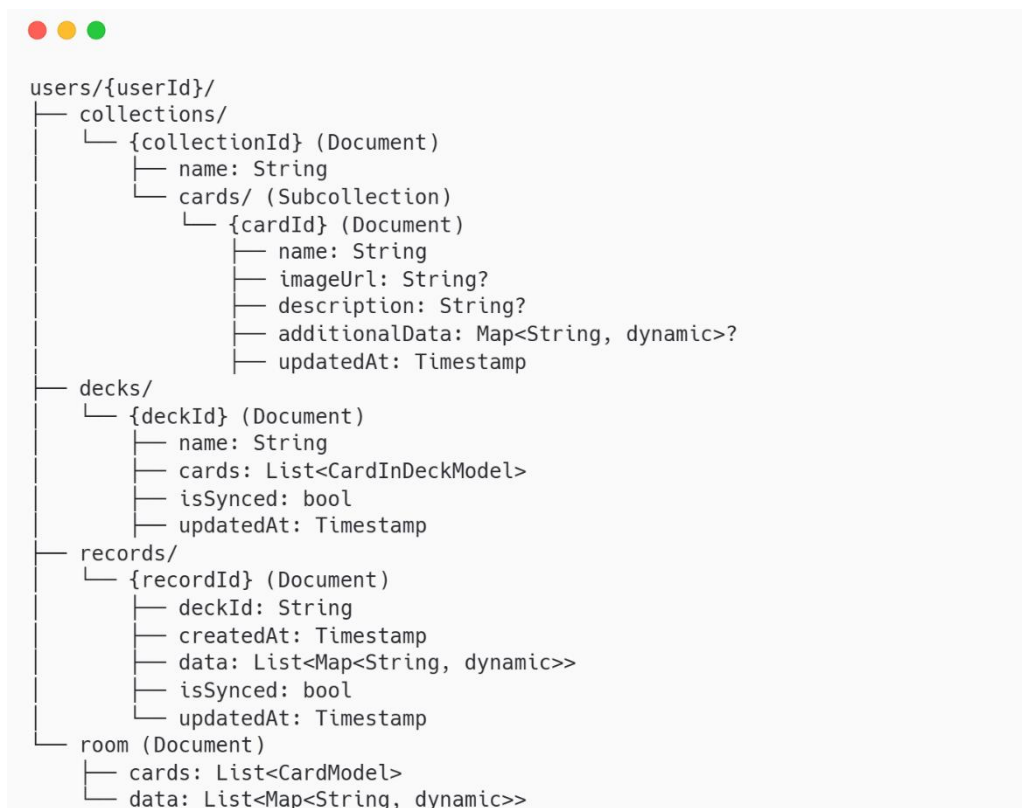
Json Schema (NoSQL / Firestore) ในขณะที่ Firebase Firestore เป็นฐานข้อมูลแบบไม่สัมพันธ์ (non-relational) จึงเหมาะสมกับการจัดเก็บข้อมูลแบบ document-oriented ซึ่งทำให้สามารถเข้าถึงข้อมูลได้รวดเร็วกว่าเมื่อต้องโหลดข้อมูลชุดใหญ่หรือแบบ nested โครงสร้างถูกออกแบบให้ใช้เพียง 3 Document หลัก คือ collections, decks และ records และเก็บข้อมูลทั้งหมดที่เกี่ยวข้องไว้ภายใน document หรือ subcollection เดียวกัน เพื่อ:

- ลดจำนวน query ที่ต้องทำซ้ำหลายรอบ
- ลดค่าใช้จ่ายในการอ่านข้อมูลที่คิดตามจำนวน document read
- สะดวกต่อการใช้งานแบบ offline-first

5.10.3 ความเชื่อมโยงระหว่างสอง schema

แม้โครงสร้างทั้งสองแบบจะต่างกัน แต่ข้อมูลยังคงสอดคล้องกันและแปลงไปมาได้ โดย collections และ cards ถูกรวมเป็น document เดียวใน Firestore พร้อม subcollection cards ส่วน decks และ cardsInDeck รวมกันโดยเก็บเป็นข้อมูลการ์ดนั้นพร้อมจำนวน เพื่อให้ใช้งานได้ง่ายขึ้นและเหมาะสมกับ non-relational มากกว่า ขณะที่ records จัดเก็บแยกเช่นเดิม โครงสร้างนี้ช่วยให้การซิงค์ระหว่าง SQLite และ Firestore ทำได้ง่าย ด้วยฟิลด์ isSynced และ updatedAt ช่วยประหยัดค่าใช้จ่ายและรองรับ incremental sync โดยมีโครงสร้างข้อมูลของฝั่ง Firestore ดังรูปที่ 5.23

หมายเหตุ: รูปภาพที่อัปโหลดจะถูกจัดเก็บไว้ใน Supabase เนื่องจาก Firebase มีข้อจำกัดบางอย่างในการจัดการไฟล์ ดังนั้น Firebase จะเก็บเพียง imageUrl เท่านั้น ขณะที่ไฟล์รูปจริงจะถูกเก็บอยู่บน Supabase

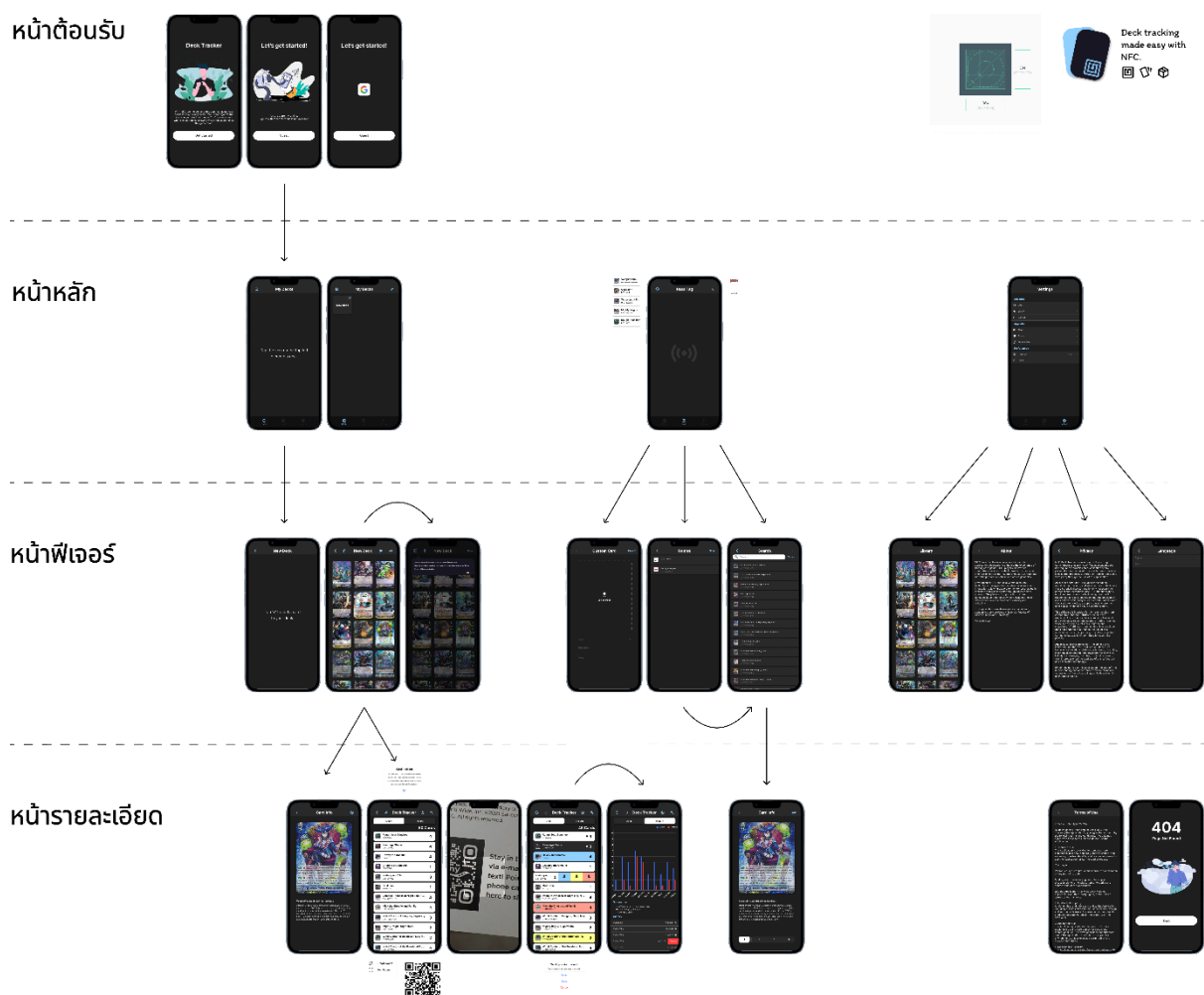


รูปที่ 5.23 โครงสร้างข้อมูลแบบ Document-Oriented ใน Firestore

5.11 การออกแบบส่วนติดต่อผู้ใช้ (UI)

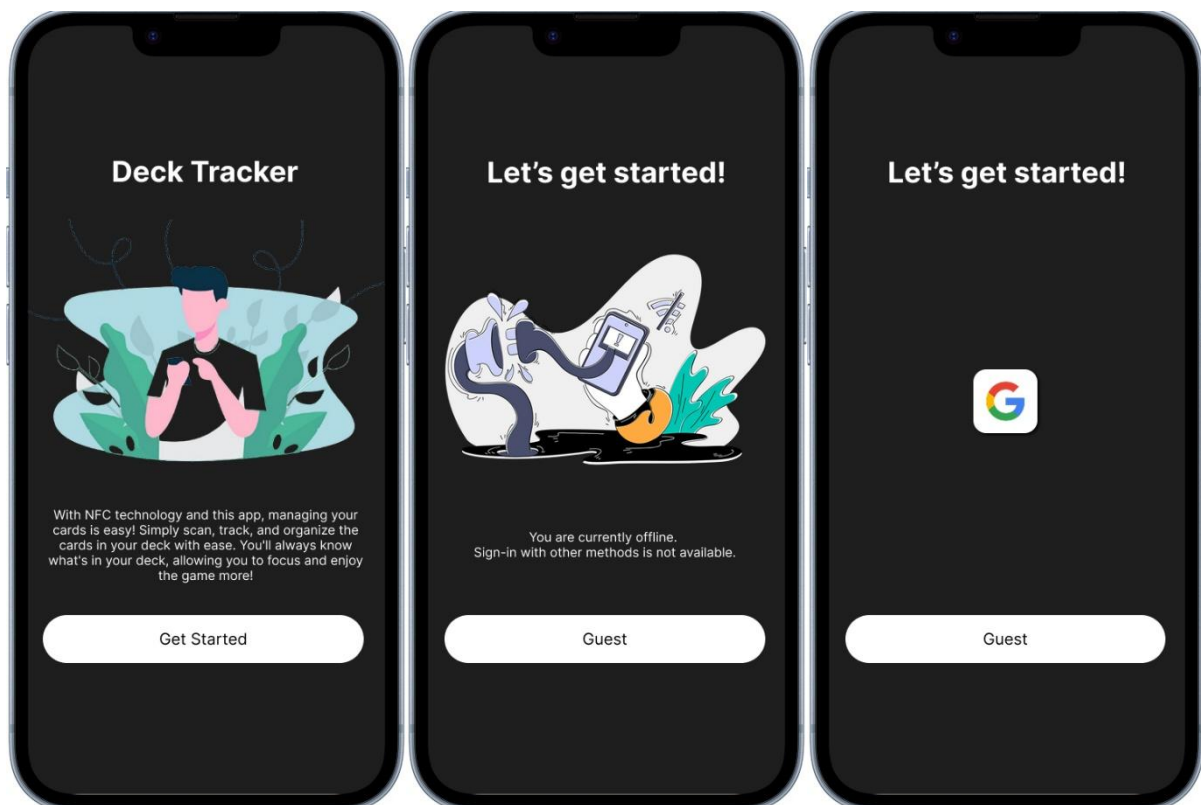
แนวคิดการออกแบบส่วนติดต่อผู้ใช้ (UI) สำหรับแอปพลิเคชันนี้ ตั้งเป้าหมายหลักไปที่ความสะดวกในการใช้งาน โดยเน้นความเรียบง่าย และเลือกใช้ Dark Theme เป็นหลัก เพื่อช่วยลดความเมื่อยล้าของสายตาเมื่อผู้เล่นต้องใช้งานเป็นเวลานาน เพื่อให้ผู้ใช้เข้าถึงฟังก์ชันต่าง ๆ ได้อย่างรวดเร็ว โครงสร้างของแอปพลิเคชันจึงถูกจัดเป็น 4 ระดับหลักดังรูปที่ 5.24 ได้แก่

1. หน้าต้อนรับสำหรับแนะนำฟังก์ชันการทำงานของแอปพลิเคชันและนำผู้ใช้ไปสู่หน้าหลัก
2. หน้าหลักสำหรับการเข้าถึงฟังก์ชันที่ใช้งานบ่อย เช่น การจัดการเต็ค การสแกน NFC และการตั้งค่า
3. หน้าฟิเจอร์สำหรับการทำงานที่มีรายละเอียดมากขึ้น เช่น การเพิ่มการ์ด การค้นหาการ์ด หรือการเลือกภาษา
4. หน้ารายละเอียดสำหรับแสดงข้อมูลเชิงลึกของรายการที่เลือกมา เช่น ข้อมูลการ์ดแต่ละใบ และสถิติการเล่น



รูปที่ 5.24 แผนผังลำดับหน้าจอที่แสดงโครงสร้าง 4 ระดับของแอปพลิเคชัน

5.11.1 หน้าต้อนรับและใช้งาน



รูปที่ 5.25 การเริ่มต้นใช้งาน: หน้าต้อนรับ การเข้าสู่ระบบแบบออฟไลน์ และการเข้าสู่ระบบแบบออนไลน์

Landing Page (ภาพถ่าย)

เป็นหน้าต้อนรับผู้ใช้งานที่เพิ่งเปิดแอปขึ้นมาครั้งแรก โดยแสดงข้อความอธิบายสั้น ๆ เกี่ยวกับวัตถุประสงค์ของแอป พร้อมปุ่ม “Get Started” สำหรับเริ่มต้นการใช้งาน ซึ่งเมื่อผู้ใช้งานกดปุ่มนี้จะนำไปสู่กระบวนการเข้าสู่ระบบหรือใช้งานในโหมด Guest

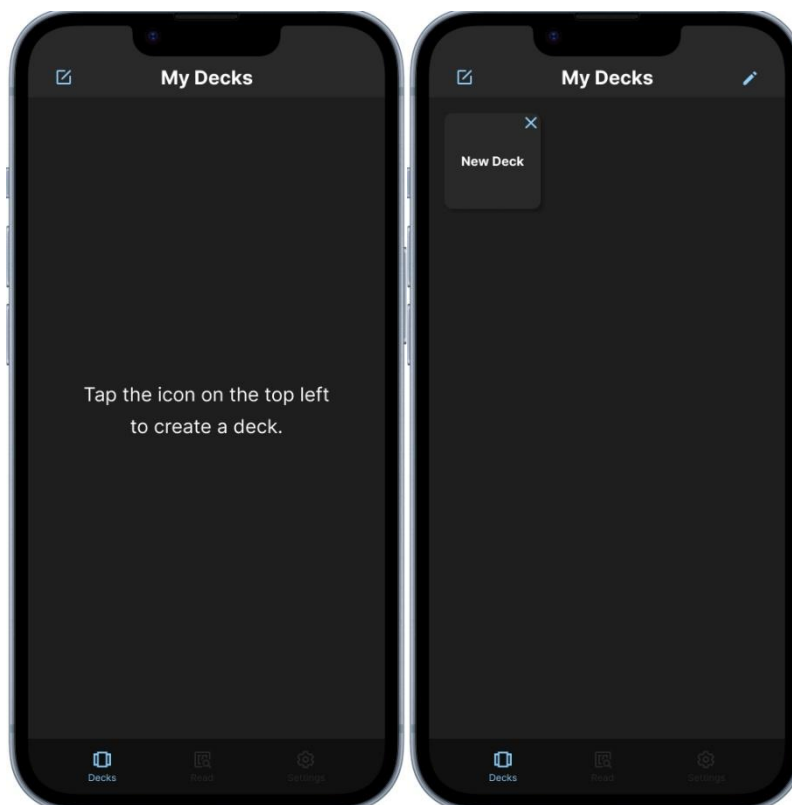
Sign In Page - Offline State (ภาพกลาง)

ในกรณีที่อุปกรณ์ไม่มีการเชื่อมต่ออินเทอร์เน็ต ระบบแสดงหน้าจอแจ้งว่าไม่สามารถเข้าสู่ระบบด้วยวิธีอื่นได้ และอนุญาตให้ผู้ใช้งานเลือก “Guest” เพื่อเข้าสู่ระบบแบบไม่ต้องลงชื่อเข้าใช้ โดยข้อมูลจะถูกบันทึกเฉพาะภายในอุปกรณ์เท่านั้น

Sign In Page - Online State (ภาพขวา)

เมื่ออุปกรณ์สามารถเชื่อมต่ออินเทอร์เน็ตได้ ระบบเปิดให้ผู้ใช้งานสามารถเลือกเข้าสู่ระบบผ่านบัญชี Google Account หรือใช้งานในโหมด Guest ได้ตามความต้องการ ซึ่งการเข้าสู่ระบบด้วยบัญชี Google จะช่วยให้สามารถ Sync ข้อมูลระหว่างอุปกรณ์ และสำรองข้อมูลกับ Firebase ได้

5.11.2 หน้าจัดการเต็ค



รูปที่ 5.26 การจัดการเต็ค: หน้าเมื่อไม่มีเต็ค การเลือกสร้างเต็คใหม่ และการลบเต็ค

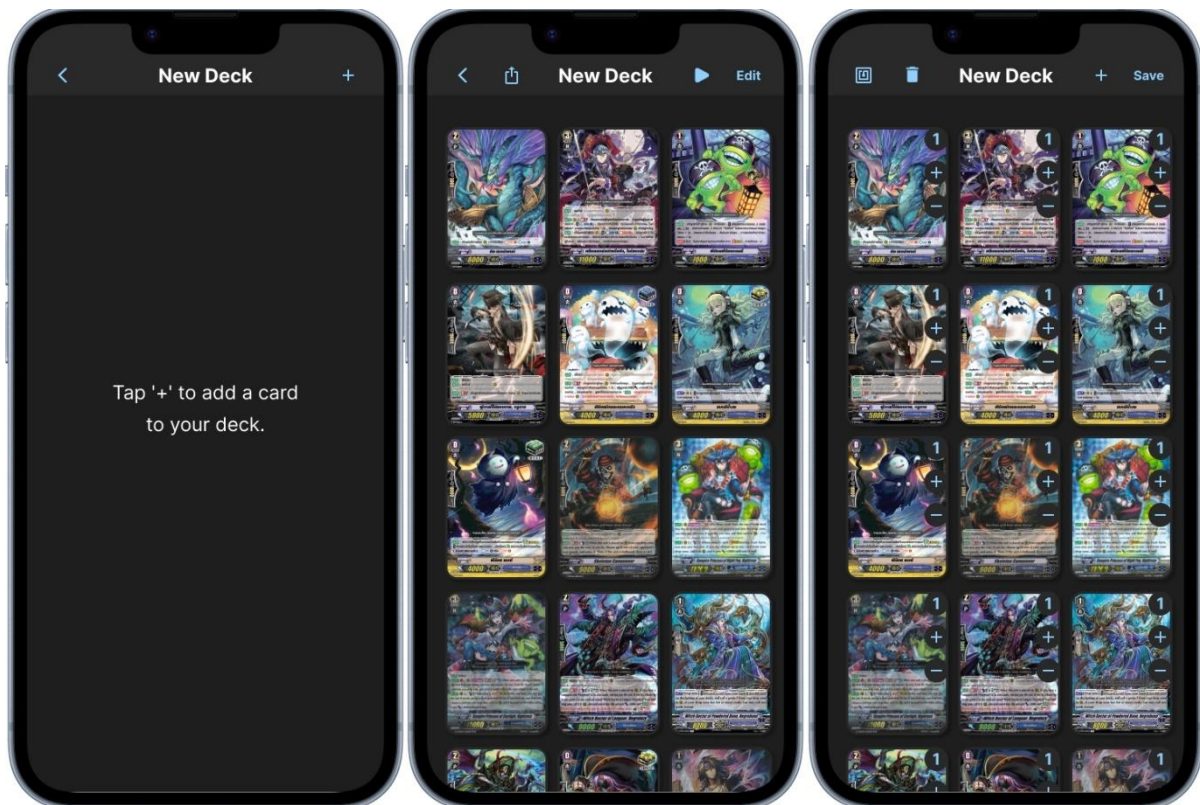
My Deck Page - Empty State (ภาพซ้าย)

หน้าจอนี้แสดงเมื่อผู้ใช้อยู่ยังไม่มีเต็คในระบบ โดยจะมีข้อความแนะนำให้เริ่มต้นด้วยการสร้างเต็คใหม่ผ่านไอคอนที่อยู่ด้านบนซ้ายของหน้าจอ เมื่อผู้ใช้แตะไอคอนดังกล่าว ระบบจะนำไปยังหน้าสำหรับสร้างเต็คใหม่

My Deck Page - Edit State (ภาพขวา)

เมื่อมีเต็คอย่างน้อยหนึ่งรายการในระบบ หน้าจอจะแสดงรายการเต็คของผู้ใช้ พร้อมแสดงไอคอน “ปากกา” ที่มุมขวาบน เพื่อให้ผู้ใช้สามารถเข้าสู่โหมดแก้ไข เช่น การลบเต็คที่ไม่ต้องการได้

5.11.3 หน้าสร้างและแก้ไขดึก



รูปที่ 5.27 การสร้างดึก: หน้าดึกเปล่า การแสดงผลการ์ดในดึก และโหมดแก้ไข

Deck Builder Page – Empty State (ภาพซ้าย)

หากยังไม่มีการ์ดในดึก หน้าจอจะแสดงข้อความแนะนำให้แตะปุ่ม “+” ที่มุมขวาบนเพื่อเพิ่มการ์ด ระบบจะนำผู้ใช้ไปยังหน้าสำหรับเลือกเกมและเลือกรายชื่อการ์ดที่ต้องการเพิ่มเข้าสู่ดึก

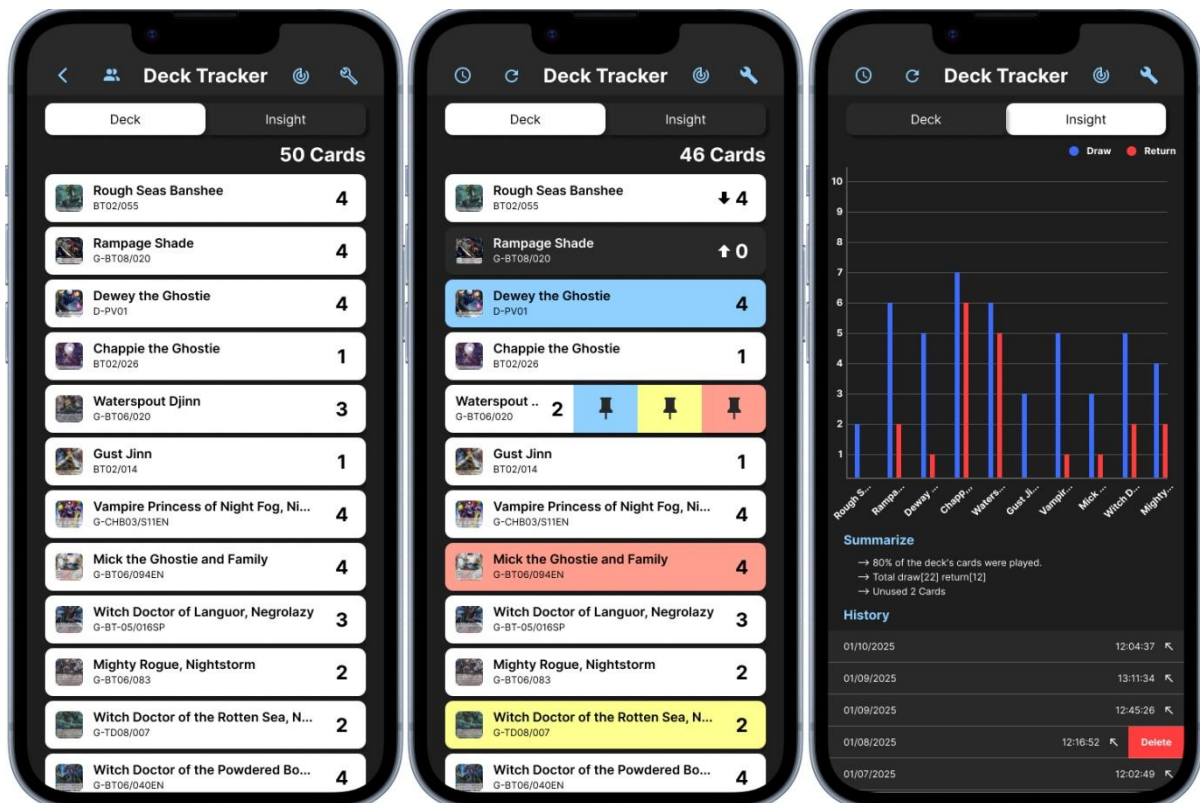
Deck Builder Page – View State (ภาพกลาง)

เมื่อมีการ์ดอยู่ในดึก หน้าจอจะแสดงรายการการ์ดในรูปแบบกริด พร้อมรายละเอียด เช่น ภาพการ์ด จำนวนที่ใช้ชื่อดึก และสามารถแตะการ์ดเพื่อดูรายละเอียดเพิ่มเติมได้

Deck Builder Page – Edit State (ภาพขวา)

เมื่อผู้ใช้แตะปุ่ม “Edit” ระบบจะเข้าสู่โหมดแก้ไข ผู้ใช้สามารถเลือกรายการการ์ดเพื่อเขียนข้อมูลลงบนแท็ก NFC หรือปรับจำนวนการ์ดในแต่ละใบให้เหมาะสมกับกลยุทธ์ของตนเอง หากเป็นการเข้าสู่โหมดแก้ไขครั้งแรก ระบบจะแสดง Tutorial แนะนำวิธีการเขียนและอ่านข้อมูลบนแท็ก NFC โดยมีคำแนะนำขั้นตอนการใช้งานอย่างชัดเจน เพื่อให้ผู้ใช้สามารถเริ่มต้นใช้งานฟิเจอร์นี้ได้อย่างมั่นใจ

5.11.4 หน้าติดตามและวิเคราะห์deck



รูปที่ 5.28 การติดตามและวิเคราะห์deck: หน้าแสดงรายการการ์ด โหมดติดตามการเล่น และโหมดวิเคราะห์สถิติ

Deck Tracker – Standby State (ภาพซ้าย)

แสดงรายการการ์ดทั้งหมดภายในเด็ค พร้อมจำนวนที่เหลืออยู่ของแต่ละใบ ซึ่งจัดเรียงในรูปแบบตารางแนวนอง ผู้ใช้สามารถดูข้อมูลได้อย่างรวดเร็วและสะดวก

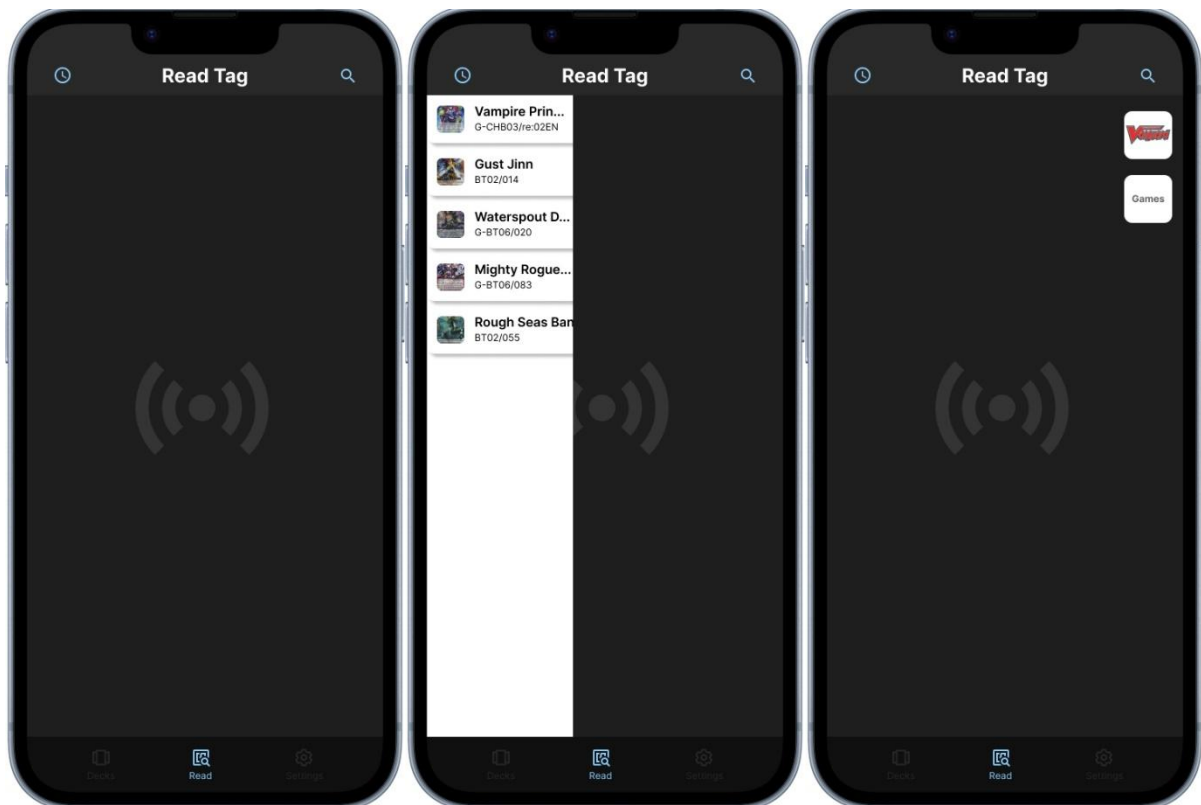
Deck Tracker – Tracking State (ภาพกลาง)

เมื่อมีการแตะหรืออ่านแท็ก NFC ของการ์ดแต่ละใบ ระบบจะอัปเดตจำนวนการ์ดแบบเรียลไทม์ พร้อมแสดงไอคอนลูกศรเพื่อระบุสถานะของการ์ดว่าอยู่ในเด็ค (เข้า) หรือถูกนำออกจากเด็ค (ออก) นอกจากนี้ ผู้ใช้ยังสามารถทำเครื่องหมายกับการ์ดที่ต้องการให้สังเกตเป็นพิเศษโดยใช้ สีฟ้า สีเหลือง สีแดง

Deck Tracker – Insight State (ภาพขวา)

แสดงผลการวิเคราะห์เด็คในรูปแบบกราฟแท่ง โดยแยกจำนวนการ์ดที่ถูกจับและจำนวนที่ถูกนำออกจากเด็คอย่างชัดเจน พร้อมสรุปสถิติสำคัญ เช่น จำนวนการ์ดที่ไม่ถูกใช้งาน และประวัติการเล่นย้อนหลังของแต่ละรอบ โดยมีเวลาและวันที่กำกับอย่างละเอียด เพื่อให้ผู้ใช้สามารถวิเคราะห์พฤติกรรมการเล่นได้

5.11.5 หน้าอ่านแท็ก



รูปที่ 5.29 การอ่านแท็ก NFC: หน้าสแกนแท็ก การแสดงประวัติ และเมนูเลือกเกม

Read Tag Page – Standby State (ภาพซ้าย)

เป็นหน้าจอเริ่มต้นที่แสดงไอคอน NFC เพื่อแสดงสถานะว่าระบบพร้อมสำหรับการอ่านข้อมูลจากแท็ก เมื่อผู้ใช้นำแท็ก NFC มาแตะกับอุปกรณ์ ระบบจะดำเนินการอ่านทันที หลังจากทีระบบอ่านข้อมูลจากแท็กสำเร็จแล้ว ระบบจะแจ้งสถานะของแท็กที่อ่านได้ เพื่อให้ผู้ใช้ทราบข้อมูลที่ได้รับ และสามารถดำเนินการต่อได้อย่างเหมาะสม

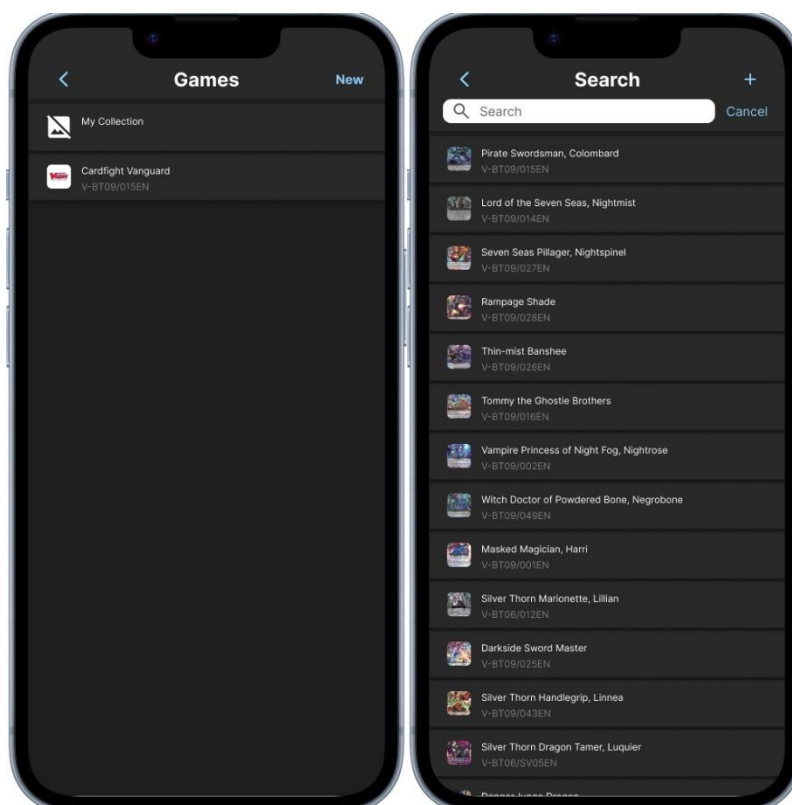
Read Tag Page – History State (ภาพกลาง)

หลังจากอ่านแท็กสำเร็จ ระบบจะแสดงรายการการ์ดที่อยู่ในแท็ก พร้อมรายละเอียดการ์ดแต่ละใบในรูปแบบรายการแนวนอน เพื่อให้ผู้ใช้สามารถตรวจสอบข้อมูลได้อย่างรวดเร็ว

Read Tag Page – Collection State (ภาพขวา)

เมื่อแตะไอคอนเมนูที่มุมขวาบน จะมีการแสดง Drawer Menu ซึ่งจะแสดงโลโก้เกมที่ค้นหาล่าสุด และปุ่ม “Games” สำหรับนำผู้ใช้ไปยังหน้ารายการเกมทั้งหมด (Collection list) เพื่อค้นหาคาร์ดของเกมอื่นเพิ่มเติม

5.11.6 หน้าเลือกคอลเลกชันและค้นหาการ์ด



รูปที่ 5.30 การเลือกและค้นหาการ์ด: หน้าเลือกคอลเลกชัน และหน้าค้นหารายการการ์ด

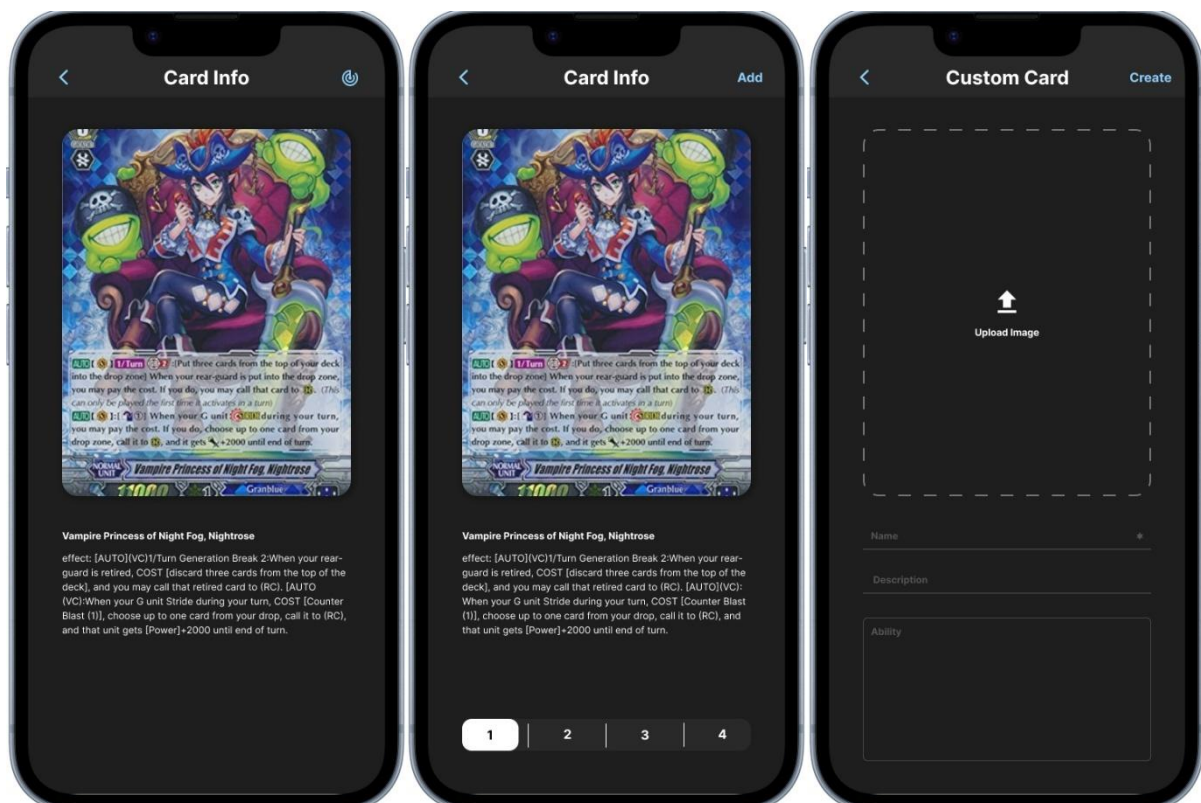
Collection Page (ภาพถ่าย)

แสดงรายการเกมทั้งหมดที่มีอยู่ในระบบ โดยผู้ใช้สามารถเลือกเกมที่ต้องการเพื่อเข้าสู่หน้ารายการการ์ดของเกม นั้น ๆ ระบบยังมีรายการพิเศษคือ “My Collection” สำหรับจัดเก็บการ์ดที่ผู้ใช้สร้างขึ้นเองหรือการ์ดที่ไม่ได้เชื่อมโยงกับเกมใดโดยเฉพาะ ผู้ใช้งานสามารถสร้างคอลเลกชันใหม่ของตนเองได้โดยแตะที่ไอคอน “New” ซึ่งอยู่มุมขวาบนของหน้าจอ

Browse Card Page (ภาพขาว)

หลังจากเลือกเกมแล้วระบบจะแสดงรายการการ์ดทั้งหมดในเกมดังกล่าว ผู้ใช้สามารถค้นหาการ์ดเฉพาะใบได้อย่างรวดเร็วผ่านช่อง Search Bar ที่อยู่ด้านบนของหน้าจอ ในกรณีที่ผู้ใช้อยู่ในคอลเลกชันส่วนตัว “My Collection” สามารถเพิ่มการ์ดใหม่ได้โดยแตะไอคอน “+” ที่มุมขวาบน เมื่อกดแล้วระบบจะนำไปยังหน้าสำหรับสร้างการ์ดแบบกำหนดเอง “Card Custom Page”

5.11.7 หน้ารายละเอียดการ์ด และสร้างการ์ดแบบกำหนดเอง



รูปที่ 5.31 การจัดการข้อมูลการ์ด: หน้าแสดงข้อมูล, การเพิ่มการ์ดลงเด็ด และการสร้างการ์ดเอง

Card Page – Info State (ภาพซ้าย)

แสดงรายละเอียดของการ์ด โดยมีทั้งภาพการ์ด ชื่อการ์ด และข้อความรายละเอียดหรือเอฟเฟกต์ของการ์ด ด้านขวามือมีปุ่มไอคอน “Radar” สำหรับเขียนข้อมูลการ์ดลงในแท็ก NFC เมื่อผู้ใช้ดำเนินการเขียนข้อมูล ระบบจะแสดงสถานะผลลัพธ์ของการเขียน ดังนี้ **เขียนสำเร็จ** – ระบบแจ้งว่าการ์ดถูกเขียนลงในแท็กเรียบร้อยแล้ว **เขียนไม่สำเร็จ** – ระบบแสดงข้อความแจ้งข้อผิดพลาดตามกรณีที่เกิดขึ้น **เขียนซ้ำการ์ดใบเดิม** – หากผู้ใช้พยายามเขียนการ์ดใบเดิมลงบนแท็กใบเดิม ระบบจะแจ้งว่ามีข้อมูลเดิมอยู่แล้ว

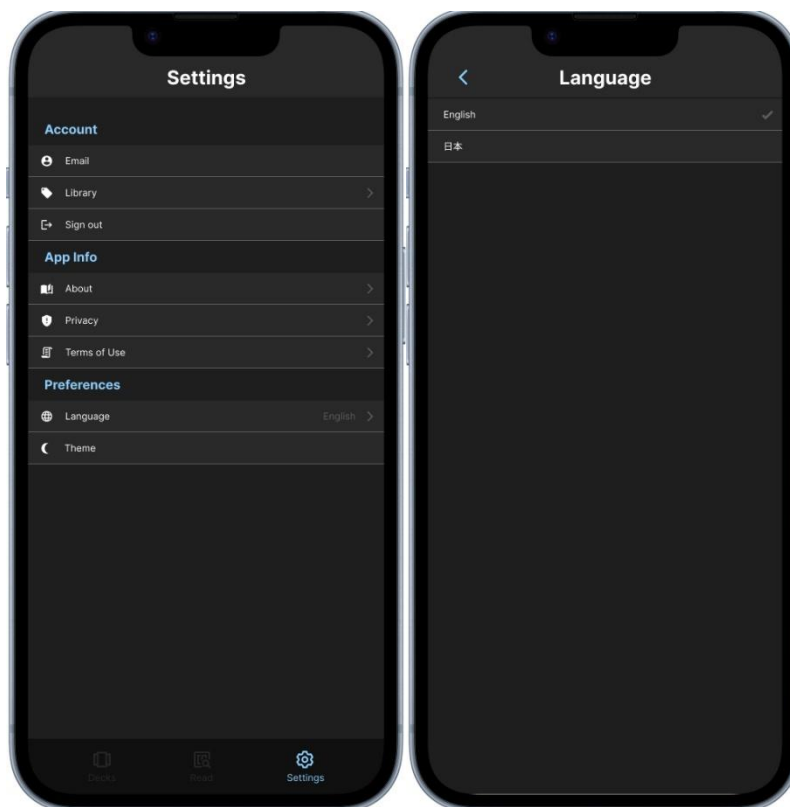
Card Page – Add State (ภาพกลาง)

หากเข้าถึงหน้านี้จากหน้า “Deck Create Page” จะปรากฏปุ่ม “Add” ด้านขวามือ เพื่อเพิ่มการ์ดที่เลือกเข้าสู่เด็ด โดยสามารถกำหนดจำนวนการ์ดที่จะเพิ่มจากตัวเลือกด้านล่างของหน้าจอได้

Card Page – Custom State (ภาพขวา)

หน้านี้มาจากไอคอน “+” ที่หน้า “Card Search Page” ใช้สำหรับสร้างการ์ดใหม่แบบกำหนดเอง โดยผู้ใช้สามารถอัปโหลดรูปภาพการ์ด และกรอกชื่อ คำอธิบาย และรายละเอียดของการ์ดได้ตามต้องการ จากนั้นสามารถบันทึกเข้าสู่คอลเล็กชันเพื่อใช้งานต่อไปได้

5.11.8 หน้าตั้งค่าและเลือกภาษา



รูปที่ 5.32 การตั้งค่าแอปพลิเคชัน: หน้าตั้งค่าหลัก และหน้าเลือกภาษา

Setting Page (ภาพซ้าย)

Account:

- Email: แสดงอีเมลของผู้ใช้งานที่เข้าสู่ระบบ
- Library: เข้าถึงรายการการนัดทั้งหมดที่ผู้ใช้มีในเด็ค
- Sign out: สำหรับออกจากระบบ หรือเข้าสู่ระบบใหม่

App Info:

- About: แสดงข้อมูลทั่วไปเกี่ยวกับแอป
- Privacy: แสดงนโยบายความเป็นส่วนตัวส่วนตัวของผู้ใช้
- Terms of Use: แสดงข้อกำหนดและเงื่อนไขการใช้งาน

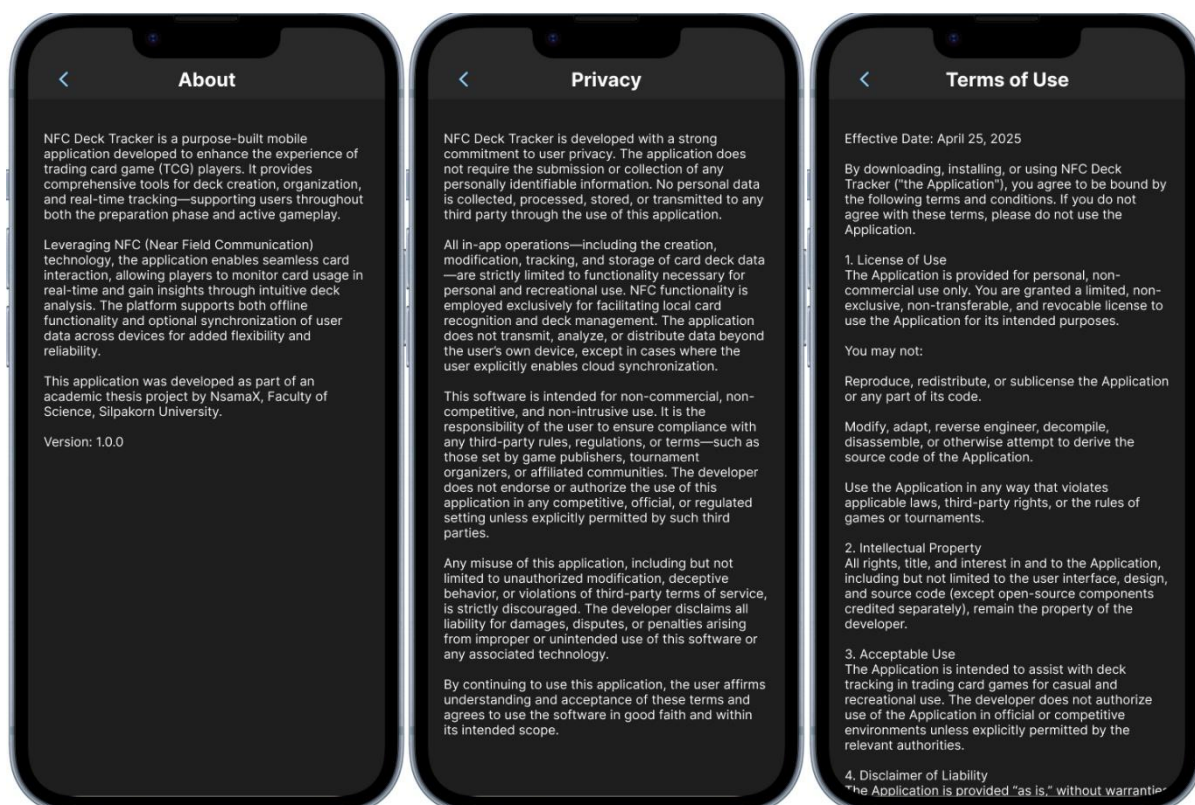
Preferences:

- Language: เข้าสู่หน้าการเลือกภาษา
- Theme: เปลี่ยนธีมของแอป

Language Page (ภาพขวา)

ผู้ใช้งานสามารถเลือกภาษาที่ต้องการใช้งานได้ โดยในเวอร์ชันปัจจุบันแอปรองรับภาษาอังกฤษและภาษาญี่ปุ่น ภาษาถูกแสดงในรูปแบบของรายการพร้อมเครื่องหมายถูกเพื่อระบุภาษาที่กำลังใช้งานอยู่

5.11.9 หน้า About / Privacy Policy / Terms of Use



รูปที่ 5.33 ข้อมูลแอปพลิเคชัน: เกี่ยวกับแอป นโยบายความเป็นส่วนตัว และข้อกำหนดการใช้งาน

About Page (ภาพซ้าย)

หน้าจอแสดงข้อมูลเกี่ยวกับแอป NFC Deck Tracker รวมถึงวัตถุประสงค์ของแอป การใช้งานหลัก และรายละเอียดผู้พัฒนา พร้อมระบุว่าเป็นโครงการในระดับปริญญาตรี และแสดงเวอร์ชันปัจจุบันของแอป

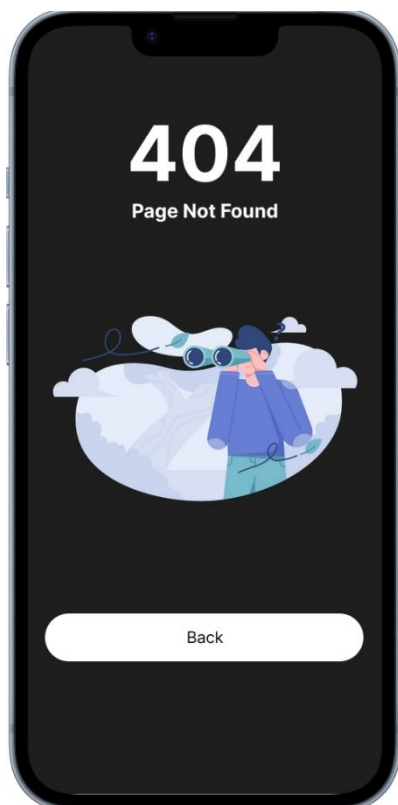
Privacy Page (ภาพกลาง)

หน้าจอแสดงนโยบายความเป็นส่วนตัว โดยระบุว่าแอปไม่เก็บข้อมูลส่วนบุคคลของผู้ใช้ และการใช้งานทั้งหมดจะถูกจัดการภายในอุปกรณ์ ยกเว้นกรณีที่ผู้ใช้เลือกซิงค์ข้อมูลด้วยตนเอง

Terms of Use Page (ภาพขวา)

หน้าจอแสดงข้อกำหนดการใช้งานของแอป เช่น ห้ามใช้ในเชิงพาณิชย์หรือดัดแปลงโค้ด และระบุว่าแอปให้บริการตามสภาพ โดยไม่รับผิดชอบต่อการใช้งานผิดวัตถุประสงค์

5.11.10 หน้าข้อผิดพลาด (404 – Page Not Found)



รูปที่ 5.34 หน้าจอแสดงข้อผิดพลาด 404 (Page Not Found)

Page Not Found

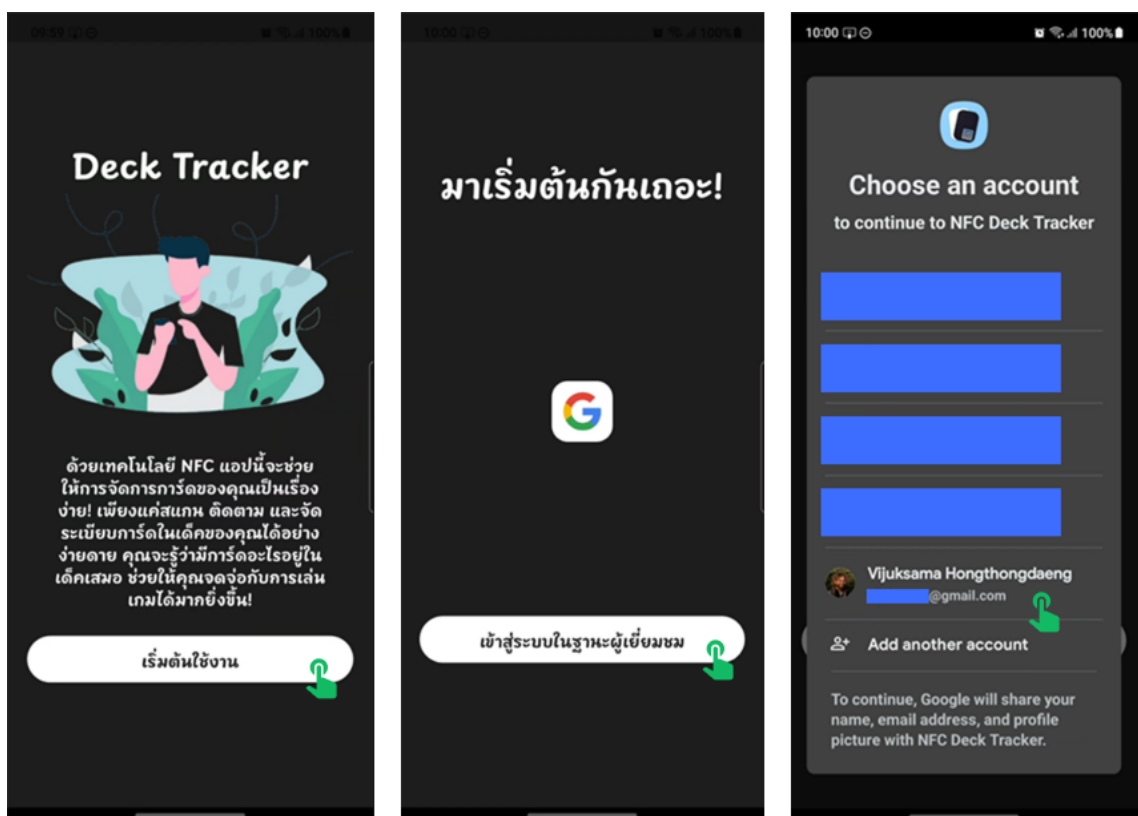
หน้าจอนี้จะแสดงขึ้นเมื่อพยายามเข้าถึงหน้าที่ไม่มีอยู่ในระบบ หรือเกิดข้อผิดพลาดระหว่างการนำทาง นอกจากนี้หน้าดังกล่าวยังถูกจัดเตรียมไว้เพื่อใช้ในการทดสอบระบบในระหว่างการพัฒนา และเพื่อป้องกันไม่ให้เกิดข้อผิดพลาดร้ายแรงที่อาจส่งผลให้แอปพลิเคชันปิดตัวลงอย่างกะทันหันเมื่อไม่สามารถโหลดหน้าที่ต้องการได้

บทที่ 6 ผลการดำเนินงาน ข้อจำกัด และข้อเสนอแนะ

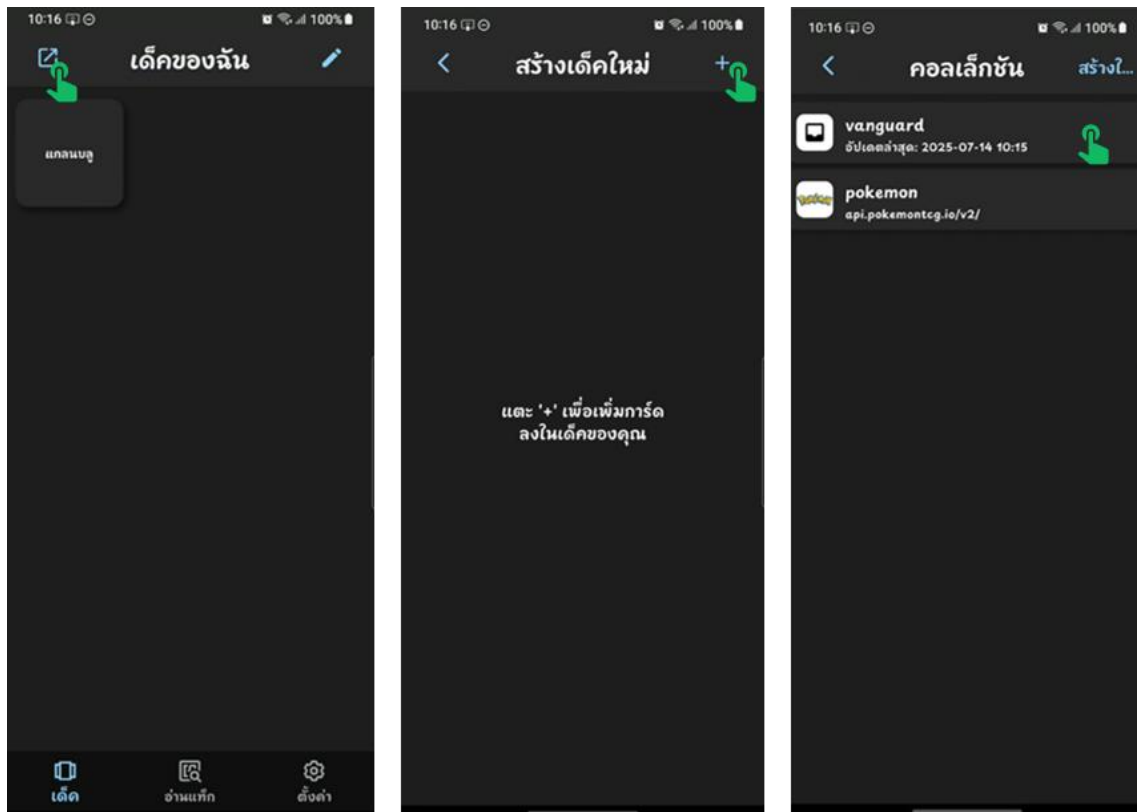
บทนี้กล่าวถึงผลการดำเนินงานที่ได้จากการพัฒนาแอปพลิเคชัน NFC Deck Tracker โดยเริ่มจากการสาธิตการทำงานของแอปพลิเคชันบนอุปกรณ์จริง จากนั้นจะกล่าวถึงข้อจำกัดที่พบระหว่างกระบวนการพัฒนา ทั้งในด้านเทคนิค การออกแบบ และการใช้งานจริง รวมถึงแนวทางที่สามารถนำไปใช้ในการปรับปรุง หรือพัฒนาเพิ่มเติมในอนาคตเพื่อให้ระบบมีความสมบูรณ์มากยิ่งขึ้น

6.1 ผลการดำเนินงาน

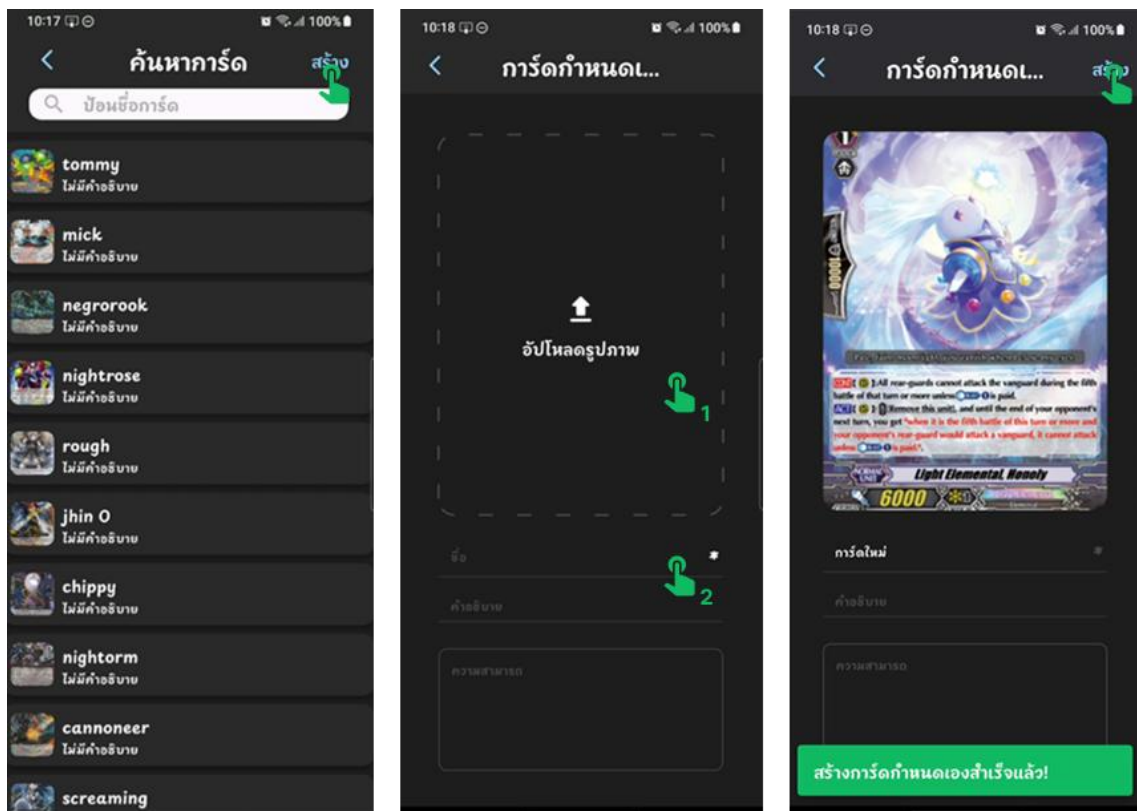
ในส่วนนี้จะเป็นการสาธิตการทำงานของแอปพลิเคชัน NFC Deck Tracker บนอุปกรณ์จริง โดยรูปภาพต่อไปนี้แสดงลำดับขั้นตอนการใช้งานตามลำดับตั้งแต่ การเข้าสู่แอปพลิเคชันไปจนถึงการใช้งานฟังก์ชันต่าง ๆ (โดยมีสัญลักษณ์  ใช้แสดงตำแหน่งที่ผู้ใช้กดในแต่ละขั้นตอน)



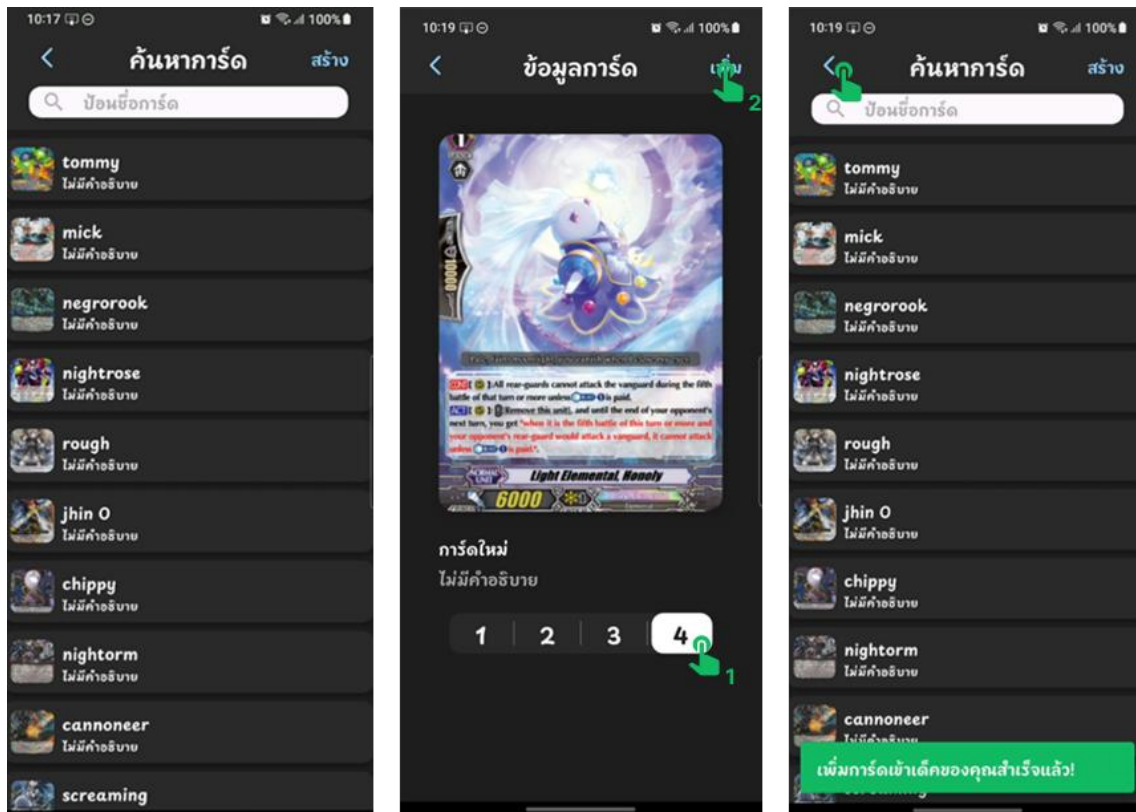
รูปที่ 6.1 ขั้นตอนการลงชื่อเข้าใช้งาน: หน้าต้อนรับ การเลือกวิธีการเข้าสู่ระบบ และการเลือกบัญชีผู้ใช้



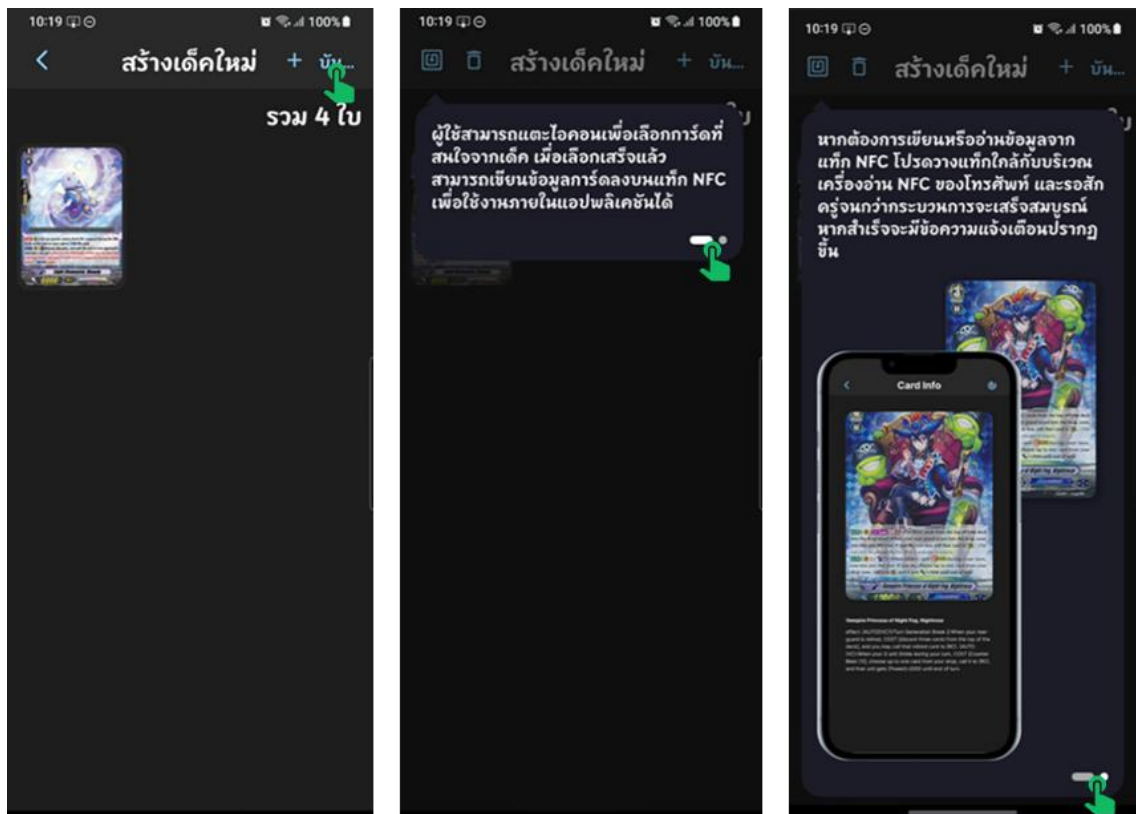
รูปที่ 6.2 ขั้นตอนการสร้างเตีคใหม่: การเลือกเมนูสร้างเตีค การเตรียมเพิ่มการ์ด และการเลือกคอลเลคชัน



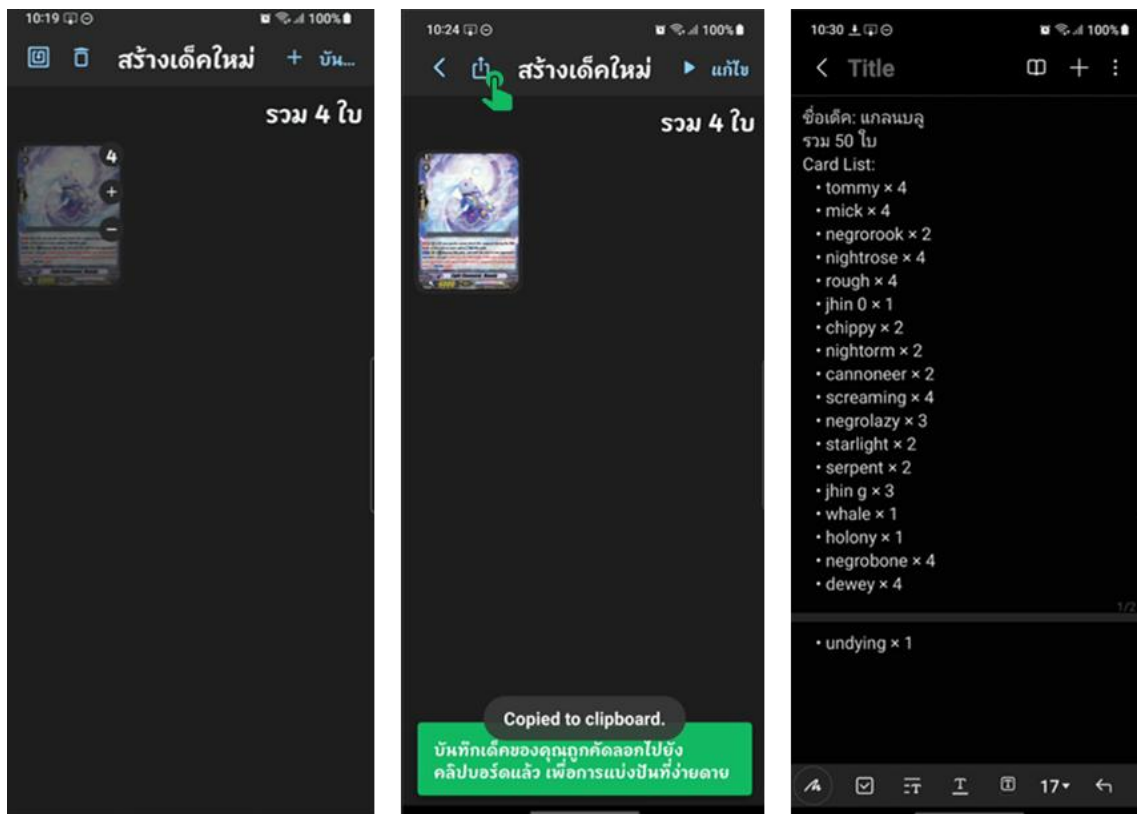
รูปที่ 6.3 ขั้นตอนการสร้างการ์ด: การสร้างการ์ดใหม่ การกรอกรายละเอียด และการยืนยันเพื่อสร้างการ์ด



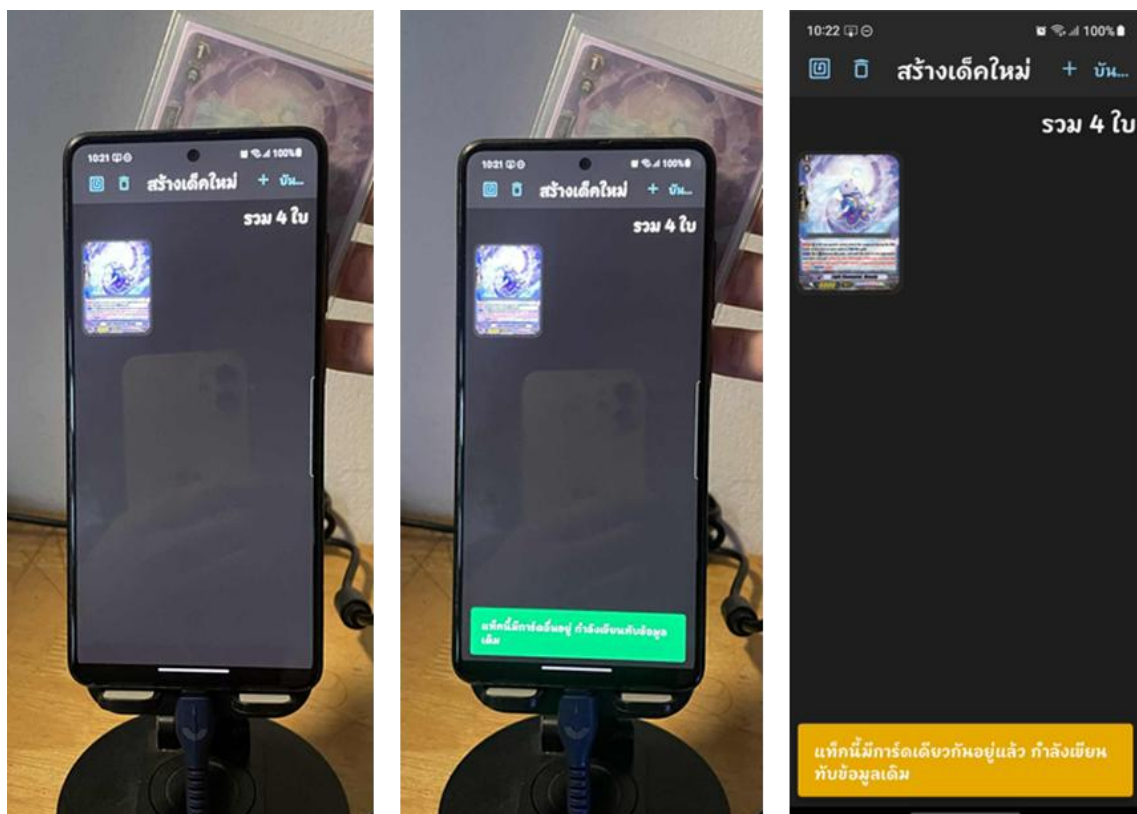
รูปที่ 6.4 ขั้นตอนการเพิ่มการ์ดเข้าเด็ค: การค้นหาการ์ด การเลือกจำนวนและยืนยัน และการแสดงผลพัชร์



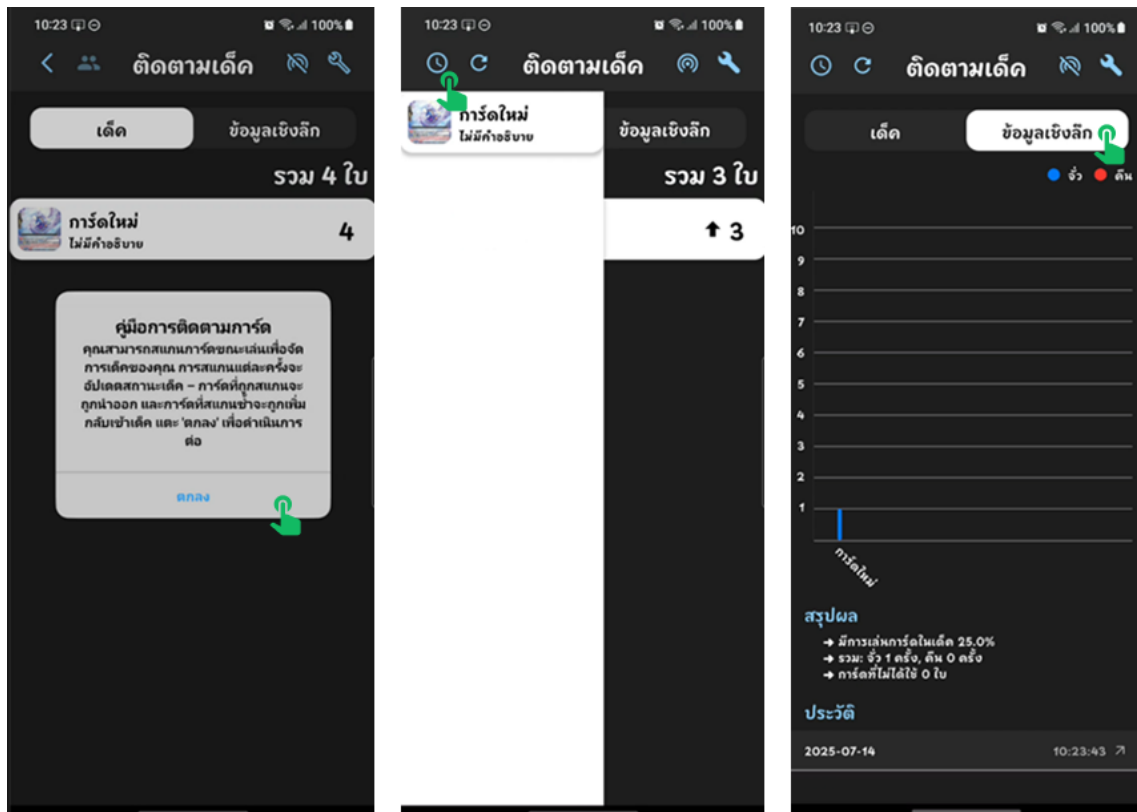
รูปที่ 6.5 ขั้นตอนการสอนใช้ (Tutorial) ฟังก์ชันเขียน NFC: การบันทึกเด็ค คำแนะนำการใช้งานครั้งแรก



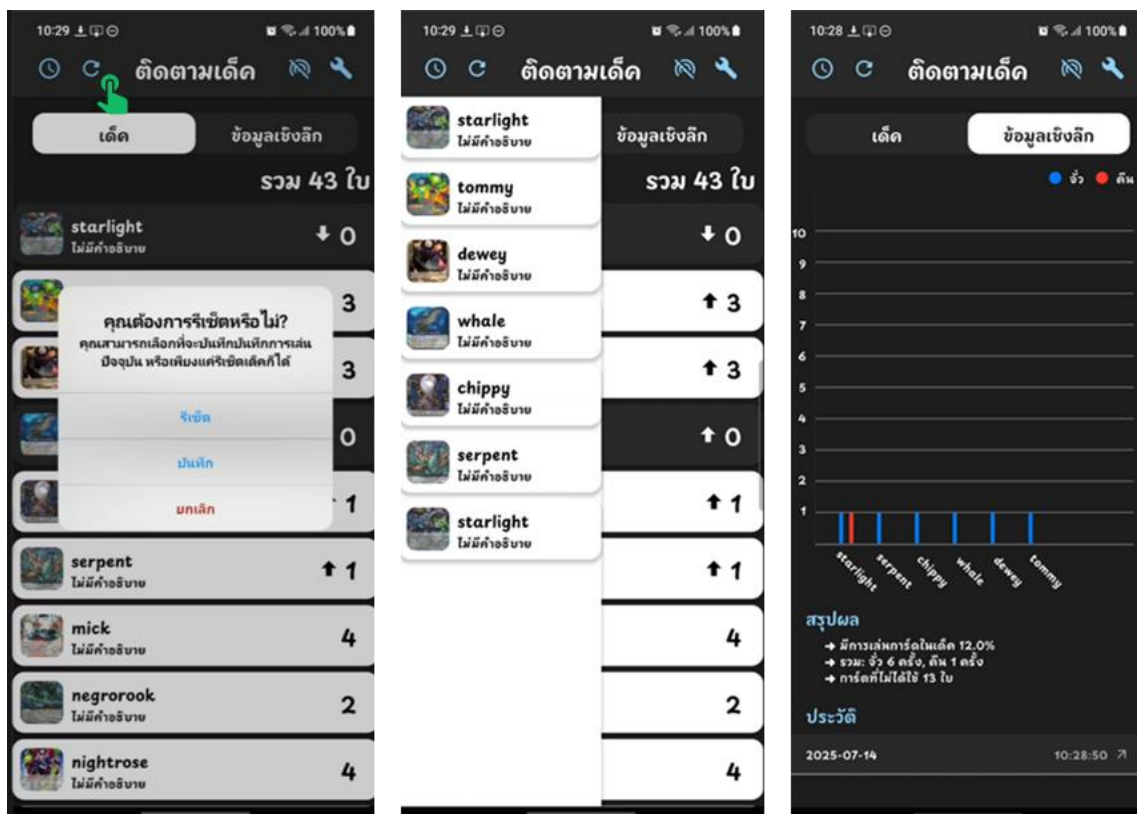
รูปที่ 6.6 ขั้นตอนการแชร์มูลเด็ก: หน้าเด็กที่สร้างเสร็จสิ้น การคัดลอกข้อมูล และตัวอย่างข้อมูลเมื่อนำไปวาง



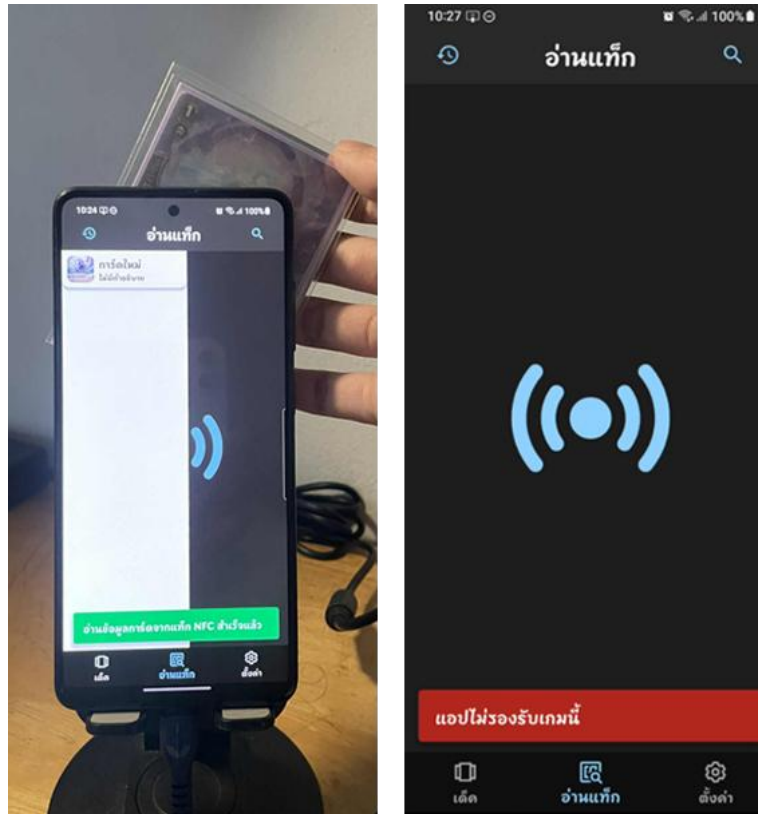
รูปที่ 6.7 ขั้นตอนการทดสอบการเขียน NFC: สถานะก่อนการเขียน การเขียนข้อมูลสำเร็จ และการเขียนข้อมูลซ้ำ



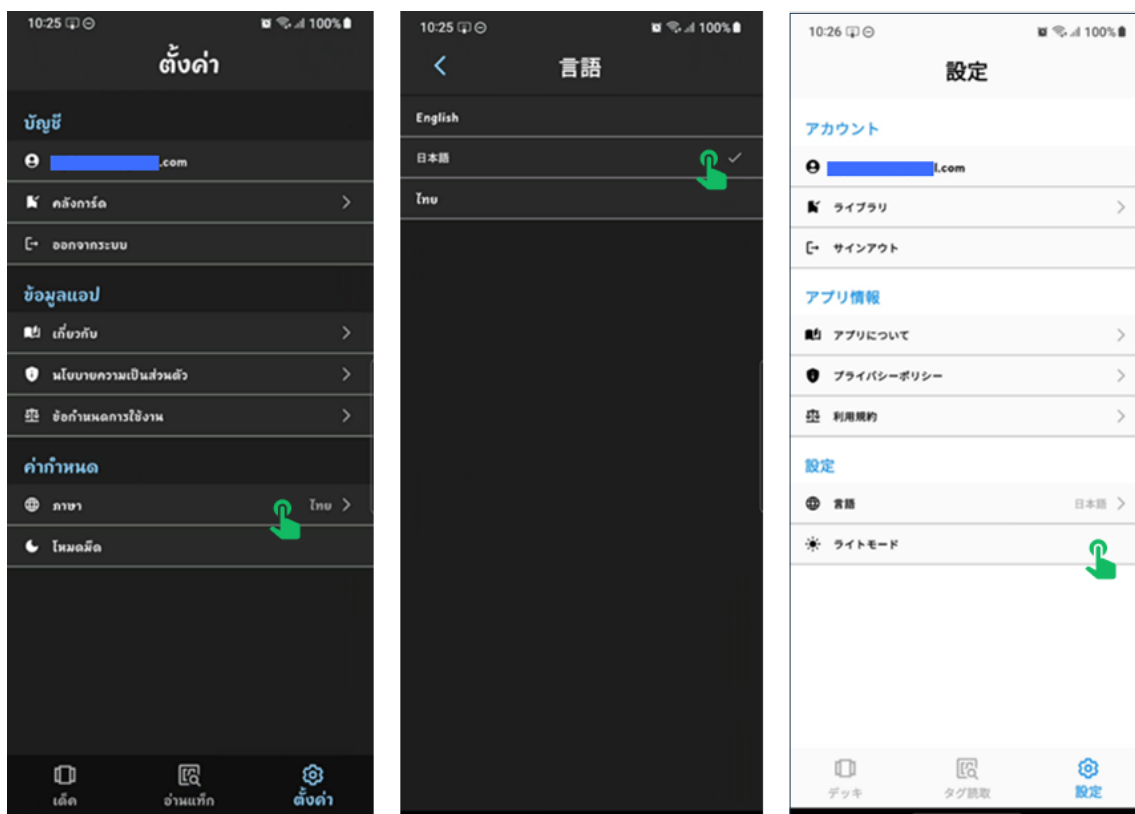
รูปที่ 6.8 ขั้นตอนการติดตามเด็ก: หน้าจอเริ่มต้นพร้อมคำแนะนำ การอัปเดตสถานะการ์ด และการแสดงผลสถิติ



รูปที่ 6.9 ฟังก์ชันรีเซตและดูประวัติ: ตัวเลือกการรีเซตเด็ก ประวัติการเล่น และสถิติการเล่น



รูปที่ 6.10 การทดสอบฟังก์ชันอ่าน NFC: กรณีอ่านข้อมูลสำเร็จ และกรณีข้อมูลไม่รองรับ



รูปที่ 6.11 ขั้นตอนการเปลี่ยนภาษาและธีม: การเลือกเมนูภาษา การเปลี่ยนเป็นภาษาญี่ปุ่น การเปลี่ยนธีมสว่าง

6.2 ข้อจำกัด

- ข้อจำกัดด้านการประเมินผลกับผู้ใช้ โครงการนี้มุ่งเน้นที่การออกแบบและพัฒนาต้นแบบของแอปพลิเคชันให้ทำงานได้ตามฟังก์ชันที่กำหนด จึงยังไม่มีผลการประเมินผลความง่ายในการใช้งานกับกลุ่มผู้ใช้จริงอย่างเป็นทางการ ทำให้ข้อสรุปด้านประสบการณ์ผู้ใช้อย่างเป็นเพียงการประเมินจากมุมมองของผู้พัฒนาตามหลักการออกแบบเท่านั้น
- ข้อจำกัดด้านฮาร์ดแวร์ ระบบต้องใช้งานร่วมกับสมาร์ทโฟนที่รองรับเทคโนโลยี NFC เท่านั้น ซึ่งอุปกรณ์บางรุ่นโดยเฉพาะในกลุ่มราคาประหยัด อาจไม่มีฟังก์ชันนี้ ทำให้ผู้ใช้งานบางกลุ่มไม่สามารถใช้ฟีเจอร์สำคัญของแอปได้
- ข้อจำกัดของแท็ก NFC แท็ก NFC ที่ใช้มีพื้นที่จัดเก็บข้อมูลจำกัด ทำให้สามารถบันทึกได้เพียงชื่อเกมและรหัสการ์ด ไม่สามารถเก็บรายละเอียดอื่นเพิ่มเติม เช่น ชื่อหรือเอฟเฟกต์ของการ์ดได้โดยตรงบนแท็ก นอกจากนี้ แท็ก NFC ยังมีความเสี่ยงต่อการขโมยหรือเสียหายจากการใช้งาน เช่น การโค้งงอหรือรอยขีดข่วน ซึ่งเมื่อแท็กเสียหายจะทำให้ไม่สามารถอ่านหรือเขียนข้อมูลได้ ส่งผลให้ไม่สามารถใช้งานฟีเจอร์ติดตามการ์ดได้ตามปกติ
- ข้อจำกัดจากการพึ่งพา API ภายนอก เนื่องจากแอปพลิเคชันในปัจจุบันยังพึ่งพา API ของผู้ให้บริการภายนอกในการดึงข้อมูลการ์ด หากผู้ให้บริการปิดระบบหรือเกิดปัญหาอื่น อาจกระทบต่อการแสดงผลข้อมูลภายในแอป
- ข้อจำกัดของการใช้งานระหว่างการแข่งขัน การใช้แอปพลิเคชันในการแข่งขันอย่างเป็นทางการอาจยังไม่ได้ยอมรับโดยสมาคมหรือผู้จัดงาน เนื่องจากอาจมีข้อกังวลเรื่องความยุติธรรมหรือการได้เปรียบจากเทคโนโลยี

6.3 ข้อเสนอแนะ

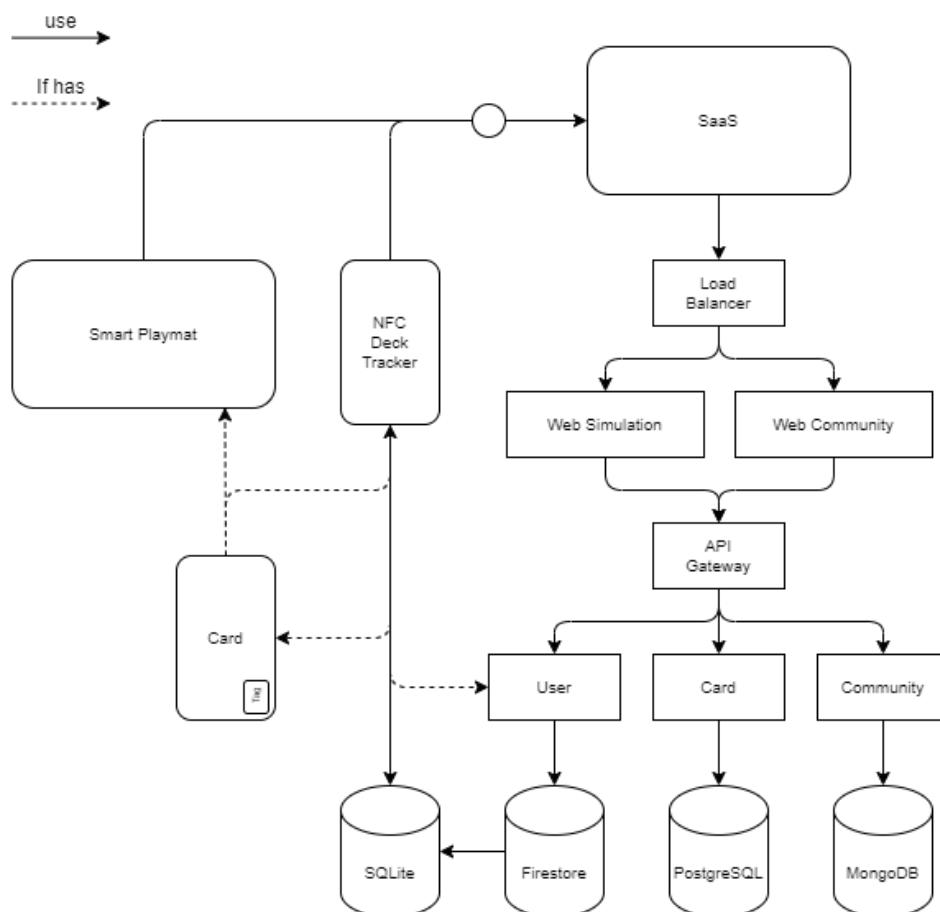
จากข้อจำกัดที่พบในระหว่างการพัฒนา และการใช้งานระบบ ผู้จัดทำจึงได้จัดทำข้อเสนอแนะไว้เพื่อใช้เป็นแนวทางสำหรับการพัฒนาต่อยอดในอนาคต ทั้งในด้านเทคโนโลยีและการออกแบบ เพื่อให้แอปพลิเคชันสามารถรองรับการใช้งานได้อย่างครอบคลุมและตอบสนองต่อความต้องการของผู้ใช้ได้ดียิ่งขึ้น ดังนี้

- ทำ Usability Test อย่างเป็นทางการ สิ่งที่ต้องทำเป็นอันดับแรกในการพัฒนาต่อยอดคือ การนำแอปพลิเคชันต้นแบบนี้ไปทดสอบกับกลุ่มผู้เล่นเกมการ์ดจริง เพื่อเก็บข้อมูลเชิงปริมาณ (เช่น เวลาที่ใช้, อัตราความสำเร็จ) และข้อมูลเชิงคุณภาพ มาใช้ในการปรับปรุงการออกแบบ UI ให้ตอบสนองต่อผู้ได้ดียิ่งขึ้น
- ข้อจำกัดด้านฮาร์ดแวร์ ควรมีการระบุข้อกำหนดของอุปกรณ์ที่เหมาะสมในการใช้งานแอปพลิเคชันไว้อย่างชัดเจนในหน้ารายละเอียดของ Store เพื่อให้ผู้ใช้งานสามารถตรวจสอบได้ว่าอุปกรณ์ของตนรองรับการใช้งานเทคโนโลยี NFC หรือไม่ อาจตั้งคำถามให้ดาวน์โหลดได้เฉพาะในอุปกรณ์ที่รองรับเท่านั้นเพื่อป้องกันการใช้งานที่ไม่สมบูรณ์
- ข้อจำกัดของแท็ก NFC มีควรเลือกใช้แท็กรุ่นที่รองรับการเขียนซ้ำ เช่น Ntag213 ซึ่งมีความทนทานและเหมาะกับการใช้งานร่วมกับซองการ์ด นอกจากนี้ ควรพัฒนาระบบแจ้งเตือนสถานะของการอ่าน/เขียนแท็กแบบ Real-time เช่น ข้อความเตือนเมื่ออ่านไม่ได้ แท็กไม่รองรับ หรือข้อมูลผิดพลาด เพื่อให้ผู้ใช้งานสามารถรับทราบและดำเนินการแก้ไขได้อย่างทันที
- ข้อจำกัดจากการพึ่งพา API ภายนอก ควรพิจารณาการพัฒนา API สำหรับใช้งานภายในโครงการเอง เพื่อลดการพึ่งพาผู้ให้บริการภายนอกซึ่งอาจมีการปิดบริการหรือเปลี่ยนแปลงข้อมูลโดยไม่คาดคิด การมี API ของตนเองช่วยให้ระบบมีเสถียรภาพและสามารถนำไปใช้งานร่วมกับแอปพลิเคชันหรือระบบอื่นที่เกี่ยวข้องได้อย่างยืดหยุ่น
- ข้อจำกัดของการใช้งานระหว่างการแข่งขัน เนื่องจากแอปพลิเคชันได้รับการออกแบบมาเพื่อการใช้งานทั่วไปหรือการเล่นกับเพื่อนเป็นหลัก ในอนาคตหากมีแนวโน้มว่าจะนำไปใช้ในการแข่งขันอย่างเป็นทางการ ควรพิจารณาจัดทำแนวทางร่วมกับผู้จัดงานหรือสมาคมที่เกี่ยวข้อง เพื่อสร้างมาตรฐานที่เหมาะสม ยุติธรรม และสอดคล้องกับการแข่งขัน

แผนการพัฒนาในอนาคตและแนวทางการต่อยอดระบบ

จากแนวคิดเบื้องต้นของการใช้แอปพลิเคชันเพื่อติดตาม และจัดการเต็คของเกมการ์ด ผู้พัฒนาเล็งเห็นว่าแนวทางนี้สามารถต่อยอดออกไปได้หลากหลายรูปแบบ โดยไม่จำกัดเฉพาะการใช้งานส่วนบุคคลหรือกับเพื่อน แต่ยังสามารถพัฒนาเป็นบริการรูปแบบใหม่ที่ยกระดับประสบการณ์ของผู้เล่น ตลอดจนรองรับการใช้งานในระดับเชิงพาณิชย์หรือชุมชนขนาดใหญ่ได้ในอนาคต (รูปที่ 6.12) เช่น

- Smart Playmat แผ่นรองเล่นอัจฉริยะที่สามารถแสดงสถานะการเล่นแบบเรียลไทม์ เสมือนเล่นการ์ดเกมบนกระดานดิจิทัล โดยยังคงอรรถรสแบบเดิมของเกมการ์ดแต่เพิ่มความสามารถในการเล่นแบบออนไลน์ผ่านระบบ
- SaaS (Software as a Service) ระบบที่ให้บริการจัดการเต็คและติดตามสถานะผ่านระบบคลาวด์ รองรับผู้ใช้งานจำนวนมาก เหมาะสำหรับร้านเกม ชุมชน หรือกิจกรรมที่มีผู้เล่นหลายกลุ่มใช้งานพร้อมกัน
- TCG Streaming & Replay Analytics แพลตฟอร์มสตรีมมิ่งสำหรับเกมการ์ดโดยเฉพาะ ซึ่งอาจใช้แสดงการแข่งขันหรือกิจกรรมของผู้เล่น พร้อมระบบวิเคราะห์การเล่นย้อนหลังทั้งในรูปแบบวิดีโอและข้อมูลเชิงสถิติ
- Web Simulation เว็บไซต์หรือแอปที่ให้ผู้เล่นสามารถทดลองจัดเต็ค ทดสอบกลยุทธ์ หรือฝึกเล่นกับ AI โดยอ้างอิงจากข้อมูลการ์ดจริงจากในระบบ
- Web Community พื้นที่สำหรับผู้เล่นในการแลกเปลี่ยนประสบการณ์ แชร์เต็ค รีวิวการ์ด หรือติดตามข่าวสารของเกมในรูปแบบโซเชียลเน็ตเวิร์ก



รูปที่ 6.12 แผนภาพแสดงสถาปัตยกรรมของระบบที่สามารถต่อยอดได้ในอนาคต

บรรณานุกรม

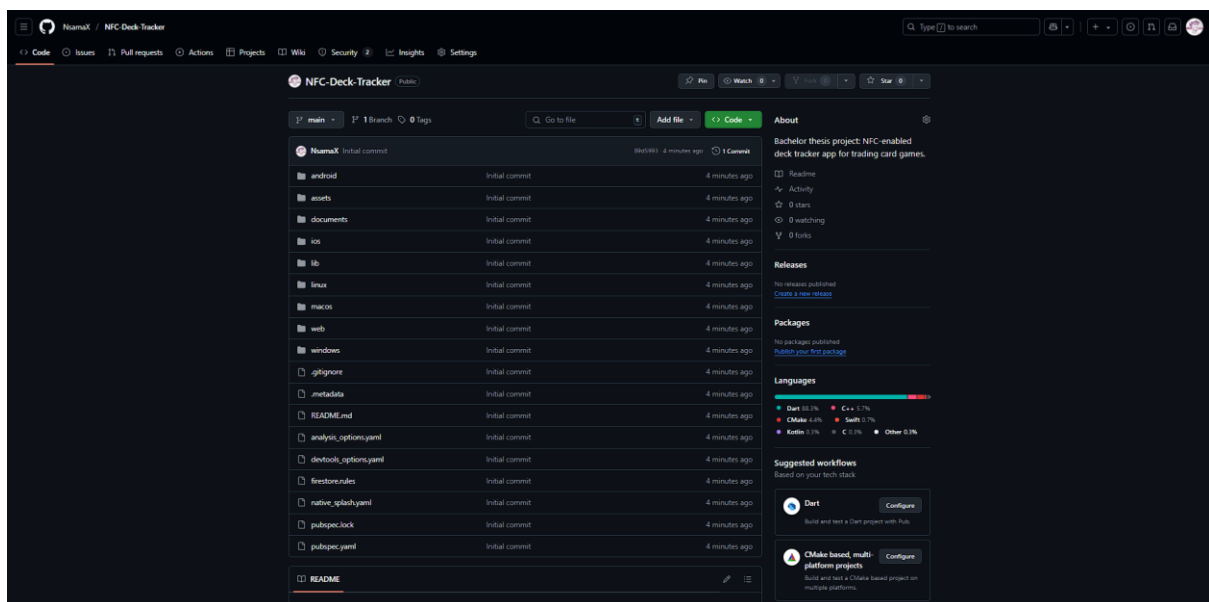
- [1] u/Pleasant-World-8751, "Multiple posts on deck tracking research using NFC in trading card games," *Reddit*, Mar. 2025. [Online]. Available: <https://www.reddit.com/user/Pleasant-World-8751>
- [2] ARCANÉ TINMEN APS, *BigAR - Vanguard Card Scanner (Version 3.0)* [Mobile application], App Store, 2024. [Online]. Available: <https://apps.apple.com/th/app/bigar-vanguard-card-scanner/id1221964973>
- [3] HSReplay.net, *Hearthstone Deck Tracker* [Software], HSReplay.net, 2024. [Online]. Available: <https://hsreplay.net/>
- [4] Wakdev, *NFC Tools (Version 2.31)* [Mobile application], App Store, 2024. [Online]. Available: <https://apps.apple.com/us/app/nfc-tools/id1252962749>
- [5] Wikipedia contributors, "Collectible card game," *Wikipedia, The Free Encyclopedia*, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Collectible_card_game
- [6] A. Rahul, G. G. Krishnan, H. Unnikrishnan, and S. Rao, "Near Field Communication (NFC) technology: A survey," *Int. J. Cybern. Inform.*, vol. 4, 2015. [Online]. Available: <https://doi.org/10.5121/ijci.2015.4213>
- [7] Thales Group, "ISO 14443: Contactless smart card standard," 2024. [Online]. Available: <https://www.thalesgroup.com/en/markets/digital-identity-and-security/technology/iso14443>
- [8] Freemindtronic, "NFC Data Exchange Format (NDEF)," 2022. [Online]. Available: <https://freemindtronic.com/wp-content/uploads/2022/02/NFC-Data-Exchange-Format-NDEF.pdf>
- [9] NXP Semiconductors, "MIFARE DESFire Light contactless application IC," *Data Sheet*, Rev. 3.3, Apr. 2019. [Online]. Available: <https://www.nxp.com/docs/en/data-sheet/MF2DLHX0.pdf>
- [10] Google Developers, "Flutter documentation," n.d. [Online]. Available: <https://docs.flutter.dev/>
- [11] Y. Abd, "Clean architecture in Flutter: MVVM, BLoC & Dio," *Medium*, 2023. [Online]. Available: <https://medium.com/@yamen.abd98/clean-architecture-in-flutter-mvvm-bloc-dio-79b1615530e1>
- [12] Google Developers, "Firebase Realtime Database documentation," n.d. [Online]. Available: <https://firebase.google.com/docs/database>
- [13] SQLite, "SQLite documentation," n.d. [Online]. Available: <https://www.sqlite.org/docs.html>
- [14] NXP Semiconductors, "NTAG213/215/216 - NFC Forum Type 2 Tag compliant ICs," n.d. [Online]. Available: https://www.nxp.com/products/NTAG213_215_216

ภาคผนวก ก ขั้นตอนการนำโปรเจกต์จาก GitHub มาใช้งานเพื่อพัฒนา

สำหรับผู้สนใจศึกษา หรือต้องการนำโครงการ NFC Deck Tracker ไปพัฒนาต่อยอด ผู้พัฒนายินดีแบ่งปัน source code เพื่อประโยชน์ทางการศึกษา โดยสามารถเข้าถึงได้ผ่าน GitHub Repository ที่จัดเก็บโค้ดของระบบไว้ อย่างครบถ้วน หากมีข้อสงสัยเกี่ยวกับการทำงานในแต่ละเลเยอร์ สามารถอ่านคำอธิบายได้จากไฟล์ README ที่จัดไว้ ภายในโฟลเดอร์ของเลเยอร์นั้น ๆ

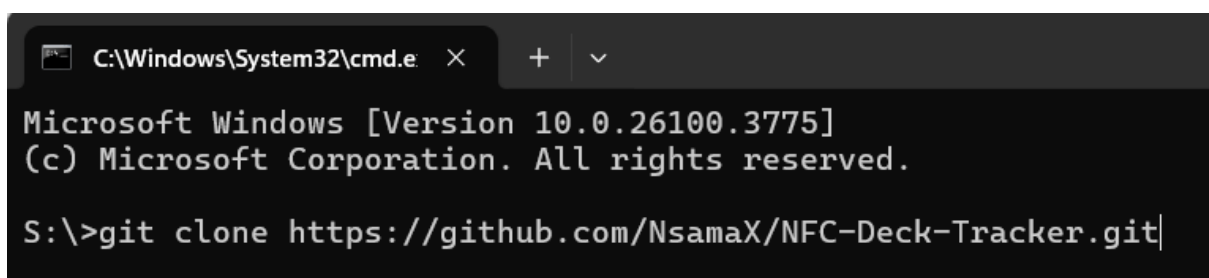
ผู้สนใจสามารถทำตามขั้นตอนต่อไปนี้:

1. เข้าถึง Repository จาก GitHub: <https://github.com/NsamaX/NFC-Deck-Tracker>



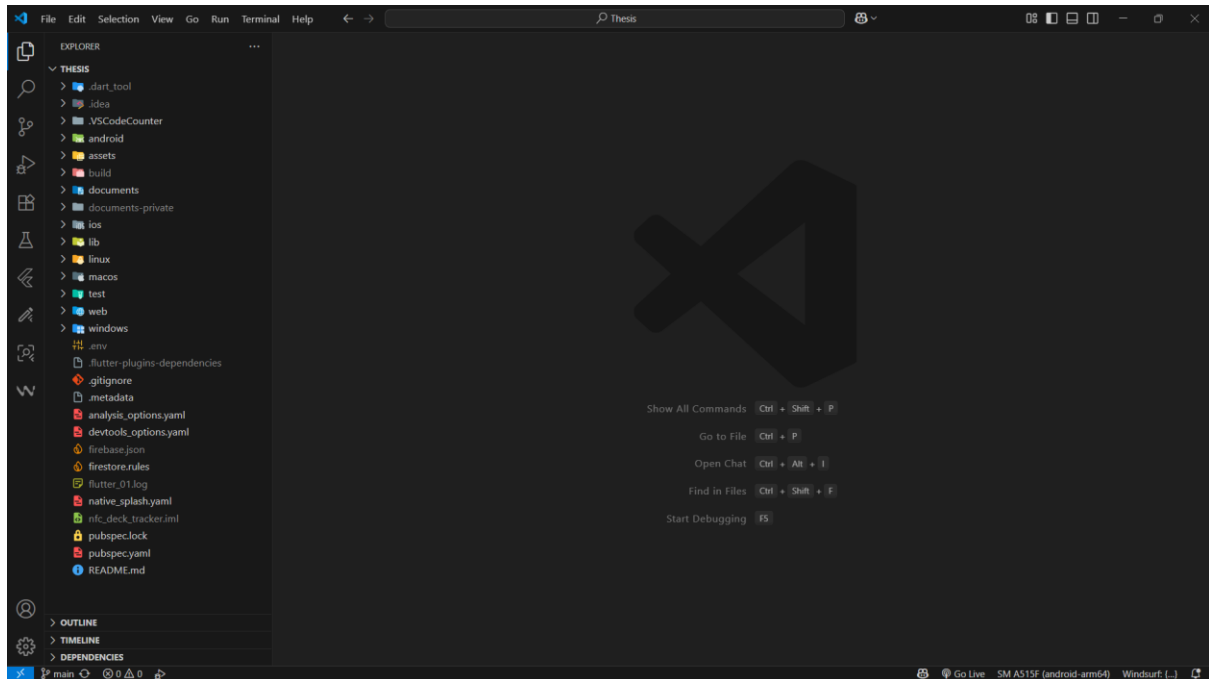
ภาคผนวก ก. 1 หน้าจอ Git Repository ของโปรเจกต์

2. Clone โปรเจกต์เข้าสู่เครื่องของตนเอง โดยเปิด Terminal หรือ Git Bash แล้วรันคำสั่ง:
`git clone https://github.com/NsamaX/NFC-Deck-Tracker.git`



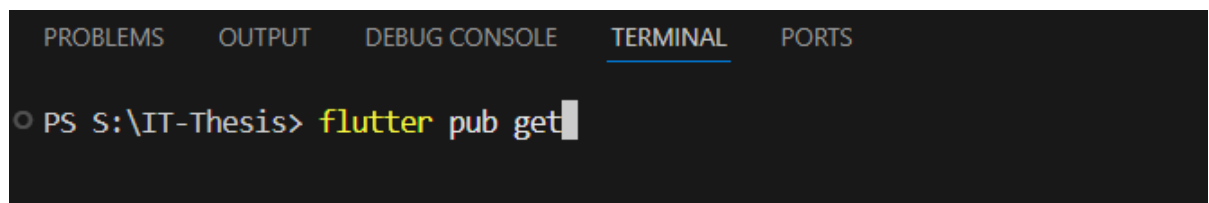
ภาคผนวก ก. 2 การใช้คำสั่ง git clone เพื่อคัดลอกโปรเจกต์

3. เปิดโปรเจกต์ในโปรแกรมพัฒนา เช่น Visual Studio Code จะแสดงโครงสร้างโปรเจกต์ดังรูป



ภาคผนวก ก. 3 โครงสร้างไดเรกทอรีของโปรเจกต์ใน Visual Studio Code

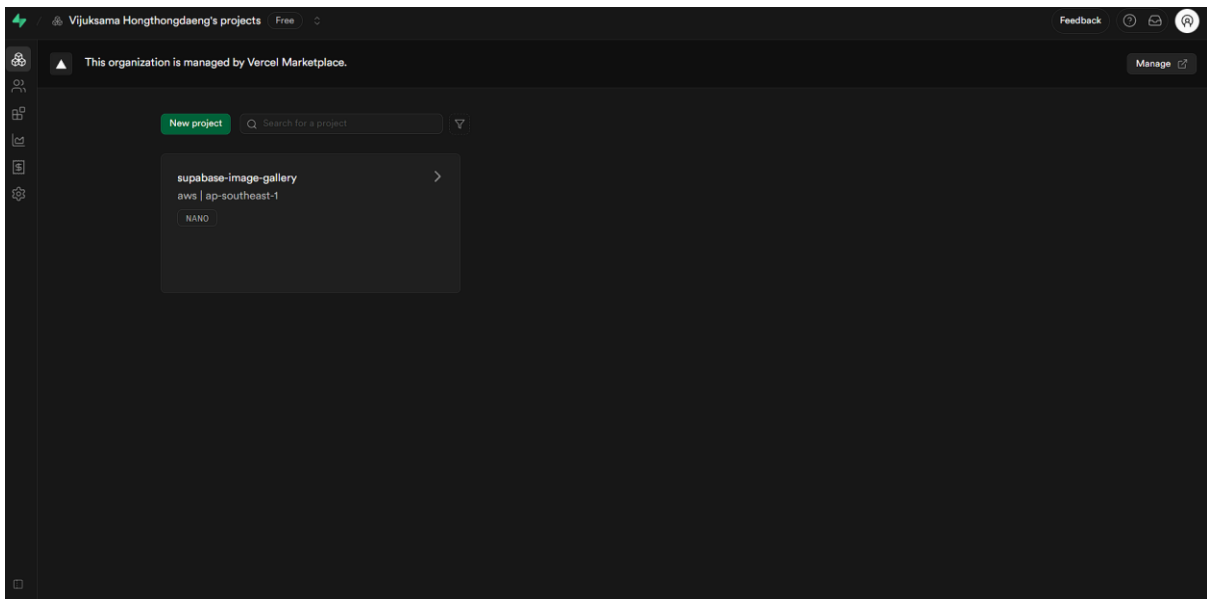
4. ติดตั้ง Dependency ที่จำเป็น โดยเปิด Terminal แล้วรันคำสั่ง: flutter pub get



ภาคผนวก ก. 4 การใช้คำสั่ง flutter pub get เพื่อติดตั้ง Dependencies

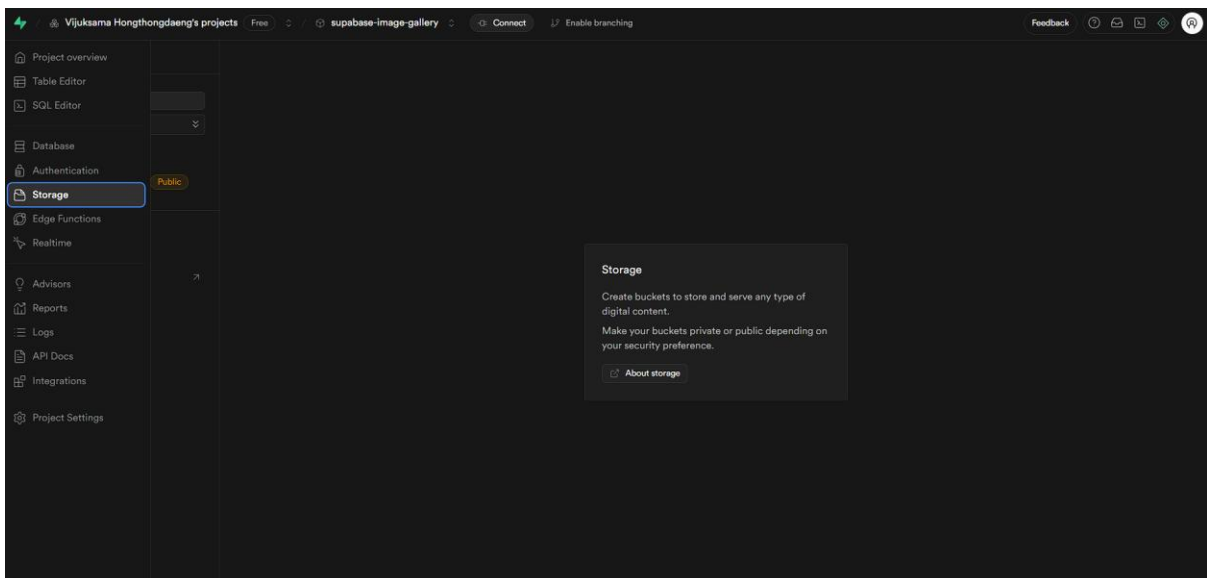
- ขั้นตอนการตั้งค่าและติดตั้ง Supabase ซึ่งเป็นส่วนสำคัญในการสร้างแบ็คเอนด์สำหรับการจัดเก็บรูปภาพ โดยมี Storage bucket ชื่อ "cards" เพื่อใช้เป็นพื้นที่จัดเก็บข้อมูลดังกล่าว

5.1 สร้างโปรเจกต์ใหม่ใน Supabase



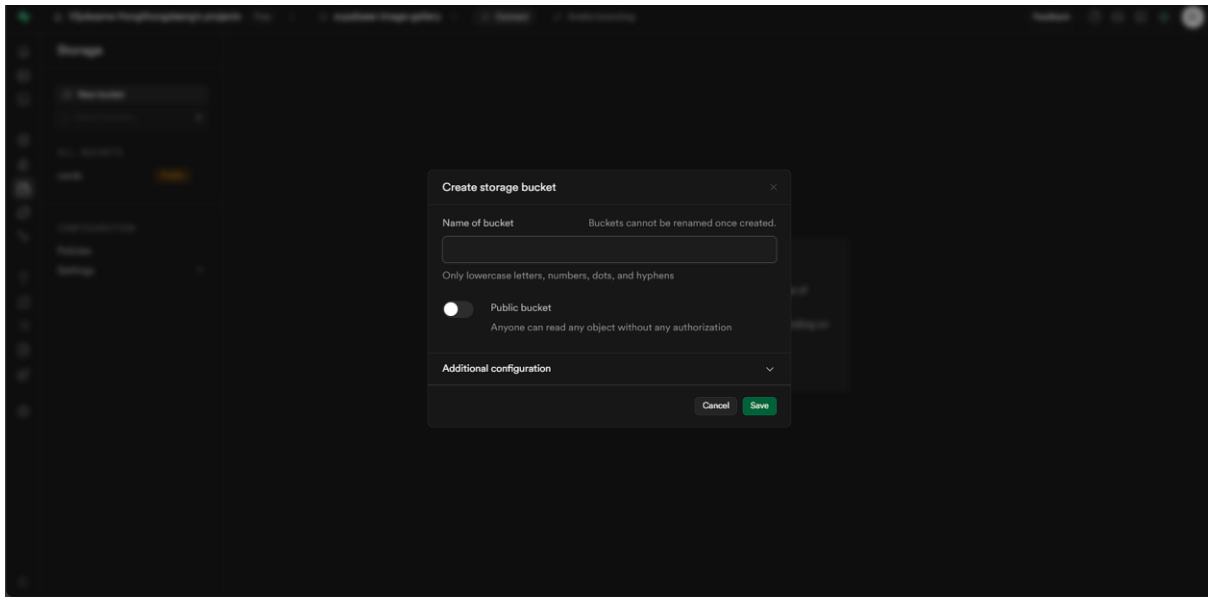
ภาคผนวก ก. 5 หน้าจอโปรเจกต์ใน Supabase Dashboard

5.2 เข้าสู่เมนู Storage



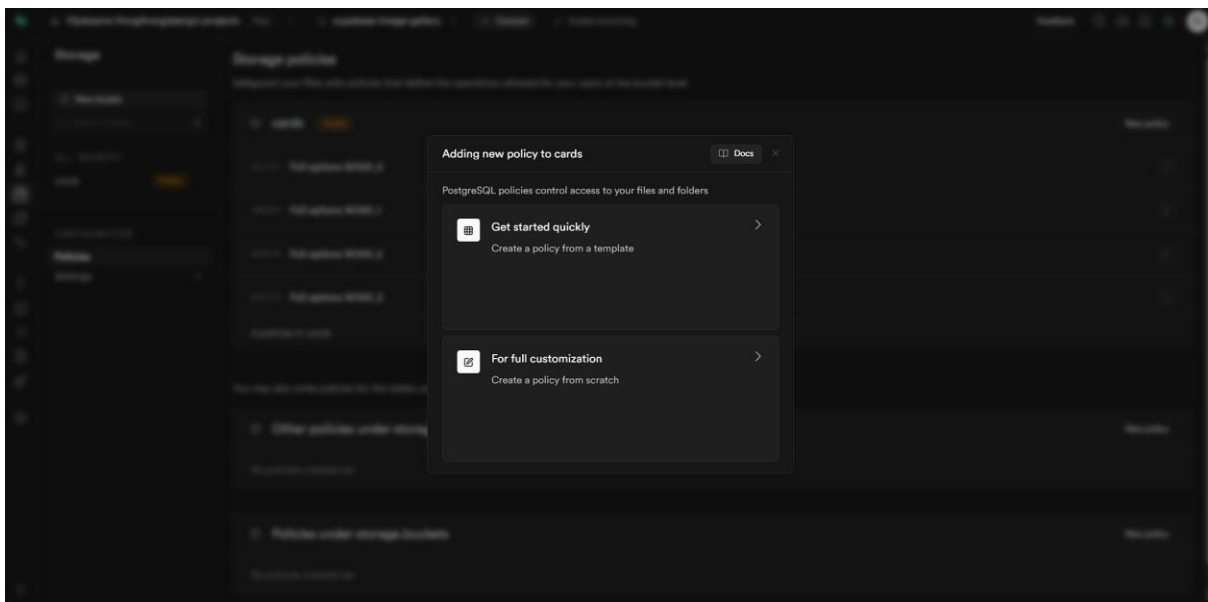
ภาคผนวก ก. 6 การเข้าสู่เมนู Storage ใน Supabase

5.3 สร้าง Bucket ชื่อ cards และกำหนดเป็น Public



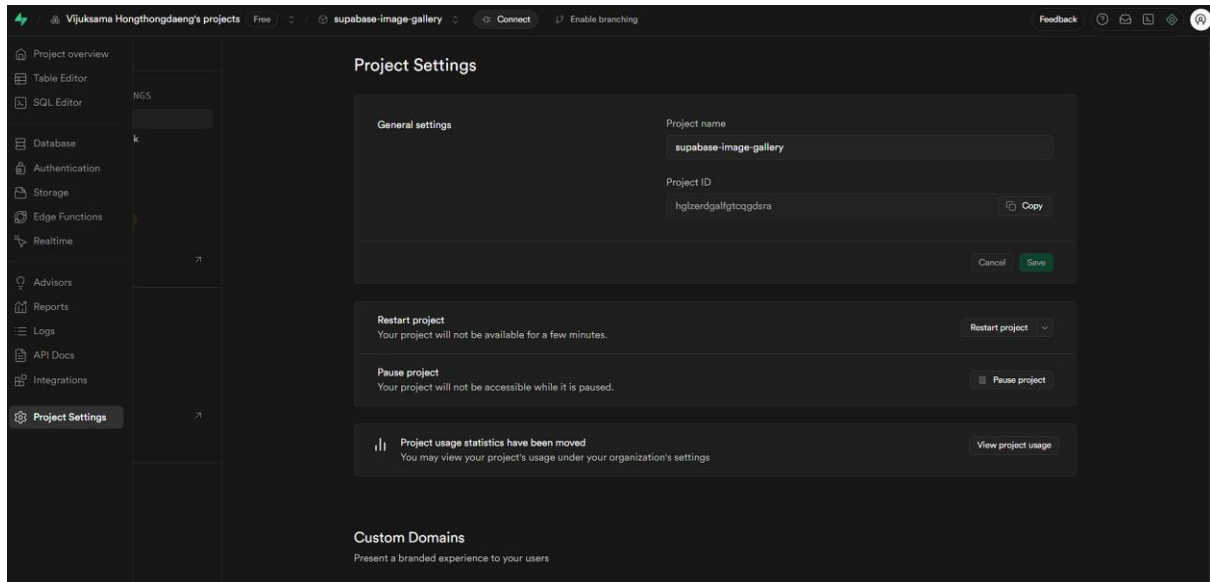
ภาคผนวก ก. 7 หน้าต่างสำหรับสร้าง Storage Bucket ใหม่

5.4 ตั้งค่า Policy โดยเลือก For full customization กำหนดให้สามารถ SELECT INSERT UPDATE DELETE



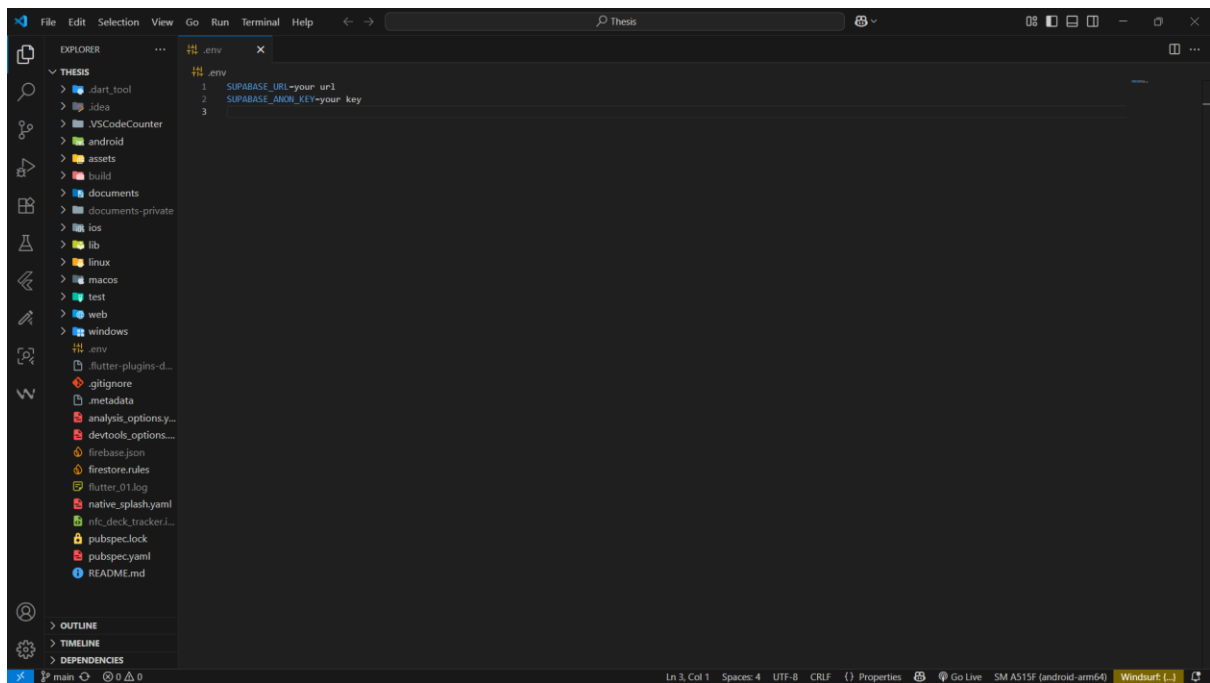
ภาคผนวก ก. 8 การกำหนด Policy สำหรับการเข้าถึงข้อมูลใน Storage

5.5 เข้าสู่เมนู Setting ไปที่ API Keys และ Data API คัดลอก anon public และ URL



ภาคผนวก ก. 9 หน้าจอการตั้งค่าโปรเจกต์ (Project Settings) ใน Supabase

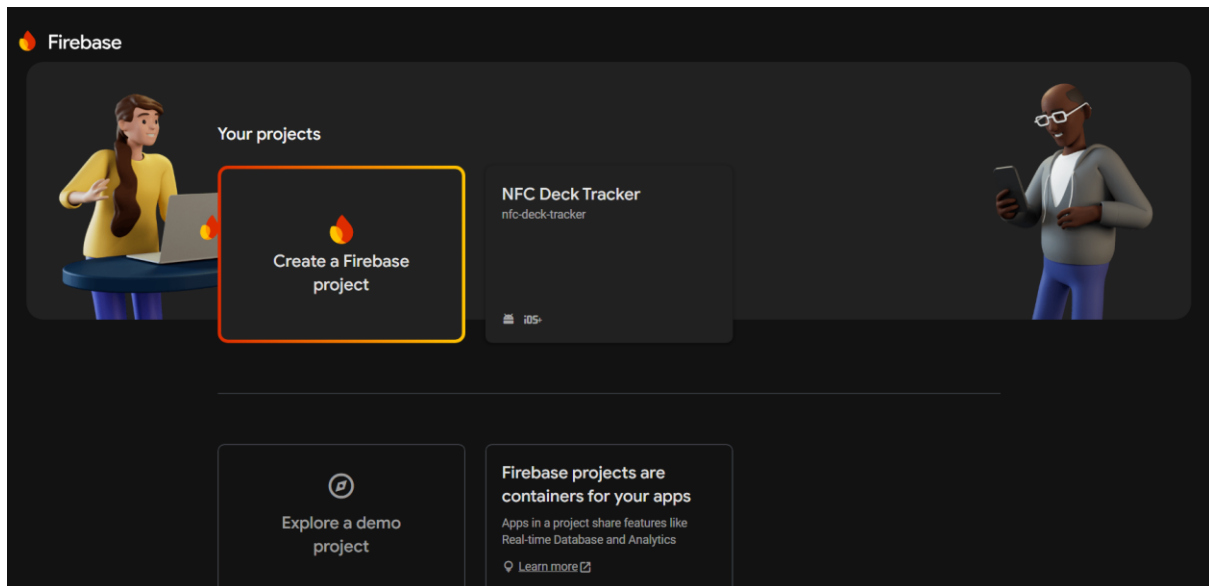
5.6 นำ Key และ URL ที่ได้มาสร้างไฟล์ .env ในโปรเจกต์ (ตามที่แสดงในภาพโครงสร้างโปรเจกต์)



ภาคผนวก ก. 10 การตั้งค่า Key และ URL ในไฟล์ .env

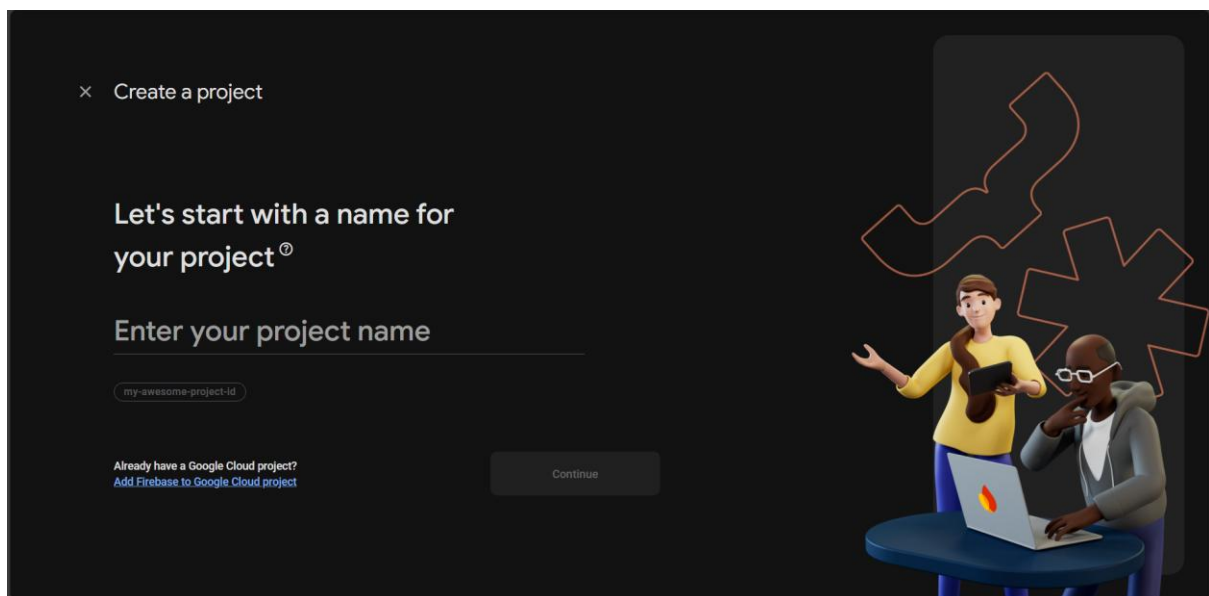
6. ขั้นตอนการตั้งค่าและติดตั้ง Firebase ซึ่งเป็นส่วนสำคัญในการสร้างแอปพลิเคชันสำหรับการจัดเก็บข้อมูลผู้ใช้ โดยมีการเชื่อมต่อ Firebase เข้ากับโปรเจกต์ Flutter

6.1 สร้างโปรเจกต์ใหม่ใน Firebase



ภาพผนวก ก. 11 หน้าจอโปรเจกต์ใน Firebase Console

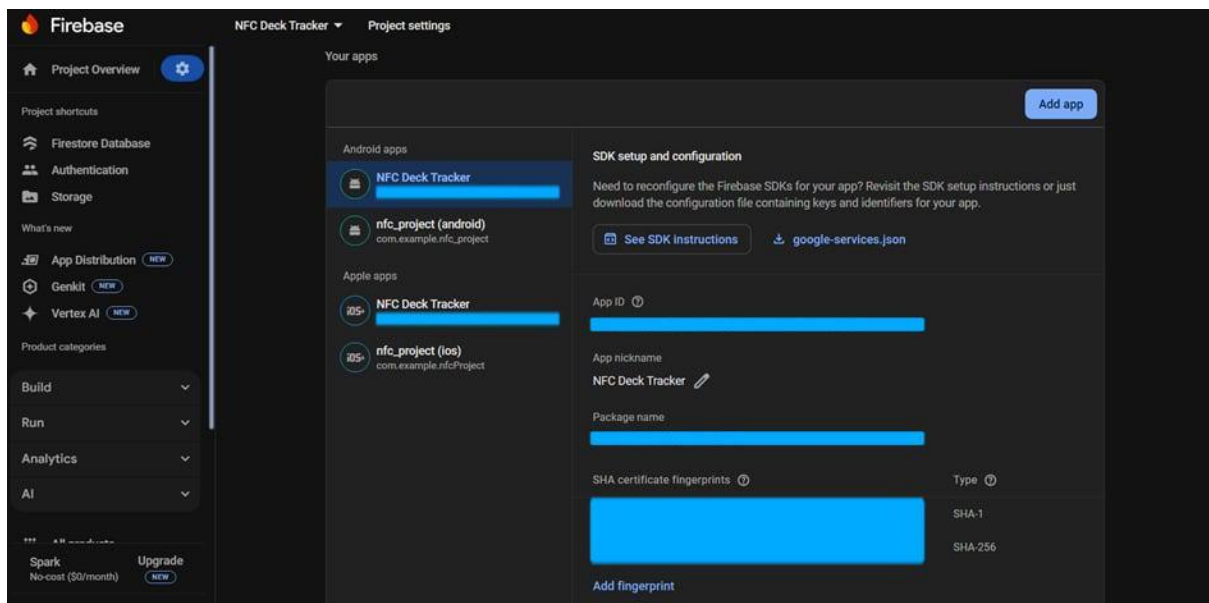
6.2 ตั้งค่าแพลตฟอร์ม Android และ/หรือ iOS ตามที่ต้องการ



ภาพผนวก ก. 12 ขั้นตอนการสร้างโปรเจกต์ใหม่ใน Firebase

6.3 ใส่ SHA-256 ที่ได้จากเครื่องแล้วดาวน์โหลดไฟล์ google-services.json (Android) หรือ GoogleService-Info.plist (iOS) แล้ววางไว้ในตำแหน่งที่เหมาะสมในโปรเจกต์

- /android/app/google-services.json
- /ios/Runner/GoogleService-Info.plist



ภาคผนวก ก. 13 หน้าจอสำหรับดาวน์โหลดไฟล์ google-services.json

6.4 ติดตั้ง Firebase CLI และใช้คำสั่ง flutterfire configure เพื่อสร้างไฟล์ firebase_options.dart สำหรับเชื่อมต่อกับ Firebase

6.5 ตรวจสอบว่าได้ติดตั้งแพ็คเกจ Firebase ที่จำเป็นผ่าน pubspec.yaml แล้วเรียกใช้งานได้ถูกต้อง

6.6 คัดลอกโค้ดทั้งหมดในไฟล์ firestore.rules ซึ่งอยู่ภายใน root directory ของโปรเจกต์ (ตามที่แสดงในภาพโครงสร้างโปรเจกต์) แล้วนำโค้ดดังกล่าวไปวางในหน้า “Rules” บน Firebase Console โดยเข้าไปที่เมนู Firestore Database > Rules การดำเนินการในขั้นตอนนี้มีวัตถุประสงค์เพื่อให้ระบบสามารถกำหนดสิทธิ์การเข้าถึงข้อมูลได้อย่างถูกต้องและปลอดภัย ตามกฎที่ผู้พัฒนาได้กำหนดไว้ในไฟล์ firestore.rules